

Reti di Calcolatori

simi

September 2021

Contents

1	Introduzione	7
1.1	Organizzazione	8
1.2	Contenuti del corso	8
1.2.1	Perche'?	8
1.3	Reti di calcolatori	8
1.3.1	Concetti fondamentali delle reti	10
1.3.2	Usi e applicazioni	11
1.4	Tassonomia delle reti: Broadcast	12
1.5	Tassonomia delle reti: Punto-Punto (Point-To-Point)	13
1.5.1	Routing	14
1.6	Infrastrutture esistenti	15
1.6.1	Tassonomia delle reti	16
1.7	Unita' di misura	17
1.7.1	Parametri caratterizzanti	17
1.8	Tassonomia dati	19
1.9	Quality-of-Service (QoS)	20
1.9.1	Throughput	21
1.10	Scambio dati	22
2	Come sono fatte	25
2.1	Servizi+Protocolli+algoritmi=architettura	26
2.2	Architetture di rete: Incapsulamento	27
2.3	Funzionalita' dei livelli	28
2.3.1	(de)Multiplexing	30
2.4	Connection-oriented service (servizio orientato alla connessione)	31
2.5	Connectionless service	31
2.5.1	Ordinamento vs Affidabilita'	32
2.6	Tecniche di commutazione	32
2.7	Commutazione in pacchetto	33
2.8	Confronto tecniche di switching	34
2.8.1	Esercizio di confronto msg vs. pkt switching	34
2.8.2	Perche' accodamento?	36
2.9	Routing a circuito virtuale	36
2.10	Routing datagram	37

2.11	Chi e' importante	38
2.12	Architetture Standard	38
2.12.1	OSI (open system interconnection)	39
2.12.2	Modello TCP/IP (Internet)	41
2.12.3	data transmission and PDU	42
2.12.4	OSI vs TCP/IP	43
2.13	Apparati di rete	44
2.14	Cenni storici	44
2.15	Accesso alla rete	45
2.15.1	Residenziale	45
2.16	Un po' di terminologia	46
2.17	Reti WIRELESS	46
2.17.1	Accesso alla rete wireless	49
2.17.2	Collegamento dispositivi	49
3	Livello Applicazione	51
3.1	Struttura applicazioni	53
3.2	Architettura generale	53
3.2.1	Indirizzamento e (de)Multiplexing	54
3.2.2	Primitive di servizio (delle socket)	55
3.3	Architettura client-server	58
3.4	Architetture Peer-to-Peer	59
3.5	Servizi a livello 7	59
3.5.1	Condivisione di file	59
3.6	WWW	60
3.7	HTTP: architettura	60
3.7.1	Funzionamento	62
3.7.2	Formato messaggi richiesta	63
3.7.3	Formato risposta	64
3.7.4	HTTP: full request / response	65
3.7.5	Trasferimento contenuti	66
3.7.6	HTTP discussione	66
3.7.7	Contenuti 2	67
3.7.8	HTTP: cookies	67
3.7.9	HTTP:caching	68
3.8	Electroinc Mail	70
3.8.1	E-mail: funzionalita'	71
3.8.2	Simple Mail Transfer Protocol (SMTP)	71
3.8.3	Schema scambio email	71
3.8.4	Struttura della mail	72
3.8.5	SMTP	72
3.8.6	Mail access protocols	74
3.8.7	MIME	75
3.8.8	Tipi MIME	76
3.8.9	Esempio MIME	76
3.8.10	email gateway	77

3.9	FTP	78
3.9.1	Control data connections	79
3.9.2	FTP protocol	80
3.9.3	FTP comandi e risposte	81
3.9.4	FTP vs HTTP vs SMTP	81
3.10	TELENET	81
3.11	Secure Socket Layer	82
3.12	Domain Name System (DNS)	82
3.12.1	Gerarchia dei nomi simbolici	83
3.12.2	Assegnazione nomi simbolici	84
3.12.3	Autorita'	84
3.12.4	root server	85
3.12.5	Gerarchia name server	85
3.12.6	DNS resource record	86
3.12.7	DNS query/reply	88
3.12.8	Risoluzione Locale	89
3.12.9	Risoluzione ricorsiva pura	89
3.12.10	Soluzione iterativa pura	91
3.12.11	Risoluzione inversa	92
3.12.12	Osservazioni	93
3.12.13	Dynamic DNS	94
4	Livello di Trasporto	95
4.1	Gestione degli errori	96
4.2	Protocolli connection oriented	97
4.3	Very silly protocol	98
4.4	Silly protocol	100
4.5	Positive Acknowledgment with Retransmission (PAR)	100
4.5.1	PAR:ack numerati	103
4.5.2	ildeRQ (Stop & Wait) - sender	104
4.5.3	idleRQ - reciver	105
4.5.4	Situazioni di errore	107
4.5.5	Utilizzo canale	107
4.6	Sliding window	108
4.6.1	Esempio sliding window 1-bit	109
4.6.2	Codice	110
4.7	Continuous RQ	111
4.7.1	Dettagli implementativi	112
4.7.2	Schema generale	114
4.8	Go Back N	115
4.8.1	Messaggio corrotto	117
4.8.2	ACK corrotto	118
4.9	Selective repeat	118
4.9.1	Selective repeat: malfunzionamenti	120
4.10	Range numeri sequenza	122
4.11	Utilizzo del canale	124

4.12	Controllo di flusso	124
5	TCP	125
5.1	TCP, Transport Control Protocol	126
5.2	Formato del segmento	129
5.2.1	TCP - code bit e pseudo header	130
5.3	Apertura connessione	132
5.3.1	Three-way handshake	132
5.4	Es. sessione TCP (tcpdump)	135
5.5	Trasferimento dati	135
5.6	Errore di scambio dati	136
5.6.1	Timer ritrasmissione	136
5.6.2	Controllo di congestione	137
5.7	Chiusura della connessione	138
5.7.1	Chiusura simmetrica	139
5.7.2	Chiusura asimmetrica	139
5.7.3	Rilascio connessione	139
5.8	Altre caratteristiche	140
5.9	UDP - User Datagram Protocol	141
5.9.1	Header UDP	142
6	Livello Rete	143
6.1	Gateway	145
6.2	Internet Protocol (IP)	146
6.2.1	Header IPv4	147
6.3	Fragmentation	149
6.4	IP addressing	150
6.4.1	Class-based addressing	151
6.4.2	Subnetting	153
6.4.3	Classless Inter-Domain Routing	153
6.4.4	CIDR e routing	154
6.5	IP routing	155
6.6	Principio di ottimalità	156
6.6.1	Rooting Statico e Flooding	157
6.6.2	Distance Vector, Bellman-Ford	158
6.6.3	Approccio Link State	162
6.6.4	Distance Vector vs Link State	164
6.7	Internet Structure	165
6.8	ICMP - Internet Control Message Protocol	165
6.8.1	Header ICMP	167
6.9	DHCP - Dynamic Host Configuration Protocol	168
6.10	Routing in Internet	168
6.10.1	OSPF	169
6.11	BGP - Border Gateway Protocol	171
6.12	Summary algoritmi di instradamento	172

Chapter 1

Introduzione

1.1 Organizzazione

Inizio lezioni h.15.45 (ci prendiamo il quarto d'ora accademico) finendo alle 17.15 senza pause. L'esame sara' uno scritto su tutto il programma + degli esercizi. C'e' una dispensa con gli esercizi svolti che mette negli scritti, sono in corsivo perche' dobbiamo saperli fare. Libro: "Reti di calcolatori un approccio top-down" Behrouz A. Forouzan and Firouz Mosharraf.

1.2 Contenuti del corso

Si parte dall'hardware fino all'utente. Si basa sul principio del "Divide ed impera", abbiamo un problema complicato da risolvere e lo spezzettiamo in sottoproblemi piu' semplici e poi ricomponiamo le sottosoluzioni per arrivare alla soluzione del problema iniziale grosso.

- Nozioni di base; architetture di rete; TCP/IP
- Livello di applicazioni: HTTP, posta elettronica, DNS (e' una cosa che traduce dal nome simbolico in un numero associato per identificare uno specifico server)
- Livello di trasporto: TCP, UDP
- Livello di network: protocolli IPv4, ICMP, NAT, DHCP
 - Algoritmi di instradamento in Internet
- Tecnologie wireless, applicazioni mobile (sono interessanti per chi fa mobile)
- Programmazione di applicazioni distribuite (socket)

E' un approccio top-down, ovvero andremo a trattare prima le applicazioni che usiamo tutti i giorni fino ad arrivare alla base. Come se partissimo dal tetto di una casa arrivando fino alle fondamenta.

1.2.1 Perche'?

Perche' ormai non si puo' prescindere dall'uso della rete, come ci ha mostrato la pandemia, infatti internet e' molto usato. Andando a lavorare come informatico bisogna sapere come funzionano reti e sistemi operativi.

1.3 Reti di calcolatori

E' importante imparare un linguaggio tecnico formale in quanto non devono crearsi incomprensioni durante la comunicazione tecnica. E' una collezione interconnessa di calcolatori autonomi, devono essere collegati con un mezzo

trasmissivo come un cavo di rete o in modo virtuale con delle tecnologie wireless; sono tutti calcolatori e sono autonomi perche' ognuno svolge funzioni differenti tra loro ed ognuno ha diverse memorie, sistemi operativi, CPU differenti con potenze differenti ed architetture differenti:

- **eterogeneita'** hw, sistemi operativi. Questa cosa implica un primo problema che e' quello del tempo, se chiedo attraverso la rete ad un computer remoto di fare qualcosa non posso assumere che venga svolto immediatamente ne entro un tempo massimo limitato (accesso ad un sito lento ad esempio), il traffico di internet e' diviso secondo la regola dell'80/20, ovvero l'80% delle ricerche va a cercare il 20% dei contenuti chiamati hotspot, che dal punto di vista della rete sono difficili da gestire perche' se ho tante richieste sulla stessa cosa vado in difficolta' per rispondere a tutti quanti.
- Scambio informazioni attraverso **media** (doppino, fibra, rete, telefonia cellulare, satellite...). Tratte di mezzo trasmissivo. Il fatto che ci siano diversi media impatta sul funzionamento della rete, mezzi diversi scambiano informazioni in modo diverso, i cablaggi proteggono da interferenze esterne, non e' la stessa cosa per tecnologie wireless (anche un forno a microonde puo' creare interferenze con le onde wireless).
- Esistenza macchine remote **esplicita**. Quando contatto un server lo dico esplicitamente quale server e' (ariel.unimi.it), su torrent ad esempio non so dove si trovano i contenuti (sistema distribuito) al contrario in una rete devo specificare dove si trova il contenuto che voglio prendere.
- **Sistema distribuito** (FB): l'esistenza di molteplici calcolatori e' trasparente, e' un livello di astrazione superiore alla rete, non sappiamo dove sono i nostri contenuti, tendenzialmente non sono in un posto solo perche' vengono replicati su piu' di un server. Questo serve per ovviare anche al problema del tempo, avendo i dati su server geograficamente piu' vicini a chi fruisce i nostri contenuti sono da lui raggiungibili piu' velocemente.

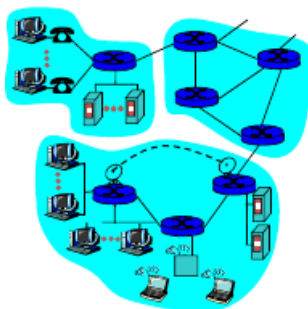


Figure 1.1: Reti

Nell'immagine abbiamo computer, router (apparati di rete, non ci sono utenti ma sono calcolatori specializzati per fare andare la rete) e server. Sono

collegati via fili (i tratti continui) ed anche tramite tecnologie wireless diverse come ci dicono le antenne che sono pc portatili che si connettono tramite un access point e wifi, inoltre ci sono anche reti satellitari (le parabole) tra terra e satellite c'è un satellite che può prendere segnali da terra e rimandarli creando un canale di comunicazione tra i due router. Una connessione tra vari computer si chiama HOP, la somma degli HOP calcola la distanza tra contenuti (in un server) e la richiesta effettuata. Da satellite a satellite sono 2 Hop, perché va nello spazio il segnale.

1.3.1 Concetti fondamentali delle reti

Chiamiamo **Host o end-system** gli apparati su cui gli utenti lavorano e su cui risiedono informazioni che gli utenti utilizzano (server, laptop, workstation, tablet, smartphone, tostapane), sono calcolatori general purpose (perché non sono specializzati) sono la parte periferica della rete dove ci sono gli utenti, sono oggetti che eseguono operazioni di rete (hanno bisogno che ci sia la rete). **Canali di comunicazione** che sono la fibra, il doppino, la radio, i satelliti. **Routers** sono calcolatori che non eseguono operazioni ma sono specializzati per far funzionare la rete (router, switch, hub ...), ad esempio trovano gli indirizzi dove cercare le informazioni, fanno comunicare gli host che cercano un contenuto con un host che possiede il contenuto.

Regional ISP : Internet Service Provider che fornisce servizio a solo strumenti di rete.

Le righe sono i canali di comunicazione che possiamo utilizzare, questi canali possono essere visti come tubi attraverso cui i dispositivi infilano dati e sono capaci di portarli dall'altra parte nell'unità di tempo. Tubi con diametro più o meno grandi in cui passano più o meno dati al secondo.

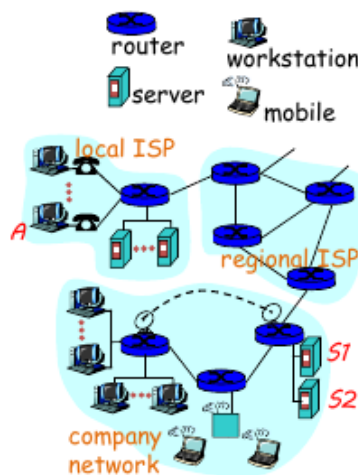


Figure 1.2: Concetti fondamentali delle reti

1.3.2 Usi e applicazioni

Cosa vogliamo fare con la rete determina i servizi che dobbiamo fornire alle applicazioni:

- **Comunicazione:** e-mail (non c'è problema di banda e di sincronizzazione, perché sono elementi asincroni, può essere un file di pochi caratteri così come può essere un file molto grosso condito di file multimediali), voce over IP (skype): è un esempio di parecchie applicazioni che possono essere utilizzate per fare trasferimento di voce sulla rete che è un cambio del paradigma della rete in quanto prima si usava la rete telefonica per metterci sopra anche dei dati la caratteristica è che voce può essere trasmessa con poche quantità di dati in questo caso c'è il problema del sincronismo in quanto ha bisogno di poco tempo per portare informazioni da sorgente a destinazione; social network è tanto asincrono quanto l'email può imporre dei vincoli sulla rete nel momento in cui il contenuto è un video e dev'essere mandato e ricevuto con continuità (altrimenti rotellina di caricamento e madonne volanti); videoconferenza è un po' di più del voice over ip perché c'è audio e video insieme, se l'audio può essere codificato con pochi byte per secondo il video dipende da cosa sto trasmettendo quindi abbiamo bisogno di molto spazio nei canali di comunicazione. Possiamo scambiare informazione tra due persone o tra più persone.
- **Condivisione risorse:** DropBox, Google docs. Con DropBox c'è un problema di concorrenza in quanto possiamo aprire un documento in più utenti e lavorarci contemporaneamente il problema è che se lavoriamo sulla stessa parte abbiamo un problema di gestione della concorrenza tra più utenti che accedono contemporaneamente allo stesso dato e di consistenza di documento. La consistenza implica che dobbiamo lasciare l'elemento in una situazione che ha senso rispetto alle operazioni fatte, se esiste un ordinamento delle operazioni mantiene l'ultima modifica fatta in ordine temporale. Se sono veramente concorrenti il sistema deve trovare un ordinamento casuale ma le cose non devono essere fatte a meta'. Si possono fare cose interessanti:
 - lavoro cooperativo, condivisione informazioni: web, cloud...
 - modello **client-server** vs **peer-to-peer (P2P)**; ci sono due paradigmi che vengono usati in rete (questi due). Client-Server è il primo paradigma che è stato realizzato che risale al tempo in cui i pc a disposizione degli utenti doveva chiedere aiuto per accedere a dati o a compiere operazioni ad un server, il quale era un computer più potente. È uno schema classico che usiamo ancora oggi, ad esempio nella gestione delle mail o navighiamo nel web accediamo a server che ci danno contenuti (ariel). Gli avanzamenti della tecnologia hanno permesso ai client di avere capacità di computazione paragonabili ai server, quindi anche loro possono ricordarsi dati e fare cose comportandosi similmente ai server, ci si è spostati verso

il paradigma peer-to-peer (P2P) la cui idea alla base e' che tutti i computer sono considerati pari, possono quindi chiedere servizi agli altri (client) sia erogare servizi agli altri (server) tante' vero che nei torrent (emule) possiamo mettere a disposizione contenuti sul nostro disco e quando scarichiamo qualcosa lo mettiamo a disposizione degli altri sul servizio cosi' facendo l'architettura diventa piatta

Le reti possono essere usate per **alta affidabilita'**, ovvero se mi danno un servizio posso fare affidamento che quel servizio esiste ed io posso richiamarlo, non e' trivialmente vero perche' esistono pc, dispositivi di rete (infrastrutture) che possono essere spenti oppure possono essere irraggiungibili. Quando si guastano cavi o router puo' capitare che la rete venga partizionata ed alcune zone risultano isolate e fuori dalla rete. Se qualcosa non e' raggiungibile e vogliamo che il servizio sia affidabile dobbiamo replicare il servizio (o dati) quindi li manteniamo su piu' end-system sulla rete. Ci sono anche applicazioni che usano la rete per **alta potenza di calcolo a basso costo**: una volta c'erano delle architetture massivamente parallele usate per grafica con enormi problemi di progettazione ad un certo punto sono state sostituite in parte dalle reti: quindi avendo tanti calcolatori sparsi in giro posso usarli per fare calcoli veramente grossi ad esempio i broker usano applicazioni (pericolosissime) e degli algoritmi che macinano enormi quantita' di dati su storico di titoli o vari comparti di titoli per dare suggerimenti ai broker stessi su cosa vendere ed acquistare, questo deve essere fatto in secondi, oppure le previsioni del tempo perche' sfruttano un ramo della fisica che e' difficile (fisica dei fluidi). Le CPU lavorano in modo parallelo sulla rete ma non in modo massivamente parallelo. Simile alle monete elettroniche (per validare transazioni elettroniche) c'e' qualcosa di malizioso perche' c'e' chi potrebbe validare transazioni invalide che permettano di spendere la stessa moneta k volte, succedono cose tipo che vengono installati software che usano la potenza computazionale del nostro computer e queste persone migrano verso i posti in cui l'energia elettrica costa meno. Ci sono molti problemi che le reti devono risolvere: problemi di sicurezza (i dati devono essere sicuri e dobbiamo permettere l'accesso solo agli utenti giusti), controllo accessi, integrita' dati (dobbiamo essere certi che nessun'entita' connessa in rete deve modificare i dati), qualita' del servizio (QoS).

1.4 Tassonomia delle reti: Broadcast

La tassonomia distingue le reti in due in base a come e' fatta la topologia della rete. Le reti broadcast sono computer che attraverso piccole tratte di cavo vanno ad accedere allo stesso cavo condiviso (senza dispositivi di rete nel mezzo, vampire tap per la connessione), la caratteristica e' che se un end-system manda un segnale sul cavo condiviso tutti i computer collegati a quel cavo tirano fuori dal cavo il segnale e quindi tutti loro possono vedere quel segnale. Sono reti broadcast per analogia con il termine broadcasting usato dalla televisioni. Potenzialmente il segnale si propaga sul cavo condiviso ed e' visibile da tutti,

1.5. TASSONOMIA DELLE RETI: PUNTO-PUNTO (POINT-TO-POINT)¹³

ma non significa che tutti lo ricevono in quanto solo l'end-system con l'indirizzo corretto "apre" i dati, tutto fatto a livello hardware. L'indirizzo di destinazione puo' essere:

- Broadcast: quindi il segnale e' veramente per tutti
- Unicast: i dati sono condivisi con tutti ma solo una stazione puo' aprire i dati, un solo destinatario
- Multicast: posso dire che un informazione va ad un gruppo di destinatari e quindi puo' essere visto da tutti loro. Non si specificano tutti gli indirizzi dei gruppi, ma metto come indirizzo l'indirizzo del gruppo

Le reti broadcast ci danno un problema, se due stazioni parlano contemporaneamente allora i loro segnali si propagano contemporaneamente sul cavo condiviso succede che ci sono interferenze distruttive ed interferenze costruttive (perche' i segnali sono onde sinusoidali). Dobbiamo quindi impedire che due reti parlino contemporaneamente evitando la contesa in modo tale che parli un solo host alla volta. Il cavo condiviso e' obsoleto, ma la rete broadcast non e' obsoleta in quanto le reti wireless sono broadcast.

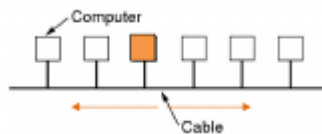


Figure 1.3: Reti Broadcast

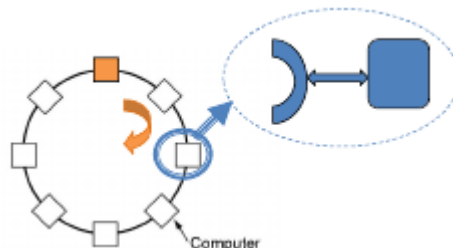


Figure 1.4: Reti Broadcast 2

1.5 Tassonomia delle reti: Punto-Punto (Point-To-Point)

Connette due apparati tra di loro (due apparati di rete o due end-system). Con topologie a stella, ad albero, a maglia completa... topologie:

- A stella: succede che se si rompe un cavo solo un dispositivo viene partizionato dalla rete, se si rompe il centro stella nessuno parla piu' con nessuno perche' e' quello che fa da smistatore e permetta agli altri di comunicare tra di loro.
- Ad anello: Se le tratte di mezzo trasmissivo sono bidirezionali l'anello potrebbe essere un po' piu' resistente perche' magari se uno dei tubi si rompe in una sola direzione possono usare l'anello per far girare i dati nell'altra direzione. Se si rompe in entrambi i lati comunque si puo' comunicare perche' rifanno il giro.
- Ad Albero: se si rompe un nodo all'interno dei sotto alberi si puo' comunicare ed analogamente se si rompe un ramo, ma ci saranno partizioni della rete.
- La maglia completa: non esiste, in quanto vorrebbe avere una coppia di mezzo trasmissivo per ogni coppia di apparati di rete o end-system. Quindi sarebbe indistruttibile. Ci permette di capire che problema dobbiamo risolvere nelle reti punto-punto.

Problema: se la tratta non e' completa dobbiamo trovare gli hop per passare da un punto ad un altro, quindi e' un problema di instradamento: dove passo? e attraverso quali dispositivi passo? ci sono dispositivi in cui i dati passano soltanto per trasportare i dati ma non li utilizzano. Il dominio di broadcast in questi casi e' la rete costituita da 2 apparati direttamente connessi da una tratta di mezzo trasmissivo.

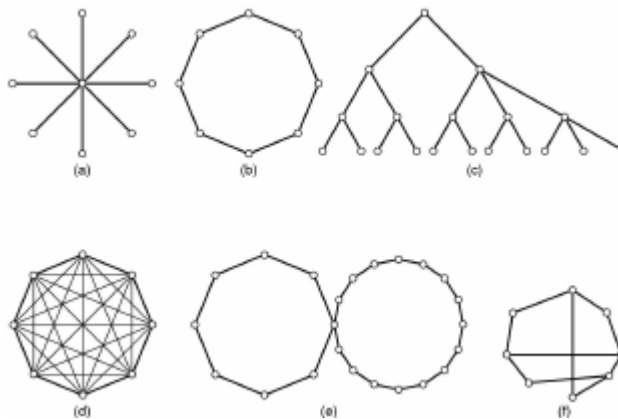


Figure 1.5: Punto-Punto

1.5.1 Routing

Routing: ricerca path ottimale. Il problema della rotta e' di trovare una rotta che e' buona secondo qualche tipo di criterio, per alcuni tipi di traffico il criterio

di cammino minore e' quello di fare meno salti possibili perche' l'attraversamento di dispositivo intermedio ci causa dei ritardi, quindi quello che succede nel nodo quando manda l'informazione deve trasformare i bit in un segnale elettromagnetico successivamente poi il segnale elettromagnetico deve essere ritrasformato in bit. Queste operazioni richiedono un certo tempo, cosi' come il tempo che occupa il dispositivo a capire se il dati sono per lui o deve trasmetterli. A volte riusciamo a far passare poca informazione per secondo, in alternativa due nodi potrebbero parlarsi lungo un cammino che e' fatto da piu' hop ma se il mezzo trasmissivo e' molto piu' veloce (linee da 100 Mbps) forse e' meglio usare il criterio di usare linee che mi danno tanta banda e poco traffico piuttosto che avere il cammino con meno hop possibile ma con banda scarsa e tanto traffico. Routing table: ricorda per ogni destinazione nella rete la migliore interfaccia di uscita. Perche' reti piccole (con pochi dispositivi) sono broadcast e reti grandi (con tanti dispositivi) sono reti p2p? Perche' le broadcast hanno un problema di collisioni, cioe' se due end-system parlano contemporaneamente i segnali collidono e si sommano, quindi se costruiamo reti grandi con tecnologia broadcast e' piu' facile che due sistemi parlino contemporaneamente ed i segnali collidano, risulta quindi difficile stabilire un turno di parola. Nelle reti p2p esiste il fatto che a differenza di quello che succede nelle broadcast, piu' comunicazioni si possono svolgere in parallelo perche' ci sono stazioni che sono collegate da mezzo trasmissivo diverso

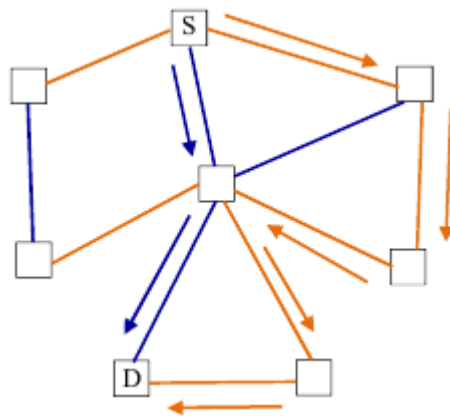


Figure 1.6: Routing

1.6 Infrastrutture esistenti

- Reti semplici: sono come quelli che abbiamo in casa (nuvola del local ISP). Noi usiamo la linea telefonica che esce da casa e va a finire in un armadietto (cabinet) di un gestore telefonico da cui poi tutto passa in fibra ottica.

- PSTN - Public Switched Telephone Network
- ISDN - Integrated Services Digital Network
- Reti aziendali: c'e' una mistura tra broadcast e punto-punto:
 - LAN - Local Area Network
 - enterprise-wide private network
 - internet = LAN conesse da backbone
- "nuvole" wireless (4G-5G, Wi-Fi, Bluetooth)

1.6.1 Tassonomia delle reti

Tipo	Taglia	C
LAN	10 m. (room) - 1 KM (campus)	unico amministratore (e' un'unica autorità), rete broadcast
intranet	campus/azienda	unico amministratore
WAN	10-100 Km (città'/nazione)	rete broadcast
internet	10-100 Km (nazione)	interconnessione di p
Internet		

Table 1.1: Caption

- LAN: Uno switch o un bridg che collegano due o piu' reti Lan le trasformano in una sola rete, al contrario se sono collegate ad un router rimangono Lan distinte ed i router pretendono che ogni loro interfaccia (nich) stia su una sua interfaccia. Una lan e' una rete piccola puo' andare da ogni pc che ci sono in una stanza e non comprende router.
- Intranet: e' ancora piccolina ed e' una rete interna ad una signola organizzazione (campus, azienda), anchessa (come il LAN) viene gestita da un'unico amministratore, pero' e' una rete punto-punto. All'interno probabilmente ci saranno porzioni punto-punto e reti broadcast.
- WAN (whide-area-network): sono un insieme di provider che coprono una regione amministrativamente significativa che tramite accordo forniscono servizi di rete.
- internet: e' piu' generico e meno amministrativo della wan. E' un insieme di reti di organizzazioni differenti, possono essere LAN, punto-punto etc...
- Internet: e' l'interconnessione di tutte le reti mondiali

Esempio intranet/internet

Anche la intranet puo' connettere diverse reti, solo che le reti della intranet sono di un unico amministratore. intranet e' un insieme di reti broadcast che sono uniti da router che collega tramite reti punto-punto altri router. Spesso hanno una struttura simile, in periferia gli host (noi) ed al centro solo apparati di rete. Nella intranet tutto e' governato da un solo gruppo di persone, in una internet no.

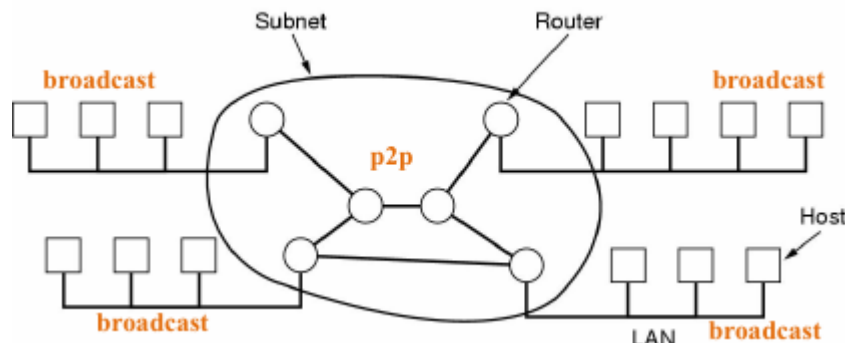


Figure 1.7: Esempio Internet

1.7 Unità di misura

- bandwidth: range di frequenze usate per la trasmissione del segnale elettromagnetico che codifica l'informazione, misurata in Hertz (Hz). Le bande sono le bande di frequenze, una radio trasmette su una determinata banda di frequenza.
- Bit rate: frequenza con cui vengono buttati i bit sul mezzo trasmissivo per unità di tempo:
 - bps (bit per secondo), Bps (byte per secondo)
 - LAN: 10/100 Mbps; wireless: 1-2 Mbps
 - cabine e cabine: terabit per secondo
 - collegamenti sottomarini atlantici: petabit per secondo (2^{15})
- BER (bit error rate): che probabilità ha un bit di essere alterato lungo il passaggio nel mezzo trasmissivo, per le cablate da 10^{-6} alla 10^{-9} su wireless siamo anche a 10^{-3} .

1.7.1 Parametri caratterizzanti

Quello che succede in un dispositivo stiamo intuendo essere: ho un calcolatore che manda un messaggio, ovvero prende i bit dell'informazione digitale e

li trasforma in un segnale elettromagnetico, non lo fa a velocita' infinita ma ci vuole tempo per passare dalla forma d'onda portante alla forma d'onda modulata correttamente. Questa grandezza si chiama BAUD rate: caratteristica dell'hardware, misurato in Hz e dice quante volte al secondo la scheda di rete riesce a cambiare la forma d'onda prodotta. Otteniamo dei fattori di ritardo:

- transmission delay T_x = quantita' di dati / bitrate. (stiamo infilando l'informazione di rete sul cavo, l'hardware (scheda di rete) deve trasformare l'informazione binaria in un segnale d'onda)
- propagation delay T_p = distanza apparati / velocita' di propagazione. La velocita' con cui l'informazione si propaga nel mezzo trasmissivo (quanto ci mette ad arrivare all'altro capo del filo), ad esempio wireless e' la velocita' della luce, nel caso di cavo in rame diventa 2/3 della velocita' della luce. m diviso m/s la risultante sono secondi (usiamo unita' di misura conformi)
- prodotto bitrate x T_p = max numero di bit che il link puo' contenere. Ci da' in qualche modo la dimensione del tubo, perche' e' come se T_p fosse la lunghezza del tubo, il bitrate e' come se fosse la sezione del tubo, quindi globalmente e' come se fosse il volume. quanto traffico ci puo' essere sopra.
- processing delay: come instrado? ogni unita' di rete deve decidere che percorso fare all'informazione che deve trasferire. E' un ritardo molto basso. Potenza della CPU dei nostri apparati di rete.
- queuing delay: ci sono dati che usano lo stesso mezzo trasmissivo, quindi puo' esserci un ritardo di accodamento. Puo' anche essere un ritardo rilevante perche' possono essere lunghe in apparati di rete critici

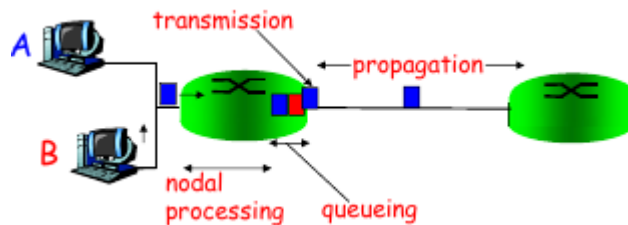


Figure 1.8: Parametri Caratterizzanti

Ritardo di accodamento

In un certo momento c'e' un'interferenza in un cavo e tutto il segnale su quel cavo viene distrutto, omogeneamente gli end-system non generano sempre informazione. Quindi il traffico in rete e' irregolare come volume e come distribuzione (hot spots), quindi e' tutto irregolare quello che dipende dal traffico e non solo dalle caratteristiche della rete. Quindi il ritardo di accodamento e' variabile in

funzione di come arrivano i dati ai dispositivi di rete. Se in un certo momento c'è un apparato di rete (router) che è bersagliato da tanto traffico che arriva quello che può succedere è che in queste code si verifichino dei buffer overflow, tutti i dati che dovrei mandare non escono e quindi qualcosa viene buttato via perché non ci sta, esaurisco la memoria. Buffer overflow significa che il dato non arriva a destinazione, può essere che sorgente e destinazione si sentano e si accordino per rimandare il dato. Questo non fa che aumentare la congestione della rete. Si può verificare un effetto valanga facendo crescere esponenzialmente il ritardo di accodamento.

1.8 Tassonomia dati

- Testo
- Immagini: rappresentazione di colore, luminosità del pixel:
 - VGA
 - SVGA
 - HD 300 PPI
- Video: sequenza di immagini (frame):
 - 4K a 60 fps sono circa 60 Mbps
 - Constant (CBR) vs Variable Bit Rate (VBR): **Constant bitrate** sono codifiche tali per cui la sorgente costantemente manda la stessa quantità di dati nell'unità di tempo (60 MPBS è una funzione che è una retta), **variable bitrate** l'idea è che non è necessario mandare costantemente tutte queste informazioni, forse si può fare qualche cosa di meglio: se io in due fotogrammi successivi ho un cambio di immagine non rilevante (i pixel rimangono uguali) per questi pixel non rimando la descrizione, ma descrivo soltanto quelli che sono cambiati e come sono cambiati (è la codifica MP4, che periodicamente mi manda un frame completo e tra un frame e l'altro ci sono dei dati di interpolazione di tipo I o B che ci dicono che pixel sono cambiati rispetto al precedente) significa che quando mando tutti i pixel ho tanti dati da inviare mentre quando mando solo quelli che sono cambiati ne ho pochi, con il variable bitrate si possono incastrare meglio le informazioni che girano per internet. Statistical multiplexing: se guardiamo mediamente cosa succede ad una sorgente quando deve mandare tanto o poco, se due medie ci stanno si possono inviare contemporaneamente i segnali di tutte e due le sorgenti, quando capita che ci sono due picchi alcuni dati vanno in buffer e vengono mantenuti dall'apparato di rete e vengono scambiati quando è libero. In questo momento le variable bitrate sono quelle preferite perché quando possono risparmiano e grazie a questo possiamo mandare più dati.

- Audio: anche con l'audio si fa qualcosa di simile al variable bitrate cioè sopprimono il silenzio, nel momento in cui una parte parla e l'altra no da quello che non parla partono 0 dati, un'alternativa che fa risparmiare sulla banda e' che se il sistema capisce che uno non sta parlando puo' mandare un confort noise (un fruscio) che da il feedback dall'altro lato che l'interlocutore c'e' ancora sia che da parte di quello che non dice niente partino pochi bit. Conversione da analogico a digitale
 - CD-quality
 - CBR vs VBR
- Realta' aumentata/virtuale: realta' aumentata (pokemon go) ovvero osserviamo la realta' con contenuti lasciati da altri utenti che ci possono dare delle informazioni. La realta' virtuale non c'e' nulla ma con sensori appositi possiamo avere l'impressione di essere in un determinato ambiente ed interagire esso. Per questo ci vogliono immagini con definizione molto alta e dei ritardi estremamente bassi per avere un buon feedback per interagire con l'ambiente virtuale. Si sale decisamente di requisiti sulla rete, perche' questi traffici possono occupare banda (bitrate) fino a 3/4/5 Gbps

1.9 Quality-of-Service (QoS)

Le reti non sono bravissime a fare queste cose, soprattutto a ridurre i ritardi, per cui qualcuno si e' occupato di definire un modo alternativo di funzionamento delle reti che riesce a garantire qualcosa di piu'. Le reti di tutti i giorni funzionano per best effort, fa lo sforzo migliore per trasferire i dati ma non garantisce neanche che i dati arrivino, potrebbero subire ritardi o addirittura neanche arrivare.

- Streaming vs downloading: lo streaming sono dati generati e da consumare con continuita'. La caratteristica principale del downloading e' l'affidabilita', voglio che ci sia ogni singolo bit, nello streaming e' piu' importante l'effetto tempo, ci e' piu' tollerabile vedere un film che fluisce con continuita' anche se magari ogni tanto l'immagine si sgrana perche' abbiamo perso qualche bit piuttosto che vedere ogni tanto il video che si freeza.
- Applicazioni interattive: round trip delay. Non solo voglio vedere il video con continuita', ma ho anche bisogno che se uno dei miei fruitori vuole intervenire (ad esempio domanda in una riunione online) possa farlo entro pochi secondi da quando riceve i dati

Come facciamo a chiedere qualcosa in piu' dalla rete, sono stati definiti dei parametri QoS:

- la sorgente puo' specificare quanto traffico occupera' cosi': i dati vengono divisi in pacchetti (pkt), la sorgente puo' indicare quante' il maxium paket size, il trasferimento medio dei pacchetti: mediamente inietto in rete tot pacchetti per unita' di tempo ed il rate (bitrate) di picco: considerando la frequenza media di immissione, la frequenza massima, la massima dimensione dei pacchetti, piu' di cosi' pero' non chiedo.
- Quanto le destinazioine possono richiedere:
 - reliability: il numero di pacchetti consegnati a destinazione:
 - * percentuale: voglio che la rete porti a destinazione un 95% del tarffico che immetto
 - * Frequenza: Puo' capitare che tutti di bit persi siano insieme, ogni tot non ce ne devono essere piu' di tot persi.
 - Latency: la rete deve garantire che i dati arrivino con una latenza non superiore a. Latenza: momento in cui la rete mette i dati sulla rete, momento in cui arrivano a destinazione. La latenza si somma per ogni hop, e' su tutto il cammino, somma dei ritardi.
 - jitter: varianza sul tempo di arrivo dei pacchetti (arrivo del primo bit di un pacchetto e l'arrivo di un primo bit del pacchetto successivo). La varianza e' lo scostamento dalla media. Noi vogliamo che il jitter sia piu' piccolo possibile. Puo' capitare che sia alto quando i dati si bloccano in una coda quando si liberano.
 - throughput: e' la quantita' di dati a destinazione in un unita' di tempo. (nell'informatica e' la quantita' di lavoro nell'unita' di tempo)

1.9.1 Throughput

I bit che escono dall'unita' di rete sono:

- istantanei: rate in un certo punto del tempo
- average: rate su un lungo periodo di tempo
- goodput: come il throughput ma senza considerare le informazioni di controllo (header)

A sinistra abbiamo la sorgente, ad esempio uno streaming di dati, tra sorgente e destinazione ci sono un po di hop. L'idea e' che hop per hop possiamo avere dei bitrate diversi per ogni tratta di mezzo trasmissivo. Cos'e' che controlla quantita' di dati che escono per unita' di tempo, e' lo hop collo di bottiglia (link) che aggancia due dispositivi adiacenti con bitrate piu' piccolo. Puo' stare da qualunque parte (ricevente o sorgente), e' il canale piu' stretto che il nostro traffico deve attraversare.

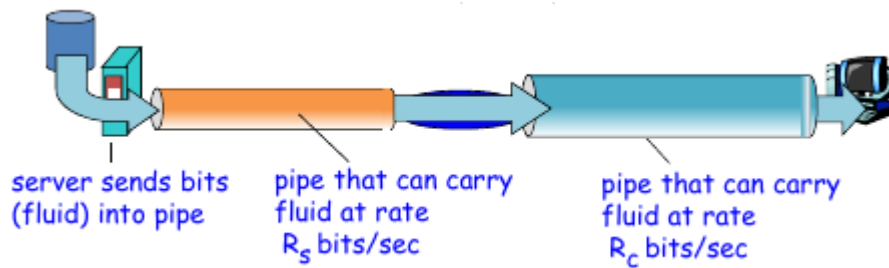


Figure 1.9: Throughput

Internet scenario

Di solito i bottleneck stanno vicino ai server provider o vicino alla destinazione (tipo dal cabinet a casa in rame). Di solito nel cuore della rete non ci sono grossi bottleneck ma c'è multiplexing, nelle grandi dorsali riusciamo a fare multiplexing, ovvero incastrare tanti flussi di traffico che riescono a passare in un canale contemporaneamente. Il cuore di internet è dove viene fatto multiplexing.

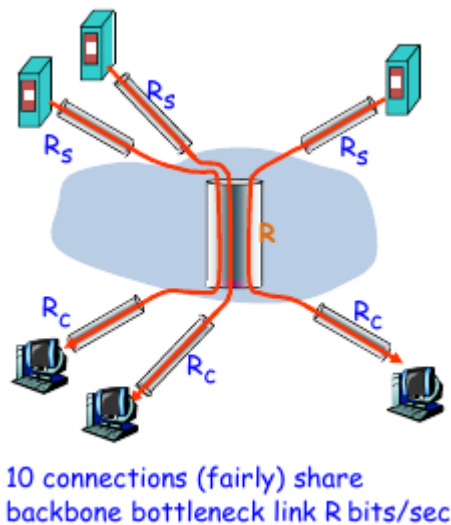


Figure 1.10: Throughput Internet Scenario

1.10 Scambio dati

Normalmente un canale di comunicazione è full-duplex, ovvero che le informazioni possono viaggiare in tutte e due le direzioni, può capitare che un cavo full-duplex potrebbe non essere usato così

- **simplex:** i dati viaggiano in una e una sola direzione: ad esempio un satellite che invia dati a Terra
- **half-duplex:** i dati viaggiano sul canale alternativamente nelle due direzioni: ad esempio una richiesta ad un web server e invio pagina html
- **full-duplex:** i dati viaggiano contemporaneamente in entrambe le direzioni
- **canali simmetrici vs. asimmetrici:** i due sensi di marcia possono avere lo stesso senso di marcia oppure no (upload minore di download)

Chapter 2

Come sono fatte

2.1 Servizi+Protocolli+algoritmi=architettura

L'idea di una rete com'è fatta è simile all'idea di un'architettura stratificata di un sistema operativo. Tutto il software che serve per fare funzionare le reti è parte del kernel, dentro il sistema operativo c'è qualcosa che regola le reti e ha il concetto di livelli: a livello alto ci sono utenti che fanno uso della rete, in basso c'è il mezzo fisico (rame, fibra, wireless) ed in mezzo ci sono i livelli che si occupano di gestire l'hardware e pian piano alziamo i livelli di astrazione fino ad arrivare all'utente. Nel sistema operativo studiamo una macchina sola. Quello che ci definisce una rete:

- **Servizi:** abbiamo su una macchina (dispositivo di rete) un po' di livelli di astrazione, un servizio è quello che sa fare un livello n -esimo e sa offrire questa sua capacità al livello superiore $n+1$. Sono verticali tra due livelli adiacenti.
- **Protocollo:** è un insieme di regole che servono perché in rete ci può essere qualsiasi cosa e questi oggetti in rete si capiscano tra di loro. Un insieme di regole che definiscono come sono fatti i dati che vengono virtualmente mandati da una parte di software che opera ad un certo livello n -esimo dello stack e che virtualmente pensa di parlare con l'entità che lavora allo stesso di astrazione n -esimo sul dispositivo che sta dall'altra parte del cavo. Quando mandiamo una mail non pensiamo alla rete che ci sta sotto, ma pensiamo di parlare con Giorgio che sta dall'altra parte. Ma in realtà stiamo creando del testo da passare al software (che sta ad un livello di astrazione più basso) e così via scendendo per i livelli fino a che non si arriva al mezzo trasmissivo. Il protocollo serve per regolare questa comunicazione logica tra livelli, che in realtà viene implementata chiedendo servizio al livello sottostante. Regolano anche le azioni intraprese a fronte di trasmissione e ricezione dati.
- **Algoritmi:** La visibilità tra i livelli è limitata tra service access point, tipo dei metodi che abbiamo a disposizione da una classe nella programmazione ad oggetti (system call, api). Un livello mette a disposizione del livello sovrastante dei servizi, il livello sovrastante sa che ha quei servizi ma non sa come. Il come è definito dagli algoritmi che implementano i servizi. Sono nascosti dentro i livelli per implementare i servizi ma anche per implementare i protocolli di comunicazione con il software che sta dall'altra parte della rete (Giorgio).

La freccia arancione ci dice come avviene veramente la comunicazione, dal livello più alto al mezzo trasmissivo e viceversa.

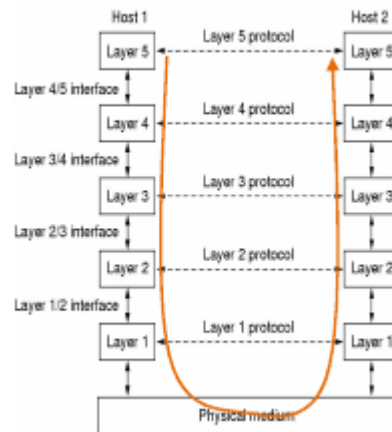


Figure 2.1: Architettura

2.2 Architetture di rete: Incapsulamento

Qui sta la fregatura della rete. Al livello piu' alto abbiamo un'applicazione (browser) che ha bisogno di usare la rete. Costruiamo dei dati che devono andare dall'altra parte e scriviamo un URL o una stringa per fare una ricerca, io pure illudendomi di parlare con il server google non l'ho fatto, ho chiesto ad una parte del kernel di mandare informazioni dall'altra parte, in particolare gli ho detto che io la voglio la risposta, quindi assicurati che arrivino, per assicurarsi che arrivino il livello sotto ha aggiunto delle informazioni per dire all'altra parte "quando ti arrivano mandami un riscontro" perche' se non ti arrivano ti rimando i dati (questo per ottenere reliability), questi sono header, informazioni di controllo che fa parte del livello di comunicazione sottostante. Il livello 4 non e' ancora sulla rete e passa i dati al livello 3, in cui i dati vengono rotti in due, il livello sopra viene considerato come un dato solo dal livello 3 che essendo troppo grosso viene diviso in 2, ottenendo un dato che e' fatto dal'header del livello 4 + porzione di dati generati dall'applicazione e un altro pezzo con solo una porzione di dati generati dall'applicazione (il livello 3 trova la rotta, per che router passare). I router successivi si parlano tramite altre istruzioni di controllo generato dal livello 3. Scendiamo ancora al livello 2, dove i dati perdono la loro forma di dati e diventano una sequenza di bit che dev'essere mandata sul filo, anche qui aggiunte informazioni di controllo (header di livello 2) e una tail di livello 2. Sono bit che sono dei pattern particolari che sono messi sul filo prima del dato vero e proprio che producono una forma d'onda particolare che dice all'oggetto dall'altra parte del filo "da qui in poi non c'e' piu' la portante (la forma d'onda che dice che sono sveglio) ma cominciano i dati", lo stesso in coda una sequenza di bit che dice che torna solo la portante.

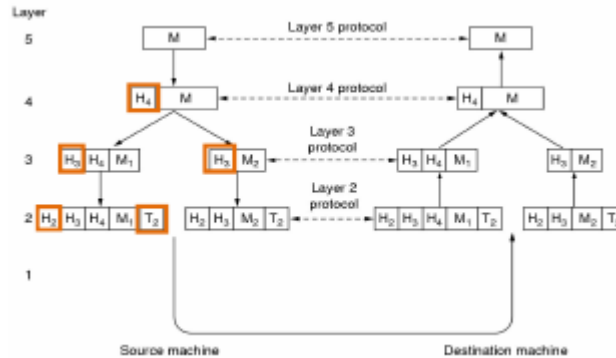


Figure 2.2: Incapsulamento

Perche' il provider ci dice che ci da una banda larga (20 Mbps) ma non ne vediamo piu' di 12 Mbps, peche' loro ci vendono la banda lora, ovvero compresi tutti gli head e la tail, quindi noi intendiamo netto e lui intende lorda.

La frattura dell'unita' dati nel livello 4 che passando al livello 3 viene divisa, perche' non e' in grado di gestirla. La parte di dati che viene passata si chiama payload (M). Quando passo i dati di livello 4 che sono dati + header, il livello 3 decide che tutta questa roba e' troppo grande da gestire, come lo decide. In sistemi operativi vedremo le tecniche di gestione della memoria, i router (calcolatori nel cuore della rete specializzati per far funzionare la rete), ci sono punti che sono molto trafficati e a questi router facciamo gestire le cose nel modo piu' semplice possibile. Diciamo che a questo livello (3) dividiamo i dati qualunque'essi siano in porzini di una dimensione massima fissata, cosi' i router sanno che piu' di tanto non gli arriva e la pagina di memoria dev'essere libera grande tanto quanto la massima quantita' di dati che mi puo' arrivare, facendo cosi' rendiamo la vita facile ai router. Tutti i dati che arrivano dal livello 4 vengono intesi dal livello sottostante 3 come payload, cioe' non guarda quello che gli e' arrivato ma divide quella roba a meta'. Quindi c'e' trasparenza tra i dati generati dal livello superiore a quello inferiore, come in posta, non guardano quello che c'e' nei pacchi.

2.3 Funzionalita' dei livelli

- regole di trasferimento
- routing
- frammentazione, ordinamento dati: la sorgente sta inviando delle informazioni che devono essere consumate nell'ordine in cui sono stato inviate (ad esempio i fotogrammi di un video). Quindi quando piu' dati (o frammenti) vengono mandati all'altra parte, se sono in realta' un dato unico

dal punto di vista semantico dell'applicazione, i vari pezzi devono essere ricomposti secondo l'ordine con cui sono stati inviati.

- convenzioni naming e addressing: come indichiamo ed individuiamo un particolare dispositivo (rete o end-system) nella rete, perche' abbiamo bisogno di identificare a chi mandare i dati, in modo tale che ogni dispositivo in rete riesca a decidere la rotta da fare. Si possono utilizzare dei nomi simbolici (www.unimi.it) facili da ricordare per noi ma meno gestibili per le macchine le quali gestiscono meglio i numeri, vengono poi tradotti in un numero (indirizzo numerico, 110.27.58.3, sono 4 numeri che stanno per gli indirizzi usati in una certa porzione dell'architettura di rete come numeri su 32 bit, e la rappresentazione decimale puntata ci dice il valore di ogni indirizzo) che poi viene utilizzato dai livelli piu' bassi della rete.
 - controllo degli errori, reliability: le nostre informazioni passano su mezzi trasmissivi, puo' succedere che il segnale venga alterato a causa di interferenze esterne ed alterare la forma d'onda modulata in un certo modo. Se non vogliamo che la destinazione capisca roma per toma deve controllare che il dato che ha ricevuto sia corretto. Il controllo degli errori puo' essere aggiunto nell'header, ad esempio con il bit di parita' (aggiungo 0 o 1 per rendere pari il numero di 1 nell'informazione). Se controllando l'informazione che ricevo il numero di 1 sono dispari sicuramente il dato e' stato alterato. E' un controllo dell'errore debole perche' trova tutti gli errori che cambiano un numero dispari di bit. Rileva il 50% di bit. Ci sono tecniche piu' complicate per fare controllo degli errori. Sono dei bit di ridondanza che permettono al destinatario di capire se quello che ha ricevuto e' quello che e' stato inviato dalla sorgente. Reliability (affidabilita') e' strettamente collegata al controllo dell'errore, vogliamo che tutti i dati arrivino. I dati abbiamo gia' visto come si possono perdere (buffer overflow) se un dato arriva a destinazione alterato e il sistema dai bit di ridondanza capisce che e' alterato non puo' fare altro che buttarlo via, ma se questi dati proprio devono arrivare a destinazione l'affidabilita' puo' essere ottenuta in due modi:
 - Se proprio dev'essere buttata la destinazione deve dire alla sorgente di rimandare il dato.
 - aggiungo ai dati cosi' tanti bit di ridondanza che la destinazione non solo e' capace di capire che un errore si e' verificato ma anche di capire quali bit si sono alterati e di ricostruire l'informazione corretta.
- Di solito e' difficile che nelle reti si faccia la correzione di reti, perche' significherebbe pesare tanto sul livello 3 e si utilizzerebbe troppa banda per l'header e rimarrebbe poca banda netta
- flow control (orizzontale), congestion control (verticale)
 - Controllo di flusso: e' un controllo orizzontale che mira a trovare un punto di equilibrio tra i dati mandati dall'entita' di un livello x e i

dati che vengono ricevuti da un'altra unit  sempre al livello x. Se la sorgente manda dati piu' velocemente che la destinazione sappia processarne sicuramente andremo incontro ad una perdita di dati, che non vogliamo se puntiamo all'affidabilit . E' un'altra informazione da head, la destinazione deve dire alla sorgente di non mandare piu' dati di un tot. Orizzontale perche' su due entit  di pari livello.

- Controllo di congestione, verticale: un livello che chiede un servizio al livello sottostante si occupa di non chiedere dati piu' velocemente di quello che il sistema sottostante riesce a processare. Oppure se il livello inferiore passa dati al livello superiore troppo velocemente, piu' velocemente di quanto il livello superiore riesce a togliere dal buffer e' un problema, perche' prima o poi il livello superiore sovrascriver  dei dati che ancora erano nel buffer. Quindi serve per mettere d'accordo due livelli adiacenti sulla velocit  con cui si parlano.

- (de)multiplexing
- Creazione di sessioni e connessioni:
 - Sessioni: hanno a che vedere con il multimedia, ad esempio codifiche in cui viaggiano separatamente dati di audio e di video, questi due flussi devono arrivare insieme. Possono essere situazioni in cui i dati vengono inviati in flussi di versi o sistemi diversi pero' devono essere consumati insieme.
 - Connessioni: legame tra sorgente e destinazione per tutta una serie di dati che la sorgente manda alla destinazione.

Non tutti i livelli fanno tutto. Ogni funzionalit  e' risolta ad ogni livello. Qualche funzionalit  e' risolta a piu' livelli.

2.3.1 (de)Multiplexing

Negli end-system l'interfaccia di rete attiva e' 1. La rete da un cavetto o da wifi, pero' ha funzionante una scheda di rete alla volta. Pero' possono essere aperte piu' schede sullo stesso browser. Quindi sono dati che vanno in due posti diversi, pero' devo usare in qualche modo l'unico canale di rete che il PC ha. Quindi devo decidere quale informazione fare passare, suddividendo all'ingresso della rete i vari traffici ai diversi destinatari.

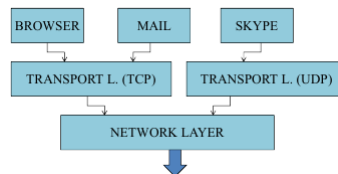


Figure 2.3: (de)Multiplexing

2.4 Connection-oriented service (servizio orientato alla connessione)

La caratteristica è che è ordinata, la sorgente manda una sequenza di dati e gli indica l'ordine, banalmente con un numero che indica che pezzo è'. Per garantire l'ordinamento le parti che comunicano devono mettersi d'accordo, legarsi in qualche modo e dirsi, la sorgente, ti manderò una serie di dati e dovrai passare prima il pezzo 1 etc. Perché possono arrivare in ordine strano? ad esempio per un bufferoverflow e dei pezzi sono stati inviati, o per le decisioni di instradamento vari pacchetti fanno strade diverse. Vengono implementati tra protocolli che sono costruiti su 3 fasi simili ad un servizio telefonico:

- apertura connessione (handshaking): deve stabilire un accordo, la destinazione li numera su un dato numero di bit per spiegare come la destinazione deve riordinare i dati.
- scambio dati
- chiusura connessione

I dati sono ricevuti in ordine:

- orientato a messaggi vs. orientato a byte
- può garantire anche affidabilità, ma può anche essere inaffidabile, magari nell'ordine c'è qualche buco.

alti tempi di trasferimento a causa della latenza iniziale di apertura connessione, non sempre desiderabile, trasferimento pochi dati, non si usa per dati multimediali perché sono tanti dati, il video ed audio si bloccherebbero per aspettare tutti i pacchetti.

2.5 Connectionless service

È un servizio non orientato alla connessione, simile alle poste, perché nessuno garantisce che una lettera arrivi prima di un'altra. Ogni messaggio è gestito indipendentemente. Non garantisce ordine ma per la necessità delle due parti che lavorino di comune accordo per mantenere l'ordine il servizio connection oriented impiega più tempo a trasferire dati (a causa della latenza iniziale). Ortogonale è la proprietà di affidabilità, perché anche connectionless può essere affidabile o potrebbe non esserlo. È più rapido, perché non c'è un sovraccarico delle informazioni che devono essere passate nello header c'è minore overhead. Non ha le fasi di apertura e chiusura.

Service	Example
Reliable message stream	Sequence of pages
Reliable byte stream	Remote login
Unreliable connection	Digitalized voice
Unreliable datagram	Electronic junk mail
Acknowledged datagram	Registered mail
Request-reply	Database query

Table 2.1: Differenze

- - 3 - 5 7 8 - 10	e' ordinato ma non affidabile
1 2 3 5 4 5 7 6	e' affidabile ma non ordinato
1 2 3 4 5 6 7	e' affidabile e ordinato
1 - 3 5 4 - 7	non e' affidabile ne ordinato

Table 2.2: Orientamento vs Affidabilita'

2.5.1 Ordimaneto vs Affidabilita'

Sono ortogonali:

La prima riga e' connection oriented e inaffidabile, la seconda e' un servizio connectionless e affidabile, la terza connection oriented e affidabile, la quarta connectionless e inaffidabile. Ci sono tutte e quattro le combinazioni. Poi noi vedremo protocolli che fanno solo la piu' rigida e la piu' naif. Per orientamento basta numerare i dati mentre per affidabilita' bisogna fare in modo che non si perdano dati, oppure ci si accorge e si ri-trasmette.

2.6 Tecniche di commutazione

L'immagine e' uno schema di rete di telefonia usata anche per trasferimento dati, dove quello che succede e' che un utente con apparecchio chiamante vuole parlare con un utente su un utente chiamato. Viene fatto un numero (indirizzamento) dell'apparecchio chiamato, quello che succede nella rete e' che vengono stabiliti dei contatti nei vari armadietti in modo che venga costruito un circuito chiuso tra chiamante e chiamato, facendo si che il cammino connette chiamante e chiamato. Questa viene chiamata rete a commutazione di circuito, costruisco un cammino fisico tra le due parti che si devono parlare, il cammino per tutta la durata della conversazione puo' essere usato tra chiamante e chiamato. Le tratte di filo sono ad uso esclusivo tra chiamante e chiamato. Quindi possono usare tutto il bitrate della rete. quindi se c'e' un cammino fisico abbiamo connection oriented gratis (i dati arrivano sempre in ordine), il fatto che il cammino sia riservato significa che non ha piu' bisogno di dire a chi vanno i dati, perche' possono fare una sola strada. Il che vuol dire che abbiamo un ritardo di costruzione del cammino ma abbiamo gratis ordinamento e non dobbiamo aggiungere ai dati informazione di controllo che specifichino indirizzo di destinazione. Punti critici:

- Se si rompe una tratta di filo la sorgente continua ad immettere dati su questo circuito ed ad un certo punto si accorgono che un sistema non riceve più il segnale di portante, però finché non facciamo rendere conto la sorgente lei continua a mandare dati, con conseguente perdita di dati. Se il cammino si rompe dobbiamo notificarlo alla sorgente e bisogna ripagare il tempo e le operazioni per ricostruire un altro cammino e poi a rimandare i dati. I dispositivi pensano che il circuito ci sia, ma da qualche parte c'è un buco in cui i dati precipitano, quindi c'è bisogno di tempo prima che si accorga che c'è un errore.
- Stiamo usando male le strutture di rete, se è in corso una trasmissione telefonica e dobbiamo passare pochi byte di conversazione sprechiamo risorse, perché il mezzo trasmissivo ad esempio può portare 100 Mbps.

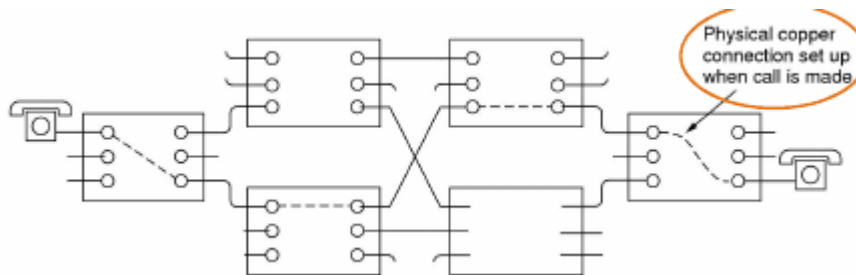


Figure 2.4: Commutazione di Circuito

E' una cosa storica e non si fa più, perché questi aspetti negativi hanno portato ad inventare altre tecniche di commutazione.

2.7 Commutazione in pacchetto

Come funzionano le reti oggi. I sistemi di rete hanno creato dei pacchetti (dati dei livelli superiori + header livello 2), nello header è indicato l'indirizzo a cui questi dati vanno (indirizzo della destinazione). Il primo pacchetto mandato verso la destinazione, occupa per un certo tempo il canale per trasmettere tutto quanto, il pacchetto 1 arriva alla stazione B, ci guarda dentro e capisce che fa del pacchetto 1. Contemporaneamente A manda il pacchetto 2, in questo tempo B ha capito che fa del pacchetto 1 e lo manda a C il quale a sua volta processa il pacchetto 1 e lo invia ad altri. Quindi mentre i sistemi computano i pacchetti la sorgente manda altri pacchetti. E' pipelining, ovvero le stazioni lavorano in parallelo, in questo modo sfruttiamo tutto il mezzo trasmissivo e il tempo sia compresso ed il trasferimento è più veloce perché abbiamo tolto la costruzione iniziale del circuito. Lo svantaggio è che se i pezzi di dati viaggiano disordinati i pacchetti possono prendere cammini diversi. Non c'è più un cammino riservato, ogni dispositivo deve rendersi conto di qual'è la topologia della rete ed in base a com'è fatta deve trovare il percorso. Caratteristiche:

- Ordinamento non e' garantito, perche' se i pacchetti sono indipendenti possono seguire percorsi diversi. Quindi pacchett2 puo' arrivare prima di pacchetto1.
- statistical multiplexing: c'e' condivisione della banda. Incastro il traffico che arriva da piu' sorgenti ed arriva a piu' destinazioni. Efficiente uso della banda.
- Non c'e' latenza per creare il circuito ma abbiamo maggiore overhead, perche' negli head ci sono i dati per l'instradamento.
- Migliore resistenza ai guasti: perche' non abbiamo un cammino scolpito nella pietra, ma ogni router prende un pacchetto che gli arriva, guarda, tramite le sue conoscenze di topologia della rete, dove mandare il pacchetto.

2.8 Confronto tecniche di switchng

Nella commutazione di circuito paghiamo una quantita' di tempo iniziale per la creazione del circuito tramite pacchetti di controllo per l'apertura del circuito.

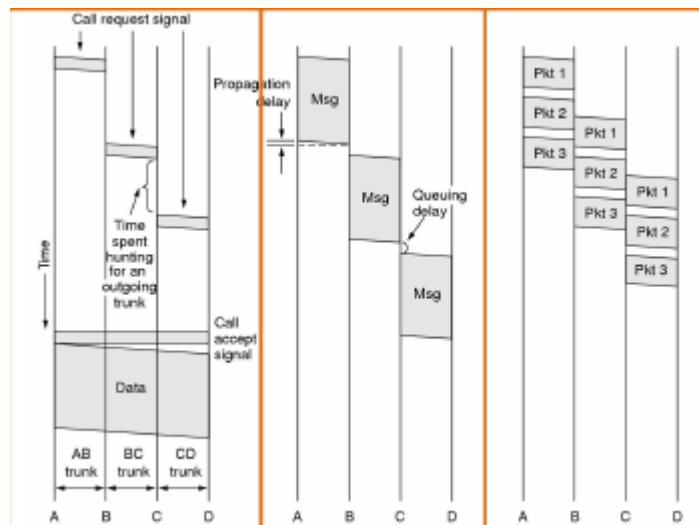


Figure 2.5: Confronto switching

2.8.1 Esercizio di confronto msg vs. pkt switching

Le fonti di ritardo:

- processing: trascurabile



Figure 2.6: Esempio

- queuing: trascurabile, stiamo assumendo che la porzione di rete che stiamo utilizzando e' vuota
- propagazione: trascurabile
- trasmissione: delay su 1 link

Rete (3 Hop):

- Sorgente
- Router
- Router
- Destinazione

Se dovessimo implementare lo scambio di un messaggio di 7.5 Mb con una rete a commutazione di circuito avremmo un transmission delay su 1 link dato da: $7.5\text{Mb di dati} / 1.5 \text{ Mbps per link (per hop)} = 5 \text{ sec.}$ La ricezione a destinazione arriva dopo 15 sec. perche' sono 3 hop. Cosa succederebbe se usassimo invece della commutazione a circuito la commutazione a pacchetti? Supponiamo che il messaggio da inviare e' frammentato in 5000 pacchetti da 1.5 Kb l'uno:

- Transmission delay su 1 link: $1.5 \text{ Kb} / 1.5 \text{ Mbps} = 1 \text{ msec.}$ E' il tempo che intercorre da quando la sorgente codifica il primo dei bit e l'ultimo.
- A T_0 (copia a casa)
- Ultimo pacchetto quando e' trasmesso? a 4.999 sec (perche' il primo a 0 msec, il secondo a 1 msec ...)
- Ultimo pacchetto raggiunge la destinazione dopo: $4.999 + (0.001 \cdot 3) = 5.002\text{sec.}$

Quindi eliminato il set-up del circuito ed aggiunto il pipelining abbiamo potuto ridurre ad 1/3 la latenza tra quando trasmette il primo bit di dati a quando la sorgente riceve l'ultimo bit di dati.

2.8.2 Perche' accodamento?

Perche' non ho piu' un percorso riservato. Il mezzo trasmissivo e' condiviso da piu' dati. Le parti di rete punto-punto sono punti in cui ci puo' essere accodamento, perche' i dispositivi ricevono dati da tanti dispositivi. C'e' un po' di ritardo di accodamento negli switch, che sono quelli che connettono e costruiscono le Lan (che e' una rete broadcast), se un router deve mandare dei dati broadcast su una rete punto-punto (ogni router esige di avere una sua interfaccia su una rete diversa), vuol dire che puo' essere un bus condiviso oppure se c'e' di mezzo uno switch, il quale e' incapace di trovare una rotta nella rete, semplicemente sa che sulle interfacce sono collegati diversi end-system, quindi fa un routing banale: se un host manda un dato che deve andare ad un altro, lo switch fa in modo di non mandare i dati a tutti i dispositivi connessi.

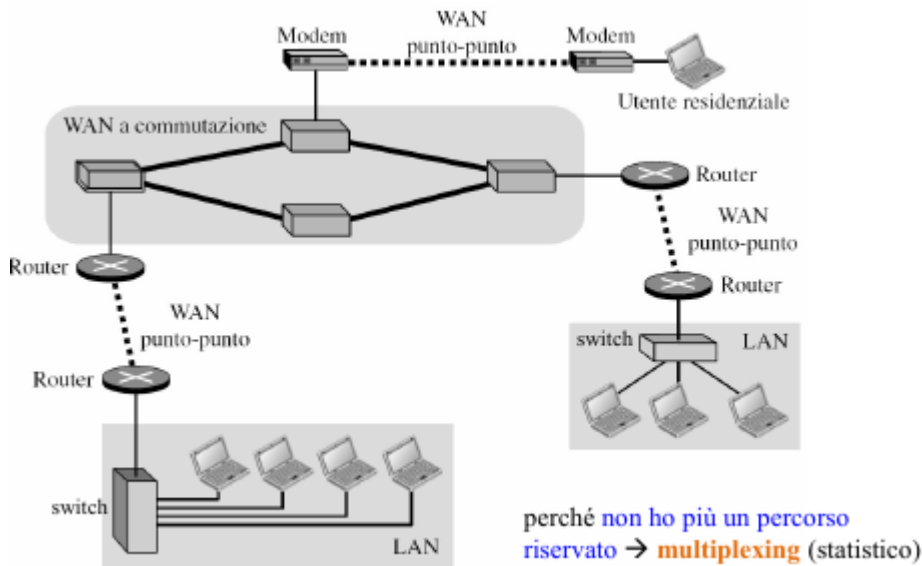


Figure 2.7: Code nel packet switching

2.9 Routing a circuito virtuale

Possiamo emulare i pro di una commutazione di circuito in una rete a commutazione di pacchetto. Lo facciamo con il routing. Abbiamo la possibilità di usare routing a circuito virtuale che sono algoritmi di instradamento che sopra rete a commutazione di pacchetto emulano comportamento rete a commutazione di circuito. Sto parlando routing, quindi possono essere solo routing. Per ogni destinazione della rete (nella rete vera significa ogni oggetto che c'e' in Internet) so che se mi arrivano dati al mio apparato devo mandarlo al mio vicino. Nelle reti a circuito virtuale quest'informazione viene usata in altro modo:

- Fase 1: costruzione virtual circuit:
 - facciamo una call request di un pacchetto di controllo verso la destinazione
 - Questo pacchetto viene spedito alla destinazione usando la routing table
 - Ogni nodo attraversato genera una tabella dei circuiti virtuali (VCI Virtual Circuit ID) dove si ricorda che ogni dato della sorgente. I pacchetti che mi manda il router i marcati devo mandarli al router j con un altro identificatore di circuito virtuale.
 - Fase 2:
 - * Il pacchetto marcato con l'identificatore di circuito virtuale viene instradato su next hop corrispondente
 - Fase 3: chiusura circuito
 - * Call clear del pacchetto instradato lungo il circuito per la rimozione della VCI.

Quindi in pratica fisso il cammino con la tabella del circuito virtuale e poi non interpello piu' la tabella di routing. Fissando il cammino ho ordine nella trasmissione dei pacchetti. Ho dei vantaggi, perche' il nodo puo' marcare i pacchetti da mandare allo stesso router in maniera diversa, quindi il router sta usando la sua uscita per fare multiplexing di vari dati. Risparmiamo sui tempi di processing, la tabella di instradamento di una rete a commutazione di pacchetti che usa routing datagram ha milioni di righe non ordinate. Trovare la rete a livello algoritmico in un circuito virtuale la meta' della tabella del circuito digitale, che e' molto piu' piccola della tabella di routing.

2.10 Routing datagram

Virtual circuit e' molto costoso e non viene utilizzato veramente in Internet. Ogni pacchetto reca l'indirizzo di instradamento ed e' instradato indipendentemente:

- per ogni pacchetto c'e' una scelta nella routing table per l'uscita migliore al momento
- I pacchetti di una sessione possono seguire path (cammini) diversi
- Possibile arrivo fuori ordine e perdita dei pacchetti

E' esattamente quello che abbiamo visto fino ad adesso e dato per scontato.

Conetion oriented, circuit switching e virtual circuit non sono la stessa cosa. Connection-oriented e' tipo di servizio offerto sopra. circuit switching e' politica di commutazione per comunicazione tra peet in alcune reti. Virtual circuit e' politica di routing in reti a commutazione di pacchetto. E' possibile fornire servizio connection oriented sopra rete a commutazione di pacchetto con routing datagram.

2.11 Chi e' importante

Ci siamo accorti che dobbiamo standardizzare i protocolli di comunicazione:

- ISO
- ITU-T
- IETF: InternetDraft propongono delle soluzioni che vengono standardizzate in request for comment
- IEEE: comunita' chiusa in cui i documenti non sono liberamente disponibili, facciamo continuamente riferimento a questi standard

Ora andiamo a vedere qual'e' lo standard delle architetture a livelli che abbiamo visto.

2.12 Architetture Standard

Il primo problema che si sono posti e' di quanti livelli fare. Abbiamo gia' detto che per affrontare un problema grosso (la comunicazione in rete) deve essere suddiviso in sotto-problemi (principio del divide et impera) a differenti livelli di astrazione. E' un problema perche' avendo un certo numero di livelli che regolano il dialogo tra due applicazioni (ad alto livello, dove la rete e' una scatola nera) trattando tutti i problemi, offrendo tutti i servizi, usando tutti gli approcci (instradamento) in modo che l'utente piu' di tanto non debba urtarsi con problemi che non vuole vedere. Impatta su due aspetti:

1. Comunicazione orizzontale: il dialogo tra livelli, richiesta di servizi tra livello superiore e livello inferiore. La comunicazione e' fatta tramite buffer, in cui vengono messi i dati o in salita o in discesa (come visto in figura 2.1). Quindi se facciamo troppi livelli stiamo appesantendo, perche' e' un continuo passare i dati e passare il controllo tra processi (thread). Troppi passaggi di mano tra processi che implementano le funzionalita' del livello (sia superiore o inferiore), vuol dire perdere tempo e rischiare il buffer overflow di dati che non sanno piu' dove essere scritti.
2. Comunicazione verticale: se ogni livello per comunicare con un suo pari livello ha bisogno di aggiungere dati (payload) questo significa che i dati inviati dall'utente vengono imbottiti da tanti header significa che stiamo usando male il canale di comunicazione, perche' grossa parte dei dati passati sarebbero gli header di ogni livello.

Questo ci dice di usare pochi livelli, ma se ne avessimo troppi pochi avremmo problemi di implementazione. Quindi per ogni livello c'e' una funzione ben definita minimizzando la quantita' di livelli e la quantita' di dati da scambiare. Sono venuti fuori due modelli standard di natura molto differente:

- TCP/IP: e' nato a posteriori, nel momento in cui la rete si chiamava alphanet, ci lavoravano poche persone perche' i computer li avevano solo i grossi enti. E' nato da qualcuno che ha iniziato a cercare di mandare informazioni non materialmente (non spostando il supporto fisico), l'universita' di Berkley (California). Quindi con la diffusione e' diventato uno standard. Lo usiamo ancora tutti i giorni ma da un po' di tempo ha iniziato a mostrare delle crepe.
- ISO/OSI: e' nato al rovescio. Pensando ad una rete ideale e a come si voleva farla funzionare hanno definito in modo pulito quali sottoproblemi dovevano essere risolti, ha individuato un livello per ogni funzionalita', anche con una visione futuribile.

Si sono invertite un po' le parti negli ultimi tempi, OSI e' abbastanza ampio e robusto da riuscire ad accogliere le novita' delle reti (ad esempio i file multimediali (che IP non riesce ad implementare)). Per la didattica faremo riferimento all'architettura OSI (piu' pulita, chiara e meno da smanettoni) poi vedremo dove si incastrano i protocolli IP.

2.12.1 OSI (open system interconnection)

E' fatta su sette livelli, li vediamo con approccio top-down. La periferia della rete sono degli end-system, mentre in mezzo gli utenti non ci sono ma ci sono solo dispositivi di rete.:

- Livello applicazioni utente (7): siamo gia' nella rete, stiamo ragionando su comunicazioni e protocolli (il browser non fa parte del livello applicazioni, ma e' solo la vera e propria applicazione utente ma in se non parla con nessuno, deve chiedere l'aiuto del servizio del livello 7 per chiedere di portare dall'altra parte delle informazioni (o di recuperare informazioni)). Quello che vediamo nel browser e' HTTP://(ci fa navigare nel WEB, e' un protocollo di comunicazione a livello 7 per il trasferimento di Hyper Testi) che ci dice che e' un'applicazione multi-servizio, noi specifichiamo quale servizio vogliamo utilizzare ed il browser ci aiuta.
- Livello di presentazione (6): e' un livello che e' rimasto vuoto per un po'. Se una volta le reti non erano a portata di chiunque ma erano solo nelle corporate le persone che ci ed avevano bisogno che la rete funzionasse e non c'erano dei malintenzionati che erano in rete per disfare il lavoro altrui. Quindi all'inizio della rete non c'erano problematiche di sicurezza. Chi aveva progettato l'OSI aveva gia' pensato al problema di sicurezza, implementando questi servizi (ad esempio la cifratura dei dati) in alto nell'astrazione, cosi' in alto perche' cosi' non facciamo la cifratura tra apparati di rete ma solo tra host ed host. L'altro problema e' del marshaling: manipolare i dati in modo che siano comprensibili tra i sistemi che si parlano (i dati devono essere comprensibili anche tra due host con sistemi operativi diversi), c'e' anche la possibilita' di avere codice distribuito su

diverse macchine e invochiamo l'esecuzione del codice che e' su altre macchine (ad esempio nel cloud), passo parametri per riferimento all'host B che mi ritorna il valore del parametro eseguendo il codice che mi serve che ha lui.

- Livello sessione (5): sincronizzazione tra i dati, dobbiamo mettere insieme i flussi di dati che partono da sorgenti diverse in modo che sia sensato per chi li riceve. La gestione del multimedia e' incastrata in questo livello 5.
- Livello di trasporto (4): e' quello piu' basso che regola comunicazione tra host e host senza vedere come la rete e' dentro (i primi 4 livelli si chiamano end to end). Deve nascondere le eterogeneita' e cambiamenti sottostanti indipendentemente da com'e' fatta la rete, deve permettere a sorgente e destinazione di comunicare a prescindere dal quello che c'e' sotto (come una coperta buttata sopra la rete). Dobbiamo risolvere problemi di Multiplexing e deMultiplexing (tanti traffici da tanti applicazioni in un'unica uscita di rete), problemi di naming (questo dato e' arrivato ma a questo punto devo avere delle informazioni che mi permettono di dire a quale utente e a quale informazione appartiene). Dobbiamo fare controllo dell'errore (quello che mi e' arrivato e' esattamente quello che mi hai inviato), controllo di flusso (mando i dati non troppo velocemente), controllo di congestione (do lavoro alla rete non troppo in fretta per non metterla in difficolta')
- Livello Network (3): funzione principe e' l'instradamento, quindi cercare di costruire in ogni nodo (end-system e router) cercare di capire la topologia di rete e trovare una rotta per portare i dati da sorgente a destinazione. Anche qui c'e' controllo di congestione per aiutare il livello sopra, collaborano per non portare la rete a situazione di traffico congestionato. Si preoccupano anche di interconnettere reti eterogenee, pezzi con determinati protocolli a canali cablati e con altri protocolli a zone wireless.
- Data link (2): e' l'ultimo che vede dei bit e ragiona con i bit, sa che stanno per essere messi su mezzo trasmissivo e sa che su questo mezzo puo' succedere di tutto. Si occupa di prendere dei frame (vanno bene tutte e due le traduzioni) che si sono persi o danneggiati o duplicati (negli ultimi casi li elimina). Fare controllo di flusso (e' una funzionalita' duplicata, regola la velocita' con cui un dispositivo di rete manda dati al successivo dispositivo di rete). C'e' un sottolivello: MAC: che in reti broadcast si occupa di gestire la contesa. Due stazioni non possono parlare contemporaneamente nel mezzo condiviso, creando dei turni e capendo se ci sono delle collisioni
- Livello Fisico (1): e' il cavo, dal punto di vista di chi invia l'informazione prende i bit e li trasforma in onda elettromagnetica e da ricevente campiona l'onda e trasforma in bit.

In linea di principio ogni livello puo' fornire il servizio a quello superiore.

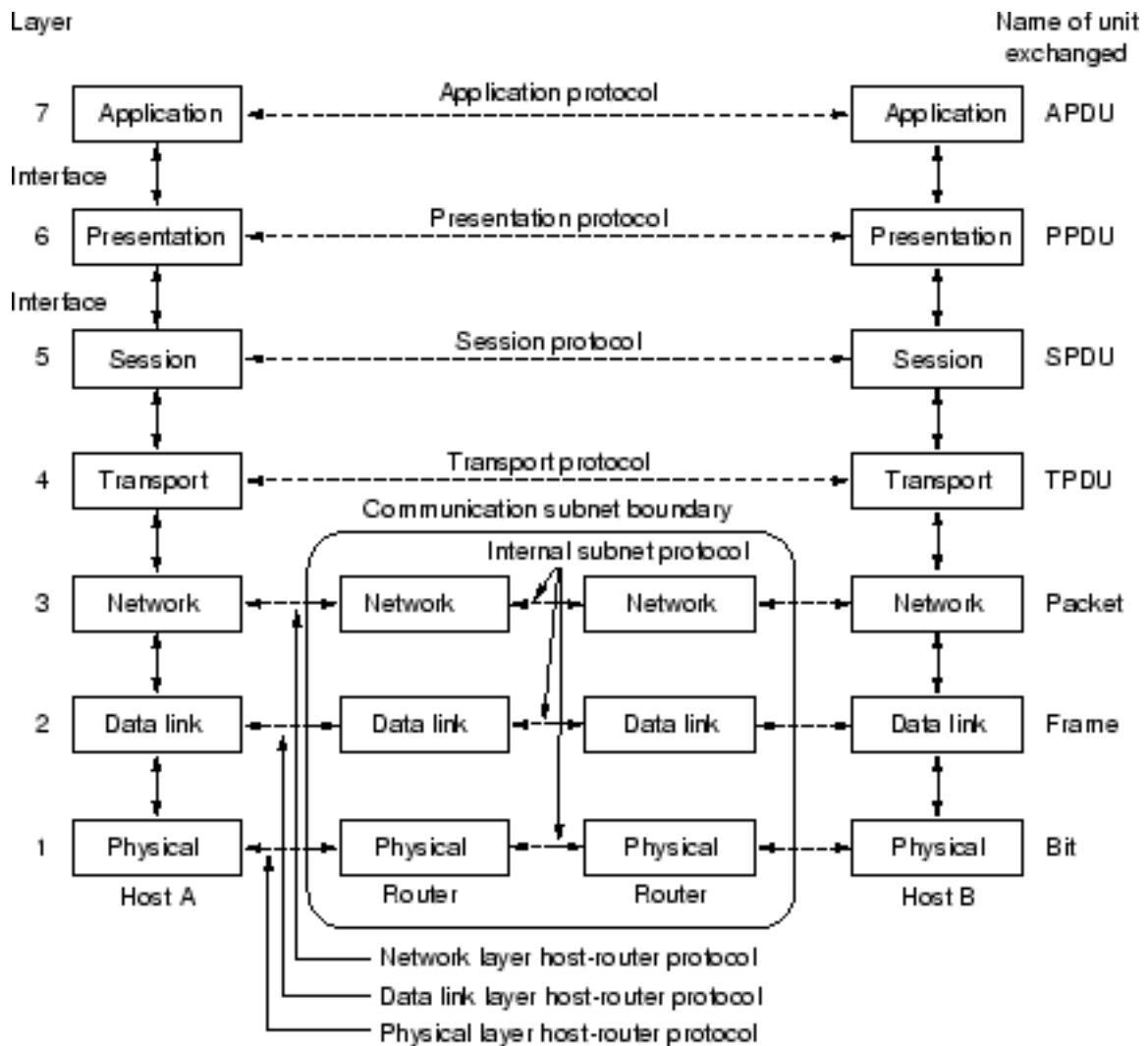


Figure 2.8: Modello ISO/OSI

2.12.2 Modello TCP/IP (Internet)

A livello internet c'è solo un servizio, a livello di trasporto 2: (Livello di trasporto)

- TCP: Connection oriented ed affidabile: i dati arrivano in ordine e tutti senza buchi.
- UDP: è connectionless e non affidabile: fa del suo meglio per portare i dati a destinazione cercando di fare il possibile, fa un controllo di errore

per capire se ci sono state alterazioni

- Livello di applicazione: e' uguale, non si sono preoccupati dei protocolli di sicurezza e di comunicazione tra le macchine.
- Livello di trasporto: puo' essere ordinato ed affidabile. Si aspetta che arrivino piu' pezzi di dati alla sorgente e li rimette in ordine per non passarli sopra in ordine casuale, visto che e' affidabile cerca di minimizzare la sua stessa possibilita' di errore (di non inviare tutti i dati) inglobando meccanismi di controllo di flusso ed esecuzione, quindi passo i dati non troppo velocemente per non fare verificare buffer overflow. Ci sono dei problemi di lentezza.
- Livello Internet: e' un servizio connectionless con approccio best-effort
- data link questi ultimi due completamente sovrapponibili
- fisico

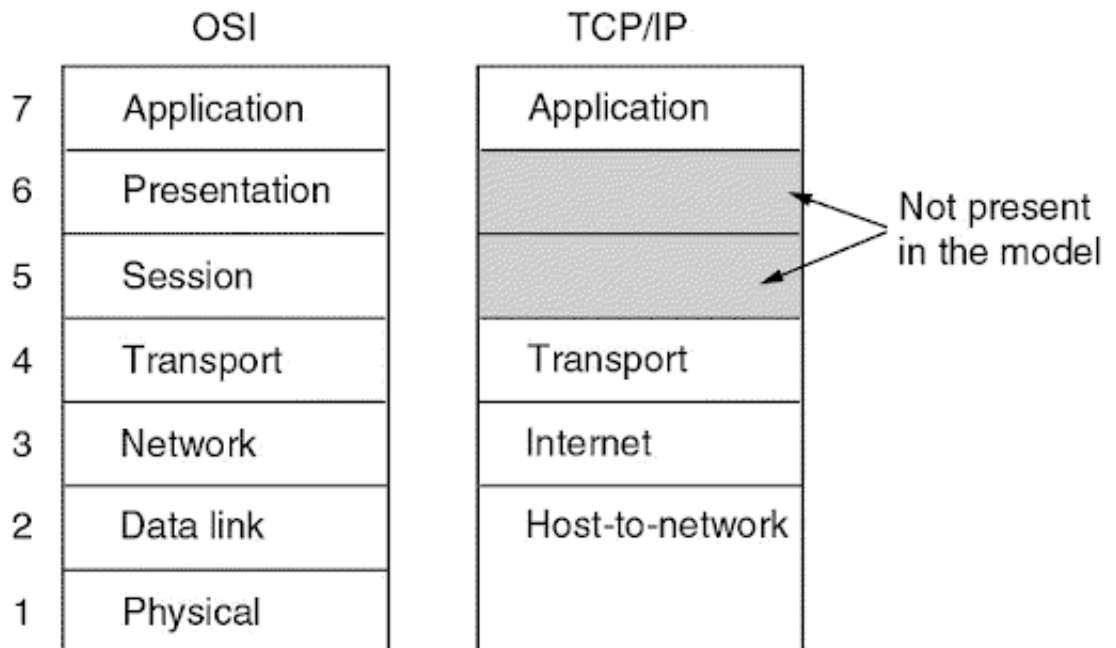


Figure 2.9: TCP/IP

2.12.3 data transmission and PDU

(solo per ulteriore nomenclature) A livello di trasporto si parla di messaggi con servizi connectionless (ogni unita' dati e' indipendente dalle altre), o segmenti

(se il livello di trasporto sta implementando un servizio connection oriented verso il livello superiore). Al livello(3) network si parla di datagrammi (questo viene dal routing datagram, l'approccio di instradamento usato in Internet) o di pacchetti, in quanto non si guarda piu' che dati sono e la rete si occupa solo di mandare avanti questo pacco fino a destinazione. Al livello 2 si parla di frame.

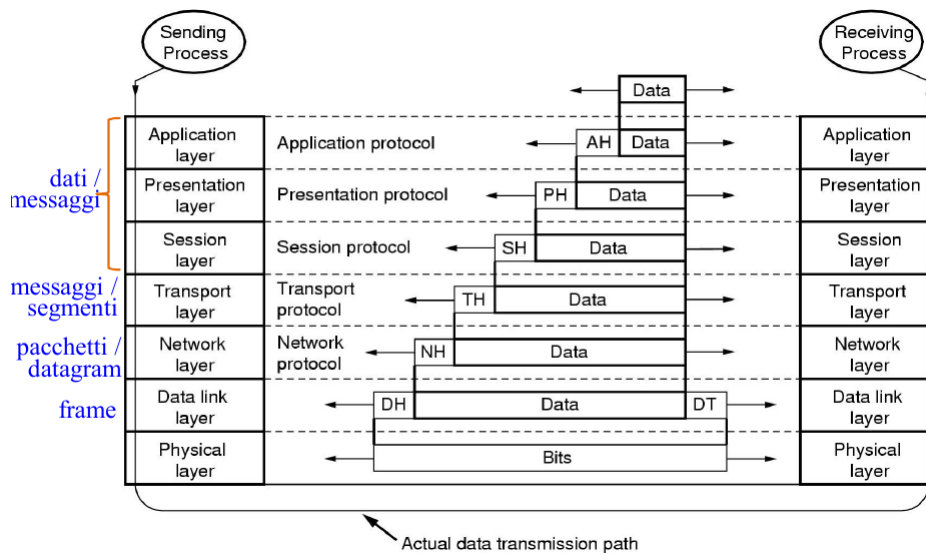


Figure 2.10: Nomenclatura dati

2.12.4 OSI vs TCP/IP

- OSI prima di implemtazione, TCP/IP dopo
- OSI piu' generale, maggiore indipendenza da protocolli:
 - Alcuni livelli sovraccarichi, altri piu' vuoti
 - Funzioni duplicate
- A livello di network OSI e' connectionless e connection oriented, TCP/IP e' solo connectionless
- A livello di trasporto OSI e' connection-oriented, TCP/IP lascia la scelta all'applicazione
- OSI non e' inizialmente decollato ma e' stato rivalutato. TCP/IP e' difficile da adattare a nuove tecnologie.

2.13 Apparati di rete

- Host : Implementano tutti i 7 livelli
- Router: Implementano i primi 3 livelli
- Bridge e switch: Implementano i livelli 1 e 2
- Hub: Implementano il livello 1: e' un'oggetto che prende un segnale su una delle sue interfacce e le rimanda su tutte le sue interfacce. Ad esempio nel wireless (hub wind). Nel mondo wired posso attaccare un hub ad un computer per vedere tutto quello che gira in rete.

NIC (network interface Card): implementa i livelli 1 e 2. Il trasporto dei dati lo abbiamo già visto, e' quello in rosso nell'immagine.

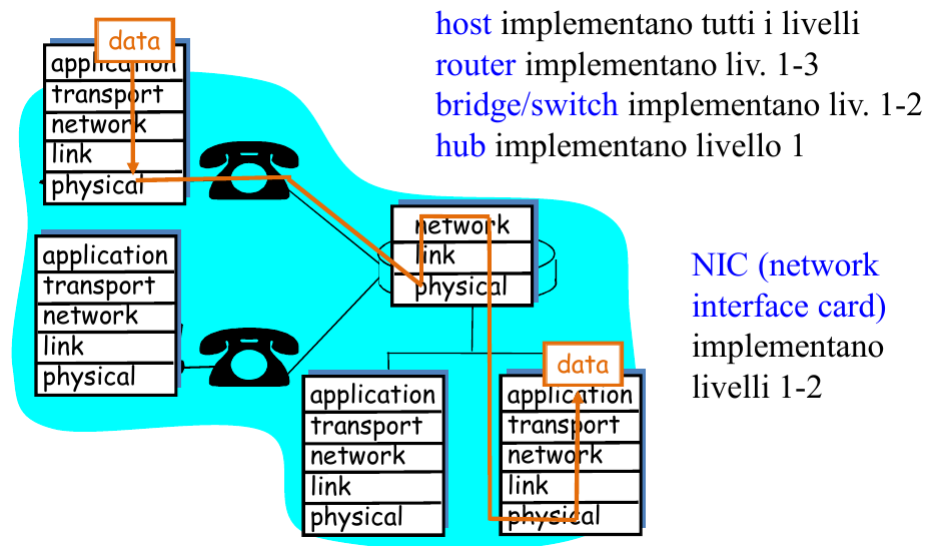


Figure 2.11: Apparati di rete

2.14 Cenni storici

Nacque la prima bozza di rete nel '67, nel '70 e' nata la prima rete satellitare (universita' delle hawaii), nel '75 sono cominciati ad essere diffusi i protocolli TCP/IP. Quello che ci interessa sono i principi guida sulla costruzione delle reti, l'idea e' che quello delle reti e' un mondo selvaggio in cui uno puo' organizzarsi come vuole, nessuno ha vincoli di usare gli stessi protocolli degli altri, nonostante questa scelta autonoma alla fine per fare Internet queste scelte devono convergere per fare in modo che le reti si parlino:

- Autonomia delle reti
- Servizio best effort
- Router stateless: gli apparati di rete devono tenere meno informazioni di stato possibili per non perdere tempo a gestire queste informazioni di stato
- Controllo distribuito: non ci dev'essere un punto in cui c'e' una sola entita' che e' il punto chiave per fare funzionare la rete, perche' se si guasta non funziona piu' tutto.

Sono le idee che hanno guidato lo sviluppo delle reti e che sono ricordati con l'acronimo KISS (keep it simple, stupid!)

2.15 Accesso alla rete

Nelle reti residenziali ci collegiamo attraverso la rete telefonica dell'internet service provider che si collega ad internet. Le reti aziendali sono un misto di chiazze broadcast (LAN) interconnesse da switch o ruter interconnesse in broadcast o punto punto dalle dorsali creando reti intranet, l'interconnessione di reti intranet che creano Internet.

2.15.1 Residenziale

Gli internet service provider sono classificati in vario livello, in funzione delle porzioni di rete che controllano. Quelli di livello 1 hanno copertura internazionale ed e' richiesto che le reti dei livello 1 devono essere tutte connesse. Costituiscono la colonna dorsale di Internet, forniscono via Multiplexing una banda di Tbps (terabit per second) e connettono gli Internet Service Provider di livello 2. Ogni ISP di livello 2 deve essere connesso ad ISP di livello 1 e cosi' via a cascata. C'e' quindi una struttura gerarchica degli Internet Service Provider.

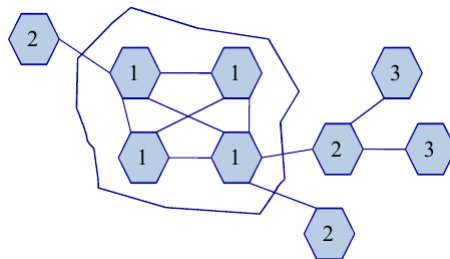


Figure 2.12: Connessioni ISP

I peering point sono i punti di connessione tra ISP di livello diverso. Sono dei punti critici, perche' ci sono avvocati che decidono quanta banda devono usare per trattare il traffico di altri ISP.

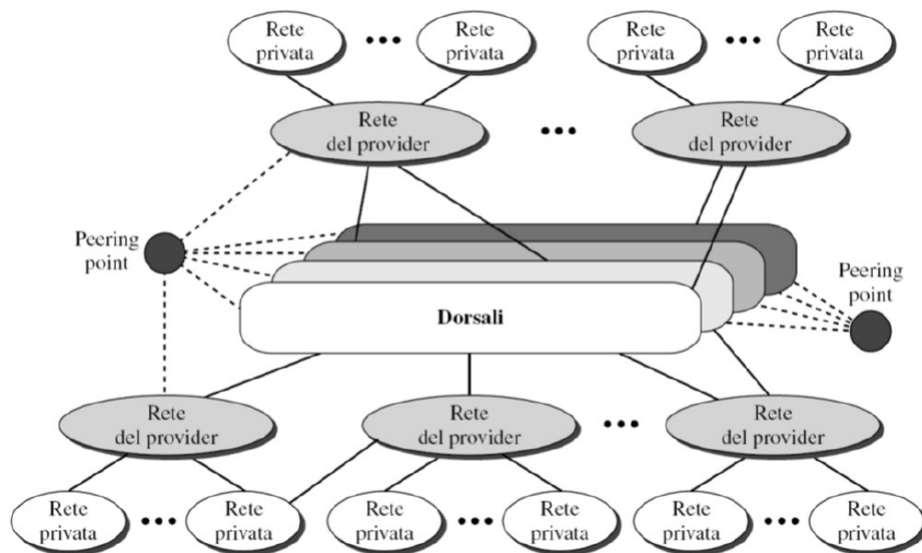


Figure 2.13: Struttura di Internet

2.16 Un po' di terminologia

Wireless e' un termine che si riferisce solo alla tecnologia del mezzo trasmissivo (e' non guidato) il segnale elettromagnetico si diffonde nelle 3 direzioni dello spazio e' puo' essere sentito da chiunque (sono reti broadcast), si riferisce ai livelli 1 e 2 dell'architettura. Mobile e' una caratteristica che e' a livello 7 e sopra, riguarda il fatto che un determinato utente si palesi in rete in punti diversi della topologia di reti di ISP differenti nel corso del tempo (indipendentemente dalla tecnologia). Nomadic e' un termine che dice che siamo mobile e sempre servito da tecnologia wireless.

Wireless	Mobile	Application
NO	NO	Station workstation in office
NO	YES	using a portable in a hotel
YES	NO	W-LAN in unwired building
YES	YES	Portable office

Table 2.3: Esempio

2.17 Reti WIRELESS

Cambiano anche le strutture delle reti, l'immagine si riferisce all'infrastruttura del 5G. Le tecnologie wireless sono tante (NFC e' una tecnologia wireless che permette di comunicare a distanze di cm e pochi byte, il bluetooth permette di

lavorare su raggi di pochi metri, WI-FI che lavora con dei range di 150/200/300 metri, WI-MAX e' una tecnologia che mira a sostituire le dorsali dei provider con una tecnologia radio che ha portata di comunicazione di alcuni km). Per questa tecnologia distribuiamo un segnale radio nello spazio e poi decade con il quadrato della distanza percorsa nel punto piu' alto, ad un certo punto il segnale non e' piu' percepibile a causa dell'interferenza del rumore di sottofondo, che e' sempre presente oltre alle interferenze di altre onde elettromagnetiche. Coesistono tutte queste tecnologie e coesisteranno nell'infrastruttura 5G (classificata come multi radio access technology), si potrebbe considerare la rete wireless che c'e' ed e' piu' strutturata come una sovrapposizione di tante reti costruite da tante tecnologie, ci potranno essere delle celle molto piccole (aree che stanno intorno all'antenna che ricevono e possono far ricevere il segnale all'antenna), la distanza e' del metro. Abbiamo tecnologie che realizzano delle PICO CELLE, con raggio di 10 metri ed hanno bande di alcuni GB/s. Ci possiamo muovere a tecnologie che costituiscono delle W-LAN (wireless local area network) della dimensione geografica paragonabile alle LAN cablate (circa 500 metri, dopo di che ci vuole un ripetitore). Arriviamo a reti in cui le celle sono piu' grandi (WI-FI, WI-MAX) il raggio puo' diventare la dimensione di un'area urbana (alcuni km). Quindi abbiamo gerarchie di aree che usano wireless differenti e ci sono dispositivi che ha interfacce (network interface card) che parlano l'una e l'altra tecnologia che riescono a connettere aree diverse. Fino a qui abbiamo parlato di architettura single-hop, gli utenti si parlano esclusivamente attraverso le antenne, anche due utenti che stanno nel raggio di comunicazione di una stessa antenna non possono parlarsi direttamente. Non e' l'unica infrastruttura che c'e', l'alternativa e' quella di avere reti multi-hop che fa diversi hop attraversando anche una rete cablata che unisce due antenne. Esiste anche la architettura ad hoc, l'infrastruttura fissa (quella costituita dalle antenne) puo' non esistere, non ci sono antenne ma tutti gli end-system degli utenti sono anche dei router e sono capaci di non essere ne sorgente ne destinazione ma sono capaci di prendere dati generati da un end system e destinati ad un altro endsystem e di fare funzioni di router per passare i dati. Nel caso di infrastruttura ad hoc tutto si puo' spostare e la topologia della rete puo' cambiare molto piu' facilmente, ci possono essere partizioni della rete se gli end system sono troppo lontani. Telefonia cellulare e' l'area in cui dispositivi comunicano attraverso la stessa antenna, esistono due problemi:

- Supporto al roaming: e' supportato grazie all'handoff. Ci si puo' muovere grazie al fatto che le antenne dietro si stanno parlando per servire l'utente.
- Problema di handoff: le antenne controllano e tengono traccia di chi sono i dispositivi che stanno nella loro cella, se i dispositivi sono mobili il loro punto di connessione e' internet. Muovendomi esco dal raggio di comunicazione di una cella ed entro nell'altra, questo viene fatto automaticamente dai dispositivi con una certa resistenza, i dispositivi mobili continuano a mandare segnali (beacon, il raggio del faro), questi messaggi sono di controllo per dire a chi c'e' in torno e sente quel messaggio puo' creare un canale di comunicazione con quella tecnologia, le antenne fanno la stessa

cosa. Alternando periodi di ascolto a periodi in cui mandano i messaggi, quando due dispositivi si sentono si agganciano e creano un canale di comunicazione wireless e anche se non c'è comunicazione si continuano a mandare il messaggio di controllo. Quando un dispositivo si muove ad un certo punto può sentire l'identificatore di un'altra antenna, i dispositivi che si stanno muovendo misurano il rapporto segnale/rumore di tutte le antenne che sentono (la chiarezza del segnale rispetto al rumore di fondo che lo può disturbare), se si sono appena accesi si attaccano a quella di cui sentono meglio il segnale. Se sono già agganciati ad un'antenna c'è un po' di resistenza nello sganciarsi da un'antenna ed attaccarsi a quella con rapporto segnale/rumore migliore. Questa resistenza è dovuta al fatto che si vuole impedire che un apparato wireless che sta in un'area intermedia comincia a saltare da una all'altra perché sente i segnali più o meno pari. L'antenna nuova e quella vecchia si parlano effettuando l'handoff, tutte le conversazioni tra utente ed antenna vecchia vengono passate in carico all'antenna nuova, in modo che l'utente senza accorgersi venga supportato nelle sue conversazioni. Le celle si coordinano tra di loro per cambiare punto di accesso.

Le dimensioni della cella sono variabili, le antenne piantate sono un po' di più di quante ne servono, perché stiamo parlando di broadcast, tutti gli altri dispositivi sulla stessa frequenza possono ascoltare e non possono parlare. Quindi l'idea è che le antenne controllano un certo numero di utenti, se ci si accorge che in una determinata cella gli utenti sono molto densi la disponibilità di canale per ciascuno diventa troppo scarsa, quindi le infrastrutture di tecnologie cellulari variano la potenza delle antenne e la quantità di antenne attive in base a quanti utenti ci sono. Se densità in una cella è alta abbasso la potenza ed accendo altre antenne per coprire le aree rimaste scoperte dall'abbassamento di potenza, se è bassa alzo la potenza. È una struttura complicata che usa tecnologie decisamente fragili.

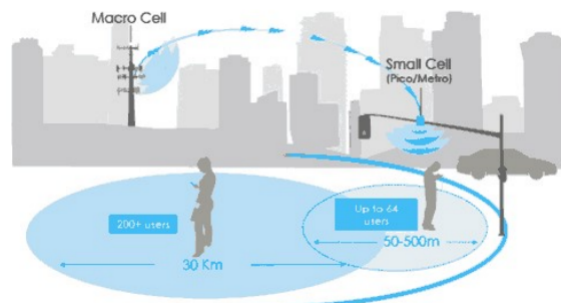


Figure 2.14: Wireless

2.17.1 Accesso alla rete wireless

Devono essere in line of vista. non devono esserci ostacoli nel mezzo della rete wireless, altrimenti il raggio diminuisce di molto.

- Bluetooth: 10-30 metri di raggio
- WI-FI (dipende dal dialetto): 100-150 metri di raggio
- Wireless LAN (W-LAN): wi-fi, 11 Mbps (nominali). Può essere usata in modo infrastrutturato (con un antenna in mezzo) o in modalità ad hoc (dove non c'è un'infrastruttura fisica)
- telefonia mobile: Pagano una licenza allo stato per usare la rete e ci recuperano con le tariffe, LTE prevede che due utenti nella stessa cella si possono parlare direttamente bypassando l'antenna.

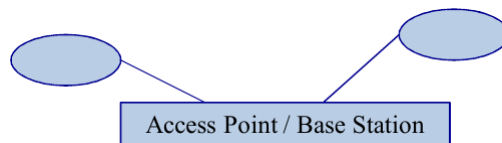


Figure 2.15: Accesso alla rete wireless

2.17.2 Collegamento dispositivi

- beacon: per annunciare la presenza di un antenna
- Associazione tra dispositivo e antenna con negoziazione parametri (tra il range di frequenze di quelle disponibili ci può essere una suddivisione, faccio usare una fetta di frequenze disponibili così non si parlano uno sopra l'altro), ad esempio indirizzo/identificatore nella rete, nel momento in cui viene fatta associazione tra dispositivo mobile ed antenna, antenna può assegnare identificativo a quel dispositivo.
- WiFi: l'access point controlla la comunicazione degli apparati mobili, fonda una rete wifi con due identificatori
 - BSSID : indirizzo dello access point, e' l'indirizzo dell'antenna, se non lo sento piu' significa che non faccio piu' parte di quella rete.
 - SSID: nome della rete, possono comunicare gli apparati che usano lo stesso SSID. Identifica anche il servizio della rete. Può essere un problema di implementazione, in funzione a diverse implementazione la lunghezza può variare, se sono stati presi troppi pochi byte il nome viene troncato e non e' piu' rilevante per l'identificazione della rete.

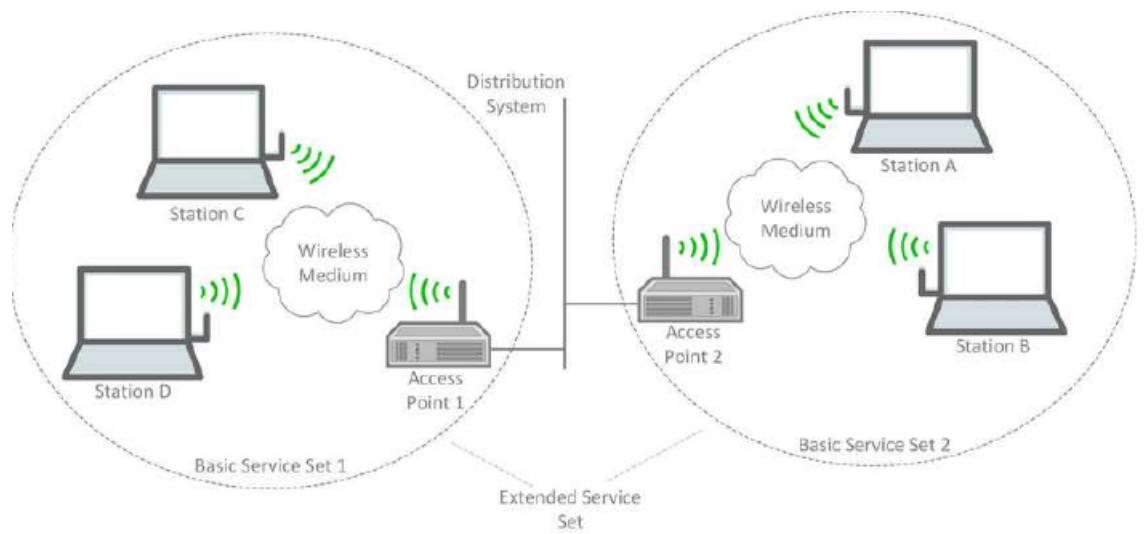


Figure 2.16: Collegamento Dispositivi

Chapter 3

Livello Applicazione

Non dobbiamo confondere le applicazioni con il codice delle applicazioni. Ad esempio i browser sono soltanto applicazioni, non ci forniscono nessun servizio di rete, quello che fa parte della rete e' l'"HTTP://", e' un protocollo che serve per dire al browser che ho bisogno di usare la rete e chiedere un servizio al software di livello applicazione per parlare con il servizio che sta dall'altra parte. Il protocollo di livello applicazione e' quello che viene prima del ://. Le applicazioni sono processi distribuiti per svolgere i loro compiti, possono stare su diversi endsystem, per svolgere le loro funzioni hanno bisogno di usare la rete, per comunicare hanno bisogno di protocolli di livello applicazioni, chiedere un servizio ad una parte del sistema operativo che implementa la rete. Nei browser abbiamo una parte di codice che realizza un cliente di un server web, questi due si parlano via protocollo HTTP. Lo stesso vale per la mail. Protocolli:

- Sono un pezzo dell'applicazione
- Definiscono i messaggi scambiati per le applicazioni e le azioni intraprese nel momento in cui ricevo un messaggio di quel protocollo.
- Usano servizi di comunicazione forniti dai livelli inferiori via **socket**

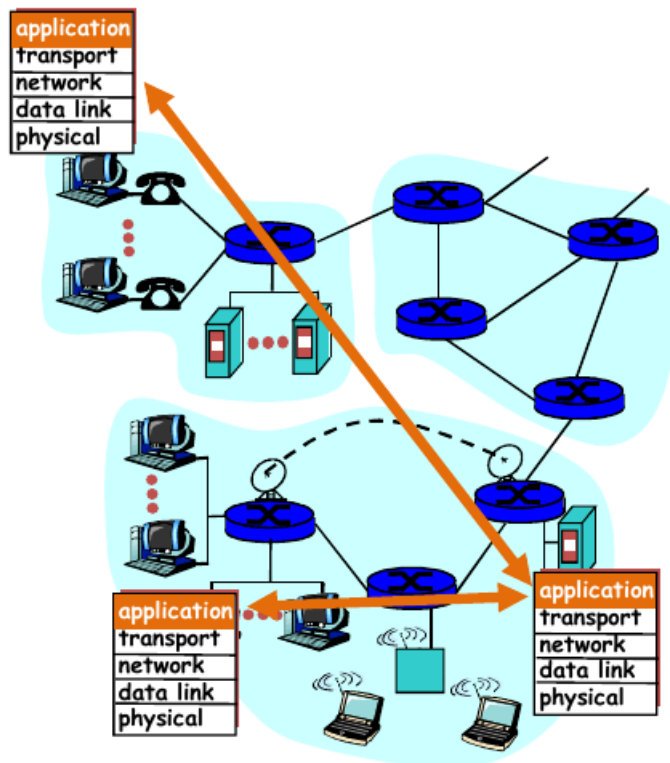


Figure 3.1: Livello Applicazione

3.1 Struttura applicazioni

Da una parte abbiamo l'applicazione (user agent) che si interfaccia con l'utente sopra e la rete sotto. Non fa parte della rete ma implementa il protocollo dell'applicazione. L'user agent e' un processo software che si interfaccia con l'utente sopra e la rete sotto, implementa il protocollo dell'applicazione.

3.2 Architettura generale

I socket sono un'interfaccia tra user agent ed i servizi di rete che ci stanno sotto. Le applicazioni che usano un protocollo di livello applicazione (HTTP) accedono ai livelli piu' bassi della rete tramite le socket. Sono delle librerie di API (Application Programming Interface) o delle librerie di system call (dal punto di vista del sistema operativo) e stanno dal livello 4 o sopra. Stando a quel punto li' della rete non si interessano di cosa c'e' sotto, quindi possiamo programmare con le socket sia che sia un'applicazione per rete wired sia per una app per sistemi mobili. Forniscono comandi alle applicazioni e dall'altro lato chiedono il servizio al protocollo di trasporto o a quello di rete. Forniscono un insieme di comandi che permettono di richiedere i servizi dei protocolli di rete, e modificarne il comportamento. La freccia tra il socket ed il protocollo di livello rete e' particolare, qualunque codice di livello utente puo' accedere tramite socket alle funzionalita' di livello di trasporto. Non qualsiasi applicazione puo' accedere al livello rete, soltanto le applicazioni che sono classificate di root possono utilizzare le socket che permettono di bypassare il livello di trasporto ed arrivare a quelle di rete. In un mondo UNIX solo le applicazioni che sono di gruppo root possono usare i comandi delle socket che permettono di accedere al protocollo di rete. Come funzionano le socket:

1. I nomi dei siti identificano un end system che eroga il servizio ed il processo sullo stesso end system che effettivamente eroga il servizio. Il WWW sta ad identificare il processo di navigazione che fornisce i servizi web. Vanno indirizzati in rete l'end system ed il processo che e' in esecuzione con cui vogliamo parlare. Significa che al livello delle socket un indirizzo che identifica con chi vogliamo parlare e' fatto di :
 - Un indirizzo di rete, identifica un dispositivo che implementa il livello 3 o superiore (router o end system)
 - Un indirizzo di un processo
 - Numero di porta

Le socket sono librerie di comandi che ci permettono di accedere a funzioni di livello 4 e livello 3 se siamo fortunati.

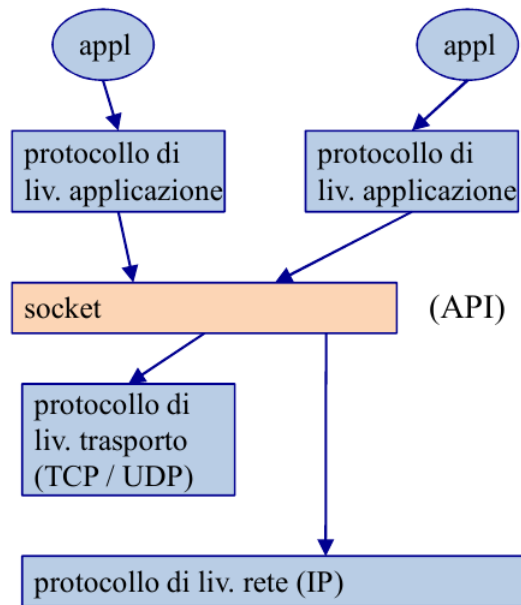


Figure 3.2: Architettura generale

3.2.1 Indirizzamento e (de)Multiplexing

Le porte sono rappresentate su 16 bit considerati i doppi, il sistema distingue un numero di porta x e lo stesso numero di porta che sta comunicando in rete usando il protocollo UDT, possiamo avere comunicanti in rete su uno stesso host molti processi. Non sono tutti usabili. Sui sistemi UNIX troviamo un file `/etc/services` che ci fa vedere un po' di servizi di rete standard con associati i loro numeri di porta che vuol dire che per ogni servizio vediamo la porta che usa ed il tipo di protocollo. Si è deciso che di questo spazio di 65k i valori di numero di porta tra 0-255 sono numeri di porta di servizi standard (quelli che dovrebbero esserci ovunque, ad esempio il servizio HTTP per tutti è la porta 80) questo ci semplifica la comunicazione, perché tutti i dati che arrivano sulla macchina con identificativo 80 vengono passati senza problemi al servizio HTTP. Per fare questa operazione di assegnamento non va bene il process ID perché non è fisso, non ho bisogno di sapere il PID perché qualunque sia il path name dell'eseguibile so che se dico passa questa roba al processo legato alla porta 80 per standardizzazione è per forza il server web. Le porte dalla 256 alla 1023 sono usate per servizi per i quali si sta eseguendo il processo di standardizzazione. Dalla porta 1024 alla 49151 sono servizi registered, nel senso che sono numeri di porta associati non a servizi standard definiti da comunità internazionale, ma sono servizi forniti da grandi aziende ai loro utenti e per evitare collisioni le grandi aziende vanno dall'autorità internazionale si fanno dare un numero di porta specifico per ogni servizio. I numeri di porta superiori a 49151 si chiamano

porte effimere (dinamiche), servono perche' nella rete c'e' una disparita' tra cliente e fornitori, i server devono sempre essere disponibili ad aspettare che qualcuno li richieda, i client non sono sempre li' ad aspettare, quando accediamo in rete tramite un client possiamo farci dare un numero di porta specifico basta che sul nostro sistema non ci sia un'altro servizio con lo stesso numero di porta. Il mio processo riempie l'HTTP si presenta al server a chiedere un contenuto e nel contempo gli dice che e' il processo che risiede a quell'end system con quel numero di porta, il server sa quindi rispondere perche' riceve l'indirizzo con la richiesta. sono porte effimere perche' durano tanto quanto il cliente ha bisogno di stare in rete, dinamiche perche' sono assegnate dal sistema operativo tornano libere e vengono riassegnate ad altri processi. Per riassumere il port Id e' unico e legato ad un processo specifico:

- in TCP/IP e' rappresentato come un unsigned short su 16 bit
- Le porte dalla 0 alla 255 sono usate per servizi standard
- Le porte dalla 256 alla 1023 sono usate per i servizi in test
- Le porte dalla 1024 alla 49151 sono usate per i servizi Registered
- Le porte superiori sono effimere (dinamiche)

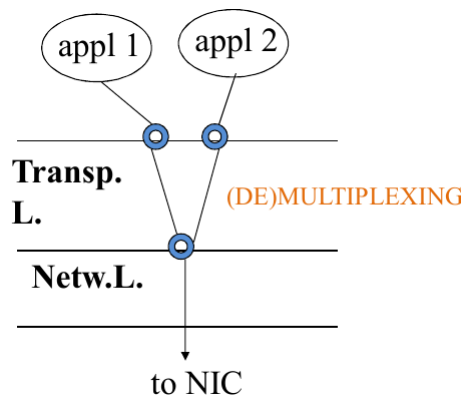


Figure 3.3: Indirizzamento

3.2.2 Primitive di servizio (delle socket)

Dobbiamo ricollegarci ai sistemi operativi, siamo nel kernel di un sistema operativo perche' stiamo chiedendo un servizio al sistema operativo. Prima di tutto dobbiamo creare una socket tramite system call, le socket ci fanno creare un punto di accesso ai servizi del livello 4 (livello di trasporto) in senso verticale, in senso orizzontale una socket e' il punto terminale del canale di comunicazione virtuale dell'entita' di pari livello sull'altro end-system. I parametri che dobbiamo specificare nella creazione di una socket sono:

- family: e' la famiglia di protocolli che dobbiamo usare: si possono usare sia tra processi sullo stesso host sia su host in rete, sono piu' flessibile delle FIFO (First In First Out), ci permettono di gestire le code e scegliere il servizio di trasporto (o connectionless o connection-oriented), se facciamo comunicare processi sulla stessa macchina la family e' AF_UNIX (e' una macro che fa riferimento ad un valore numerico, quindi il sistema capisce che ci stiamo riferendo a quei protocolli, AF sta per address family), per la comunicazione tra processi su end-system diversi abbiamo AF_INET.
- type: ci dice che tipo di servizio vogliamo, i servizi si distinguono in:
 - DATAGRAM: e' un servizio connectionless inaffidabile
 - STREAM: e' un servizio connection oriented affidabile
- protocol: OSI se e' datagram e IP se e' stream

Materialmente nel sistema quando facciamo la call socket: il sistema operativo per ogni processo in esecuzione si mantiene il process control block (dove c'e' dentro tutto), qui dentro abbiamo una tabella dei file aperti dentro cui ci sono 3 descrittori di default (0,1,2). Non sono altro che le posizioni 0,1,2 in questa tabella che e' un array, all'interno c'e' un puntatore ad una tabella non piu' privata al processo ma globale al sistema che ricorda tutti i file in uso da parte di tutti i processi in esecuzione e ricorda come accedervi. L'estremo del canale di comunicazione e' rappresentato come una posizione nella file description table (come fosse un file aperto, perche' effettivamente dobbiamo scrivere e leggere dati), quindi la socket torna un valore intero che e' la posizione nella table da cui parte la catena di puntatori (ad esempio: 5, nella quinta posizione dell'array c'e' un puntatore che va nella description file table, che punta un'area di memoria che ci serve come buffer di comunicazione tra livelli). Quindi il sistema operativo si crea un buffer, uno per i dati passati dal livello applicazione alla rete ed uno per il percorso inverso. Stiamo pre-allertando il sistema operativo dicendogli che ci serve ad esempio un protocollo UDT di tipo datagram. La seconda cosa che facciamo e' usare un metodo (system call o API) BIND(), lega la entry della global file table a cui si arriva dalla socket (posizione nella file description table) alle posizioni di memoria in cui possiamo leggere e scrivere, inoltre lega anche un'indirizzo (per i dati che scriverai in rete verrai identificato come il servizio che sta su quell'host con un determinato numero di porta) ed un numero di porta. Il passaggio del numero di porta si fa con il parametro che passiamo alla bind, se mettiamo 0 ci da una porta effimera (la prima libera).

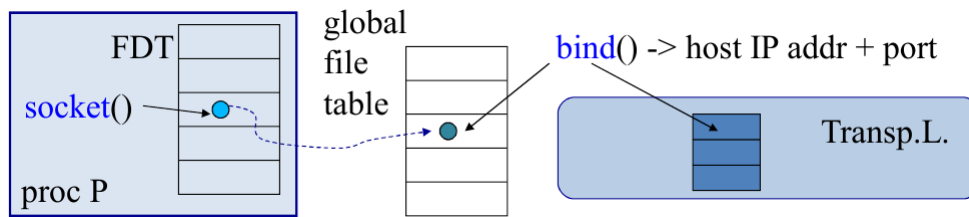


Figure 3.4: Primitive di servizio

TCP

Le due parti (i diversi end-system) devono mettersi d'accordo su un certo numero di cose, dobbiamo prima creare la connessione e lo facciamo tramite le system call:

- liste: ha un'azione locale, e' eseguita soltanto dal server. Dice al sistema operativo: "guarda che quel processo vuole agire da server" (quindi sara' passivo aspettando che altri gli chiedano qualcosa). Dato che potrebbero arrivare piu' utenti contemporaneamente crea una piccola coda per mettere le richieste di comunicazione con il server in modo che non vengano sovrascritte.
- connect: viene chiamata dal client e specifica l'indirizzo del server con cui vuole parlare, fa partire la costruzione della connessione. Manda a lato server una richiesta di connessione che viene messa nella buffer della listen.
- accept: guarda nella coda della listen, se ci sono richieste prende la prima (piu' antica) e la elabora per gestire la comunicazione. Se e' vuota la accept diventa bloccante, va in waiting e non fa nulla.

Al termine di questa costruzione ricordano il canale di comunicazione da una quintupla **associazione ;protocollo in uso(TCP), IPsrc(sorgente), port-src(sorgente), IPdst(destinazione), port-dst(destinazione);**. Non posso usare la socket per comunicare con nessun'altro quindi non e' piu' necessario dire da chi si riceve ed a chi invia, la sorgente tratta solo con la parte remota, manda semplicemente i dati al livello di trasporto. Per ogni connessione TCP ricorda un connection record.



Figure 3.5: Connessione

Nel caso di servizio connectoin-oriented si usano le socket read e write che sono uguali a quelli di lettura e scrittura da/al file. Quando le due parti hanno

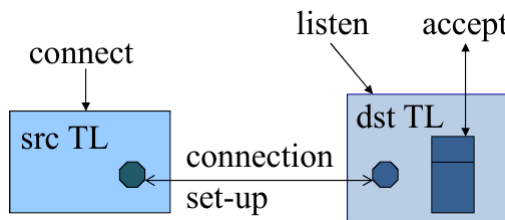


Figure 3.6: Connection set-up

finito di parlarsi devono invocare una *close*, la quale distrugge il canale di comunicazione e fa dimenticare al livello di trasporto tutte le informazioni della socket e la catena di puntatori. Nel caso *connectionless* inaffidabile (servizio UDP) non possiamo dire *read* *write*, in questo caso posso usando la stessa socket parlare con parti remote diverse ogni volta che invio un messaggio, devo usare delle altre system call *sendTo* e *recvFrom* (non vuol dire voglio ricevere dati da questa sorgente). In entrambe le situazioni. Rimane a carico del programmatore di applicazione definire il protocollo dell'applicazione che sta implementando, quindi implementare il formato dei messaggi (che verranno incapsulati con gli header), che formato, che tipi di dati, in che ordine nel messaggio.

3.3 Architettura client-server

Lato server:

- mette a disposizione risorse
- sempre in attesa di clienti, ha un indirizzo ben noto
- deve:
 - Controllare la sicurezza, lo fa tramite il protocollo AAA:
 - * Authentication: quando arriva la richiesta di un cliente devo essere certo della sua identità
 - * Autorization: una volta capito chi è il cliente devo capire cos'è autorizzato a fare.
 - * Accounting: contabilizziamo quante risorse del server sta utilizzando.
 - Gestire più clienti contemporaneamente: si deve avere l'illusione di avere un server dedicato per ognuno.
- Server farm: ci sono tanti server nella stessa posizione, potrebbe essere nascosto nel termine di cloud. Esiste anche il concetto dei mirror in cui i server sono in giro per il mondo e mantenuti allineati.

Lato client:

- Usa le risorse, inizia la comunicazione presentandosi al server con un indirizzo.
- L'indirizzo del client puo' essere qualsiasi, serve solo per la risposta
- Il client puo' terminare quando vuole ed avere qualunque problema e questi problemi non devono interferire con gli altri server

3.4 Architetture Peer-to-Peer

Ogni componente e' contemporaneamente client o server, in ogni nodo ci devono essere le funzionalita' di tutte le parti con la complessita' del server. Lavoro distribuito, piu' resistente a guasti e malfunzionamento. Non guardiamo i problemi.

3.5 Servizi a livello 7

La cosa che ci interessa e' che sia alta l'affidabilita' quando scarichiamo un file. L'email e' asincrona quindi non ha senso parlare di latenza. Il web necessita di una latenza media con affidabilita' alta. Per l'affidabilita' alta usiamo il protocollo TCP. Quando invece dobbiamo avere una latenza bassa ed un'affidabilita' medio/bassa usiamo il protocollo UDP.

APPLICAZIONE	LATENZA	BANDA	AFFIDABILITA'	PROTO L7	PROTO L4
trasferimento file	alta	varia	alta	FTP	TCP
e-mail	NO	varia	alta	SMTP	TCP
web	media	varia	alta	HTTP	TCP
A/V streaming	100 ms.	fino a 5 Mbps	bassa (?)	proprietario	UDP
A/V buffered	sec.	fino a 5 Mbps	media	proprietario	UDP
giochi interattivi	100 ms.	fino a 10 Kbps	media	proprietario	UDP
instant messaging	media	varia	alta	proprietario	TCP
terminale remoto	sec.	bassa	alta	telenet, ssh	TCP
telefonia	100 ms.	bassa	bassa	Skype	UDP
file remoti	ms.	varia	alta	NFS	UDP/TCP

Table 3.1: Servizi a livello 7

3.5.1 Condivisione di file

- Approccio on-line (NFS)
 - modifiche immediatamente visibili a tutti
 - problema di sincronizzazione tra utenti e di bottleneck al server

- Approccio off-line (FTP, HTTP)
 - Copia replicata su host utente
 - C'e' un problema di riconciliazione delle copie
 - Trasferimento batch vs interattivo :
 - * Batch: noto e con meno latenza
 - * Interattivo: risolvo subito gli errori

FTP e' un esempio di trasferimento interattivo.

3.6 WWW

Lo user agent e' qualunque browser. Ci fa accedere a svariati contenuti di svariati tipi (testo, immagine, audio, video), il testo stesso puo' essere ASCII o contenente hyperinformazioni, dobbiamo usare i browser per usare il protocollo dell'applicazione che chiede ai server di accedere ai contenuti. Indirizzamento: **Uniform Resource Locator (URL)**:

- protocollo di comunicazione://hostname:port#/pathname. Il nome dell'host tendenzialmente e' simbolico o un indirizzo di livello network (numerico). Specificare la porta e' opzionale, si potrebbe rendere disponibile un servizio web solo ad alcuni end-system. Indicando i pathname (contenuto del file system) a cui vogliamo accedere. A volte qualche cosa e' dato per scontato, se non si specifica il pathname cerca sempre il file index.html.

Servizi:

- **Pull**: tutti i protocolli in cui il cliente deve attirare a se i contenuti dal server. HTTP e' un servizio PULL.
- **Push**: protocolli e servizi in cui e' il server che spinge il contenuto verso il cliente. Quelli che si sono registrati (la mail)

3.7 HTTP: architettura

E' il protocollo che e' usato per la navigazione nel web. Ricordiamo che il protocollo e' un insieme di regole per cui le parti comunicanti riescono ad intendersi inoltre ricordiamo che siamo nel livello applicazione. Tramite il browser non chiediamo molte funzioni (non e' vero in HTTPS in cui chiediamo servizi di sicurezza, quindi sono servizi di livello 6 (sessione)). Chiede servizi al TCP (transfer control protocol) e' connection oriented ed affidabile, perche' gli utenti stanno scaricando dei contenuti, siamo disponibili come umani a portare pazienza per avere il contenuto ma dev'essere perfetto. Attenzione che quando il contenuto da scaricare dal web e' uno streaming audio e video che la sorgente sta producendo al momento, non stiamo utilizzando il server web ma lui delega un'altro server

dedicato per inviarti quel contenuto. Quando un cliente HTTP richiede un contenuto al server HTTP il server capisce cosa sa riprodurre il client, capito questo il server manda il comando al server dei contenuti multimediali perché lui peschi dal suo repository e poi mandi direttamente al client il contenuto desiderato nel formato desiderato fruibile dal cliente. Di per sé il dialogo tra client-server HTTP utilizza TCP (connection oriented affidabile) ma la latenza (streaming a parte) accettiamo che possa essere un po' più alta perché ci portiamo la copia del contenuto in locale. Il browser è uno user agent, è capace di formulare le richieste per il server HTTP in modo corretto, perché ingloba anche il protocollo di livello 7 cliente e quello capace di fare la richiesta, parla con il server, il quale ha il suo filesystem. Virtualmente il client e server si scambiano messaggi HTTP. È un servizio standard quindi ha sempre la porta 80, è un protocollo pull, può trasferire contenuti distribuiti su più host, nel momento in cui vediamo una pagina web non è detto che sia tutto lì, l'esempio classico è una pagina contenente banner pubblicitari i quali stanno su altri server, questo significa che contattiamo anche altri server web, tutto questo noi non lo vediamo. Inoltre è stateless, che significa che non ricorda informazioni sull'utente che sta navigando, i cookies sono un'aggiunta posteriore al protocollo HTTP che servono per aggirare questa caratteristica, significa che se navighiamo in una serie di pagine del browser in assenza di cookies il server su cui navighiamo non si ricorda la sequenza di pagine che visitiamo, non si ricorda nel tempo i campi di form che possiamo riempire, non si ricorda se introduciamo informazioni di identificazione (user name e password), può essere un po' scomodo perché se noi accediamo più volte ad un sito web non si ricorda nulla, significa anche che stiamo attraversando il filesystem del server ma tutti i pathname sono assoluti (partono dalla radice del filesystem).

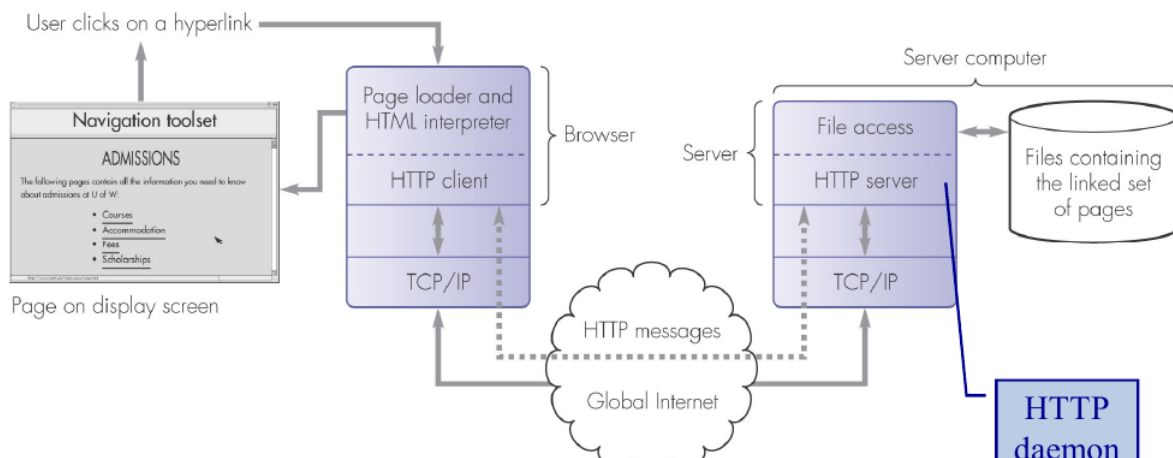


Figure 3.7: Architettura HTTP

3.7.1 Funzionamento

Se lo guardiamo piu' in dettaglio nell'immagine 3.8 abbiamo l'architettura che ci mostra l'interazione di HTTP con un'altro protocollo di livello 7, il domain name system. Troviamo il browser che ha lo user agent, lo user agent chiede servizio a due protocolli. Il browser dialoga con il resolver, e' un servizio che serve a trovare l'indirizzo di livello network di un nome simbolico, il resolver ha un processo sulla macchina dell'utente che e' un cliente del sistema che serve in internet per trasformare i nomi simbolici in indirizzi network, prima cerca in locale se gia' esiste una corrispondenza (nei sistemi UNIX cerca nel file: /etc/hosts che contengono nomi simbolici o indirizzi ip dei server usati frequentemente). Quindi un utente umano mette un nome simbolico, il browser chiede al resolver di trovare una corrispondenza, se non lo trova in locale attraverso un protocollo UDP va ad interrogare un server del DNS. E' un servizio che e' standard e legato sempre alla porta 53 con servizio UDP. Interroga il domain name system gli ritorna l'indirizzo rete corrispondente, il browser passa il numero al client HTTP il quale utilizzando servizi TCP manda la richiesta al server web (porta 80) il quale a sua volta rimanda la risposta. In-band significa che (rispetto a FTP) HTTP mantiene una sola connessione TCP su cui viaggiano i comandi (le richieste) sia i dati veri e propri (i dati che vengono trasferiti).

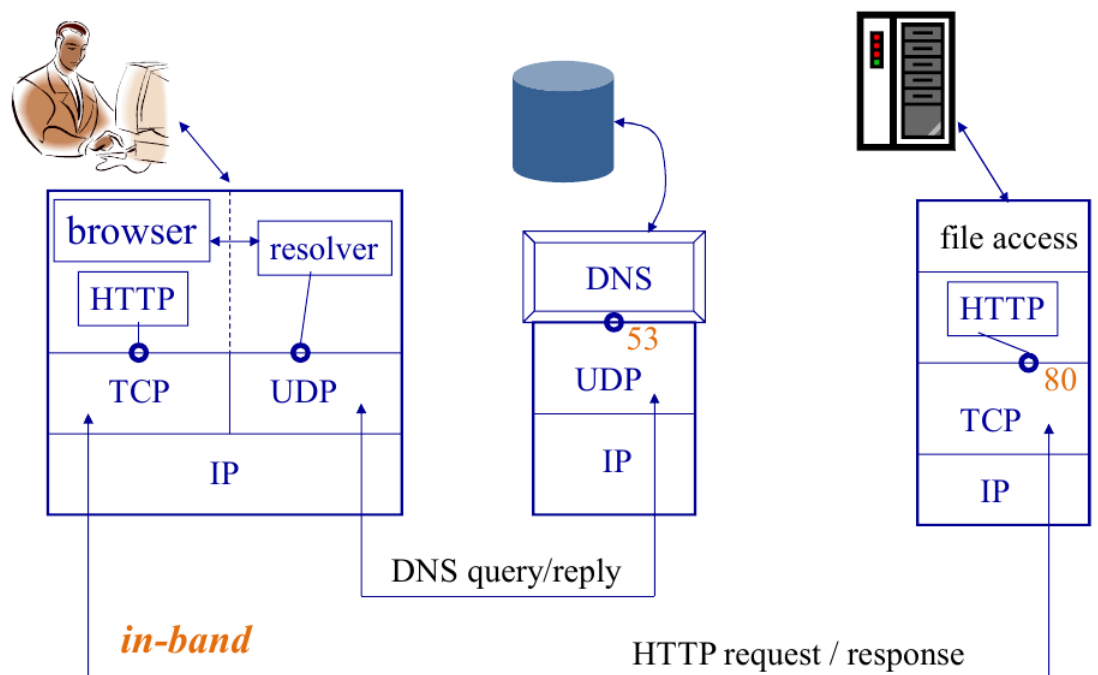


Figure 3.8: HTTP funzionamento

3.7.2 Formato messaggi richiesta

Vediamo ora com'è formato il messaggio di richiesta di contenuti HTTP. C'è una prima riga in cui il client dice al server il metodo (operazione che il client chiede al server di eseguire o vuole eseguire sul server), URL e la versione dell'HTTP, in due versioni diverse di HTTP le cose vengono comunicate in un certo modo nella versione nuova si usa un'altro simbolo, di solito i client implementano una sola versione di HTTP mentre i server sono capaci di capire più versioni. I metodi sono le operazioni ed i più importanti sono:

- GET `{filename}`: per ottenere il contenuto nella URL
- HEAD `{filename}`: richiede soltanto le header del contenuto, della pagina web che dobbiamo scaricare. Nei file HTML c'è l'header che da informazioni sul file HTML che stiamo guardando, in alcuni casi può essere utile chiedere solo l'header
- PUT `{filename}`: io posso (se autenticato come cliente autorizzato) aggiungere un file nel filesystem del server
- POST `{filename}`: permette di inviare le informazioni che abbiamo inserito in un form, il client non rimanda indietro al server tutta la pagina, ma rimanda indietro solo i campi. Può essere fatto con il metodo post seguito da un file (salvato in locale) con i campi della form, oppure vedere una URL con un punto di domanda ed una coppia nome-valore.
- DELETE `{filename}`: io posso (se autenticato come cliente autorizzato) eliminare un file nel filesystem del server

Le righe successive sono coppie (nome campo-valore) che possono dire varie cose, sono di solito informazioni su quello che il cliente sa fare, ad esempio ci dice la risoluzione dello schermo del cliente. Il dispositivo che ha fatto questa richiesta sa ad esempio visualizzare jpeg, png ma non gif, questo significa che il server può avere lo stesso contenuto in molteplici formati e quando viene interpellato va a pescare il contenuto giusto mettendo insieme le componenti (dell'hypertexto) in modo che ogni componente sia comprensibile dal cliente. Riga vuota. Poi ci può essere qualcos'altro usato dal metodo post in cui mettiamo ad esempio le informazioni di una form.

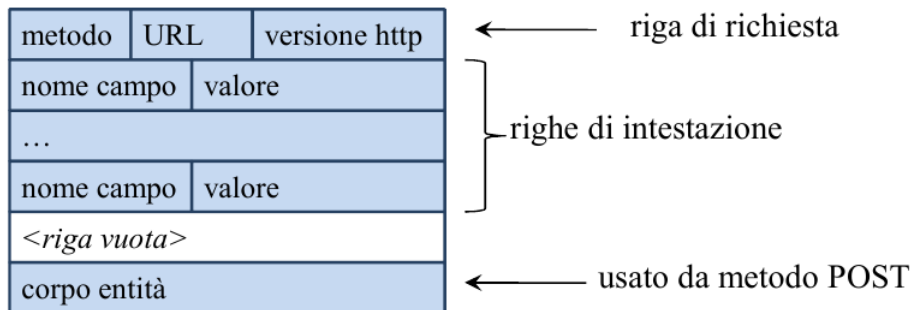


Figure 3.9: Formato messaggi di richiesta HTTP

3.7.3 Formato risposta

- riga di stato: la versione ci dice che sa parlare quell'HTTP, il codice stato ci dice se va tutto bene o ci sono degli errori :
 - 301 Move Permanently
 - 400 Bad Request: non ho capito la richiesta
 - 404 Not Found
 - 505 HTTP Version Not Supported: non sa parlare l'HTTP del client
- Righe di intestazione: coppa nome campo-valore, quando il contenuto torna indietro al client lui sa capace di chiamare le application per aprire ad esempio le immagini png tutte le volte che servono. E' una descrizione del contenuto ed e' simile a quello fatto nella posta elettronica.
- Riga vuota
- Corpo: il contenuto che il client ha richiesto

L'header e' simile al MIME (delle e-mail) per la descrizione del contenuto.

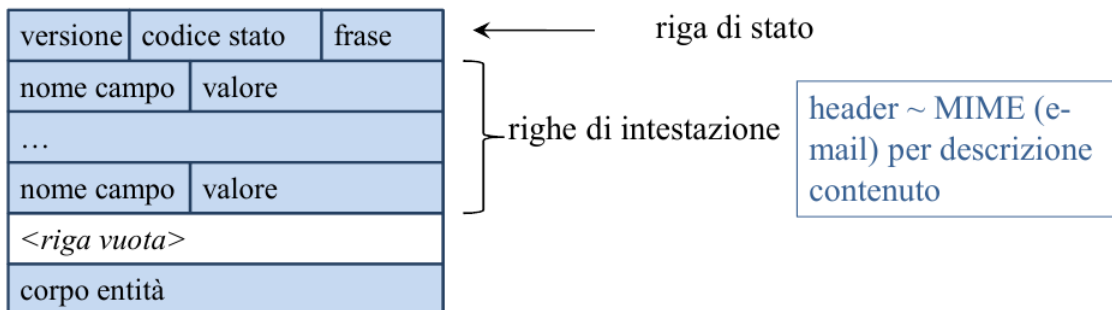


Figure 3.10: Formato messaggi di risposta HTTP

3.7.4 HTTP: full request / response

E' un esempio di dialogo client-server in HTTP. Richiesta:

- Vogliamo che il browser utilizzi il protocollo HTTP, deve contattare il server con quel nome simbolico e su quel server deve pescare quel determinato file
- Esempi di coppie nome-valore:
 - Metodo Get+URL.versione HTTP, e' come se il browser spaccasse in due l'indirizzo, la prima parte e' usata per avere l'indirizzo network e poi il browser passa al livello 7 i dati che sono messi nel payload e la prima parte viene mandata come informazione di controllo che salta giu' di livello in livello fino al livello network il quale mette l'indirizzo numerico al posto di quello simbolico.
 - Connection:close, noi vogliamo che la connessione TCP dopo transazione venga chiusa. Avere il client che dice di chiudere la connessione TCP equivale a dire al server di dimenticarsi del client e svuotare la memoria per parlare con me, e' un favore fatto al server.
 - User-agent e' un determinato tipo di browser name/version number
 - Accept: quello che il client e' capace di capire

Risposta:

- Sa parlare l'HTTP ed il codice 200 dice che va tutto bene
- Server: Aname, ci viene ridato il nome simbolico del server nel caso in cui la richiesta sia stata rediretta da un altro server, ed e' importante sapere quale server ci ha effettivamente servito
- Il nome della pagina che ci sta mandando
- Quando il contenuto e' stato modificato
- Le caratteristiche del contenuto: la lunghezza sono il numero di caratteri ASCII codificati su 7bit

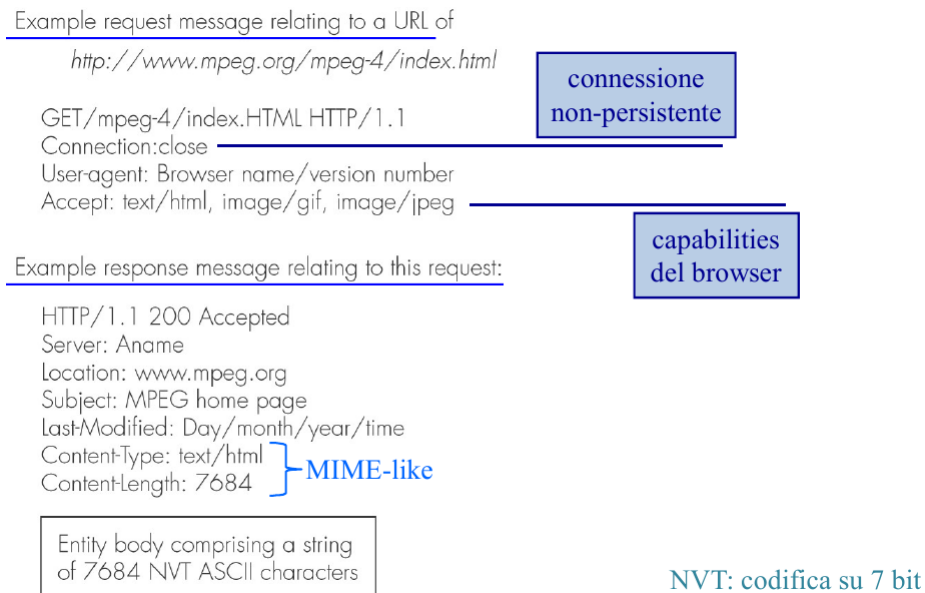


Figure 3.11: Esempio di comunicazione

3.7.5 Trasferimento contenuti

Un contenuto e' un insieme di oggetti (HTML, immagini, audio ...). Quello che veniva fatto nelle prime versioni di HTTP il client (C, l'inclinazione serve per ricordarsi che ci sono tutte le componenti di ritardo) manda al server una richiesta di apertura di una connessione TCP, dopo che e' aperta il cliente manda il messaggio HTTP di livello 7 incapsulato ed e' una richiesta di ottenere un contenuto, il server riceve la richiesta e rimanda indietro il file (index.html) richiesto dal client. Il client deve ragionarci un'attimo quindi il server chiude la connessione, il client capisce se tutto quello che contiene il file e' arrivato, allora il client per ogni componente incluso nel file di ipertesto il file interroga il server su cui la componente sta per andarla a recuperare aprendo e chiudendo la connessione TCP. Round-trip time e' il tempo per fare avere i dati del cliente al server e perche' la risposta del server torni al client, e' la somma di tutti i ritardi che ci sono tra client e server (elaborazione su end-system e anche nel trasferimento), dobbiamo moltiplicare la latenza perche' il client manda la prima richiesta a quando ha messo insieme le componenti round-trip time * 3 per ogni contenuto che c'e' nel file, puo' non essere poco, oltre alla rete calcoliamo il peso del processing sul server.

3.7.6 HTTP discussione

Una prima cosa che si e' fatta per cercare di ridurre la latenza osservata dal client e' di considerare le operazioni che un client deve fare per recuperare un pezzo

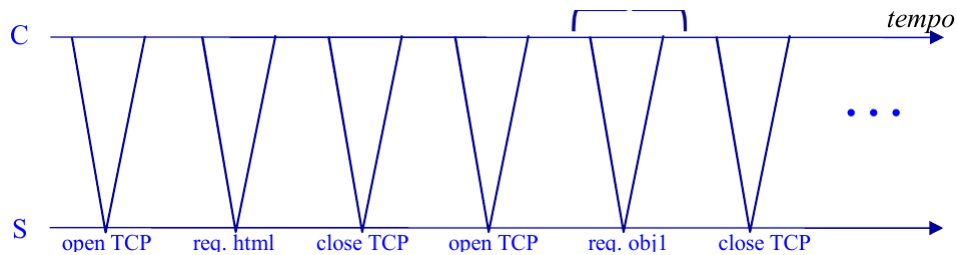


Figure 3.12: Trasferimento Contenuti

dell'ipertesto come un'entità atomica ed anche se il server ci sta mandando la risposta spezzata possiamo quando è arrivato al client un pezzo di testo aprire subito una nuova connessione TCP con lo stesso od un altro server per richiedere subito gli oggetti che il client deve recuperare. L'aspetto negativo è che i client deve tenersi informazioni di tutte le connessioni TCP aperte, ed eventualmente anche il server. Comunque anche parallelizzando per ogni oggetto aprono e chiudono una connessione, ma se gli oggetti sono su un unico server posso far viaggiare tutto sulla stessa connessione (tanti messaggi di controllo per gestire le connessioni), quindi perdiamo tempo aprendo tante connessioni. Dalla versione 1.1 di HTTP si creano connessioni persistenti nella connessione di un server.

3.7.7 Contenuti 2

Intanto che il server manda pezzi dell'ipertesto al client, il client può cominciare a processare e rendersi conto che c'è un oggetto e manda sulla stessa connessione la richiesta per quell'oggetto. Abbiamo messaggi di controllo solo all'inizio, le connessioni sono considerate persistenti perché nel momento in cui il client si collega con il server ci resta collegato anche quando abbiamo ricevuto tutte le parti fino a che seguendo il link nella sua navigazione oppure mettendo un'altra stringa nel browser il client non dice che si sposta su un altro server, oppure l'utente umano non naviga su altri server ma per un po' di tempo non naviga proprio, non ci sono richieste che chiedono qualcosa di nuovo al server la connessione viene chiusa dal server. I vantaggi sono un risparmio di tempo per le aperture e chiusure delle connessioni, risparmio tempo sul parallelismo, riduco messaggi di controllo e di stato e posso fare il pipelining delle richieste.

3.7.8 HTTP: cookies

I cookies non cambiano in niente il formato dei messaggi. Se il server è capace di gestire i cookies nella sua risposta in uno delle coppie nome campo-valore una ha "setcookie:valore". Il client ha iniziato a navigare su un server che tiene in cookie, l'ha contattato ed il server si ricorda che l'utente con quell'indirizzo ha richiesto l'accesso a quel contenuto, il server se le ricorda in un database interno (al server) NON le manda in giro. Dobbiamo pensare ad un database come una serie di righe, in ogni riga ci sono informazioni e posso avere una

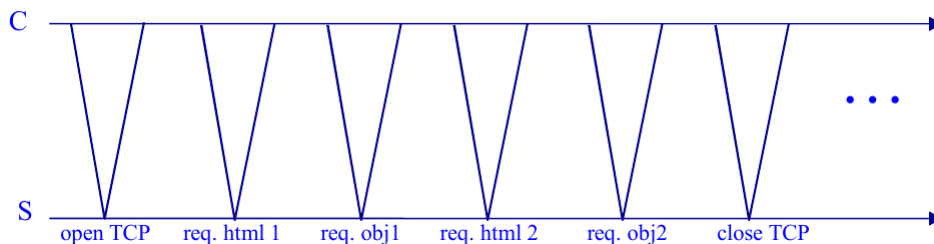


Figure 3.13: Trasferimento contenuti pieling

chiave per cui cerco il record (riga) il cui valore del campo chiave e' quello li'. Nel nostro caso il server ricorda nel record tutte le informazioni che vuole dei clienti che hanno navigato su di lui (log-in name, password ...) ma sono ricordati solo internamente al server, associa ad ogni cliente un'identificatore di cliente che fa il ruolo della chiave del record, il server torna indietro al cliente nella sua risposta una coppia "set-cookie:valore identificatore cliente;", noi non sappiamo e non vediamo le informazioni che il server mantiene su di noi perche' NON CE LE TORNA INDIETRO, l'unica cosa che torna indietro nei cookie e' un identificatore del cliente che e' piccola per non occupare spazio. Nel seguito delle interazioni browser-server, quando fa partire una richiesta puo' mettere in un nome campo il valore del cookie che mi ha restituito prima con cui il server va a pescare nel suo database. I cookie sono nati per facilitare il cliente, per non continuare causa stateless ad inserire sempre le informazioni, oggi vengono usati per marketing, le nostre informazioni vengono vendute a terze parti, ci sono societa' di marketing che pagano chi raccoglie questi dati (i nostri interessi) che vengono usati per fare marketing e rivenduti ad altre aziende si chiama data mining. I dati secondo la legge dovrebbero essere anonimi, anonimizzati e passati in modo aggregato.

3.7.9 HTTP:caching

Proxy vuol dire qualcuno che svolge il compito al posto di qualcun'altro, oltre ai server HTTP proxy esistono anche altri proxy (HCP, ARP). In questo caso l'idea che c'e' dietro e' di localita' ed il concetto di localita' si collega ai sistemi operativi quando si parla di algoritmi di paginazione, in os localita' significa che quando un programma lavora su strutture dati lavora su un sottoinsieme di dati dichiarati e non sui dati globali. Il concetto di localita' c'e' anche in Internet, l'interesse degli utenti e' distribuito su sottoinsieme di contenuti del web, e' ancora piu' vero se consideriamo solo gli utenti di una comunita' (dipartimento, azienda, ateneo...) in quanto utenti con interessi simili sono interessati agli stessi contenuti. L'idea di proxy e caching: se dobbiamo navigare su un server lontano la somma dei ritardi hop per hop ci pesano di piu' rispetto se dobbiamo accedere a server geograficamente vicini. Vicino ad una comunita' di utenti piazziamo un server che cerca di aiutare la navigazione facendo un po' di lavoro per i server web in giro per il mondo. Supponiamo di avere una comunita'

omogenea per interessi, ad esempio i docenti di informatica, allora in vicinanza della comunit  piazziamo un server che fa il proxy http in questo modo: quando un utente della comunit  cerca un contenuto nel web (WWW) la sua richiesta http non va direttamente al server desiderato (se abbiamo configurato un proxy nel browser), bens  la manda al proxy che ha configurato all'interno che   quello che serve la comunit , il proxy guarda nella propria cache se quel contenuto ce l'ha gi , se sono il primo della comunit  a cercare il contenuto non c'  l'ha e va a recuperarlo dal server remoto, lo porta in cache e se lo ricorda poi lo manda a chi ha fatto la richiesta originaria. Nel momento in cui qualcun'altro della comunit  richiede quel contenuto e se sul suo browser ha configurato il proxy manda la richiesta al server proxy che ha gi  il contenuto in cache e quindi togliendosi la latenza (data dalla distanza geografica del server che ha il contenuto richiesto) trova il contenuto erogato da un server vicino. Il proxy HTTP fa quindi caching di contenuti, riesce ad erogare i contenuti con minore latenza perch    vicino a chi fa richieste, riduce il traffico in rete. Se il proxy ha il contenuto lo fornisce altrimenti lo richiede al server remoto e lo porta in cache. Ha un doppio ruolo: se lo configuro sono il cliente e lui   il fornitore, ma se il proxy il contenuto non c'  l'ha si spaccia per client e si fa dare il contenuto da un altro server. Si puo' costruire una gerarchia di server proxy, ad esempio c'  un server di dipartimento che prima di cercare lontano puo' riferirsi prima al server proxy di ateneo. Nel momento in cui porto un contenuto in un server proxy   una copia locale (non confondiamo il server proxy con la cache del nostro browser, il proxy server non   ovviamente sul nostro pc   un server che sta da qualche parte) e puo' essere difforme da quella sul server originario perch  potrebbe essere stato modificato, anche qui il proxy ci da' un vantaggio perch  utilizza il metodo Head:

Metodo Head o Get condizionale

Il server proxy fa una Get condizionale. Il browser dell'utente chiede un determinato contenuto (con un determinato pathname che sta in un determinato server), il browser cerca il contenuto ed ha un proxy configurato, prende la URL e la incapsula in una delle richiesta HTTP che va al proxy (freccia 1). Il proxy (dipende da come l'ente che amministra il server lo ha configurato), se ha una copia del contenuto puo' erogarla direttamente, puo' anche guardare lo header del file e puo' o chiedere solo lo header al server che possiede il file o fare una get condizionale, puo' decidere di chiedere al server originale se il contenuto con quel file name che lui possiede   stato modificato dopo una certa data:

- Get condizionale: tra le coppie nome valore abbiamo: if modified Since: valore che il proxy server trova nella copia che possiede localmente
- Mandare al server originale un metodo head mandando solo l'intestazione del contenuto.

In caso di Get condizionale il server che la riceve cerca il contenuto, lo trova, guarda la data dell'ultima modifica e restituisce il contenuto al proxy se la data

e' piu' recente. In alternativa con il metodo head il server originario manda solo l'header del file, il proxy confronta se l'header ha una data di modifica piu' recente ed in questo caso chiede una copia piu' recente. Una volta che il proxy server ha capito che la copia che possiede in cache e' aggiornata la restituisce al browser che ne ha fatto richiesta. Anche con questa configurazione si puo' avere un vantaggio, i contenuti possono essere voluminosi, nel momento in cui il proxy fa una get condizionale o head e sul server originario non ci sono copie nuove quindi l'interazione e' molto veloce perche' torna solo una risposta o solo l'header, e' piu' veloce perche' nel caso in cui non ci sono modifiche il messaggio e' molto piccolo.

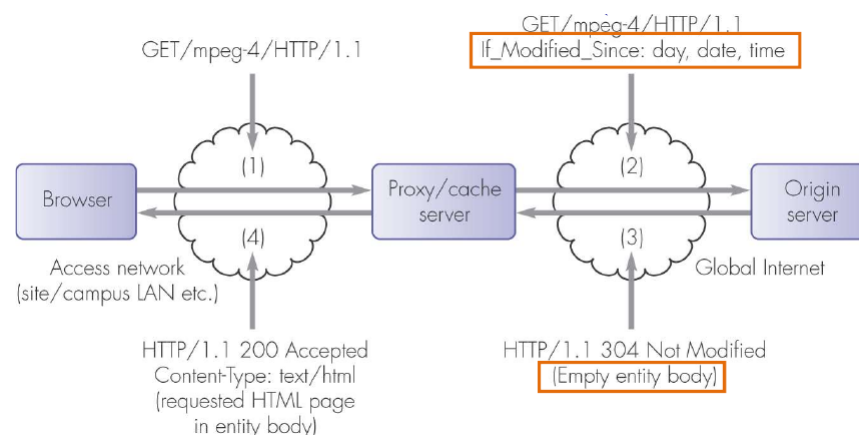


Figure 3.14: Proxy server

3.8 Electorinc Mail

Anche nel caso della posta elettronica non e' sempre ben definito chi e' server o client. Anche qui (dato che siamo a livello applicazione) ci sono degli user agent, chiunque permetta di comporre (comporre il testo ed indicare a chi va una mail, puo' essere un gruppo di utenti o alias) e chiunque permetta di leggere una mail, significa non solo leggere il testo ma fruire di tutte le altre parti (immagini) con tutte le application che servono. Le tre componenti principali sono:

- User agents
- Mail servers
- simple mail transfer protocol: SMTP

Gli agenti, corrispondono ad una casella di posta ma devono appoggiarsi su un server, anche perche' SMTP usa una filosofia che e' diversa da HTTP (HTTP e' protocollo pull), mentre SMTP e' push (non sappiamo quando ci arrivano le

mail) . Se siamo amministratori in UNIX con il comando mailq vediamo la coda di messaggi in uscita, quando mandiamo un messaggio di posta le mail vengono mandate nel server che gestisce le caselle di posta degli utenti che serve.

3.8.1 E-mail: funzionalita'

In un messaggio c'e' l'header (intestazione, il corpo della mail e gli allegati. L'header ad esempio contiene le informazioni per la mailing list, chi vuole mandare una mail la manda al server gestore ed e' il server che la gestisce che guardando chi sono gli associati manda loro la posta, gli alias sono utenti fittizi che corrispondono ad utenti reali, la copia per conoscenza, la copia nascosta). Due porzioni del messaggio:

- Header: tutte le informazioni per il server mail
- Corpo: testo e allegati

Lo componiamo e inviamo al server mail con SMTP, perche' anche l'interazione tra client e server mail e' un'operazione di push.

3.8.2 Simple Mail Transfer Protocol (SMTP)

Il servizio di email e' uno dei primi che e' stato realizzato in Internet, quando e' stato realizzato era stato pensato per mandare solo caratteri, per cui e' un po' rozzo perche' prevede che quello che viene scambiato sono solo caratteri ASCII codificati su 7 bit (0-127), i caratteri che vengono trasferiti da una parte all'altra in righe (una riga di testo alla volta) per dire che abbiamo trasferito tutto mandiamo CRLF.CRLF e' un punto solo sulla riga per dire al ricevente che tutto quello che ha ricevuto prima e' il corpo della mail. Se questo e' tutto quello che si possono scambiare in SMTP devo stare attento che non ci siano caratteri non permessi. Quando compongo un testo di una mail non c'e' niente che mi proibisca di mettere come corpo un punto da solo sulla linea, il problema e' che quando l'user agent trasferisce tramite SMTP il messaggio e trasferendolo riga per riga trasferendo quella vuota con il punto il ricevente capisce che il corpo e' finito. Ci vuole una codifica per dire che tutto quello che non e' nel set di caratteri ammessi lo devo manipolare per trasferirlo dall'altra parte in altra forma in modo che venga capito e non mal'interpretato. I mail server possono essere sia client (quando mandano il messaggio a qualche altro server) oppure server quando un user agent gli manda un messaggio. Quindi tutti gli User Agent si interfacciano con User Agent Server. I server che si scambiano messaggi sono chiamati Message Transfer Agent (MTA), aggiungono un envelop al messaggio con informazioni di controllo per il dialogo tra le entita' SMTP.

3.8.3 Schema scambio email

Lo user agent sorgente spinge verso il suo mail server tramite SMTP il suo messaggio (fatto dallo header che noi componiamo e il corpo della mail). il

mail server aggiunge un envelop (le sue informazioni di controllo per dialogare con l'altro mail server), il mail server di ricezione tira via l'envelop e passa il messaggio al suo user agent. Il mail server ha dentro il Mail transfer agent e il mail access agent che aspetta le richieste dell'user agent destinatario e gli permette la gestione della stessa.

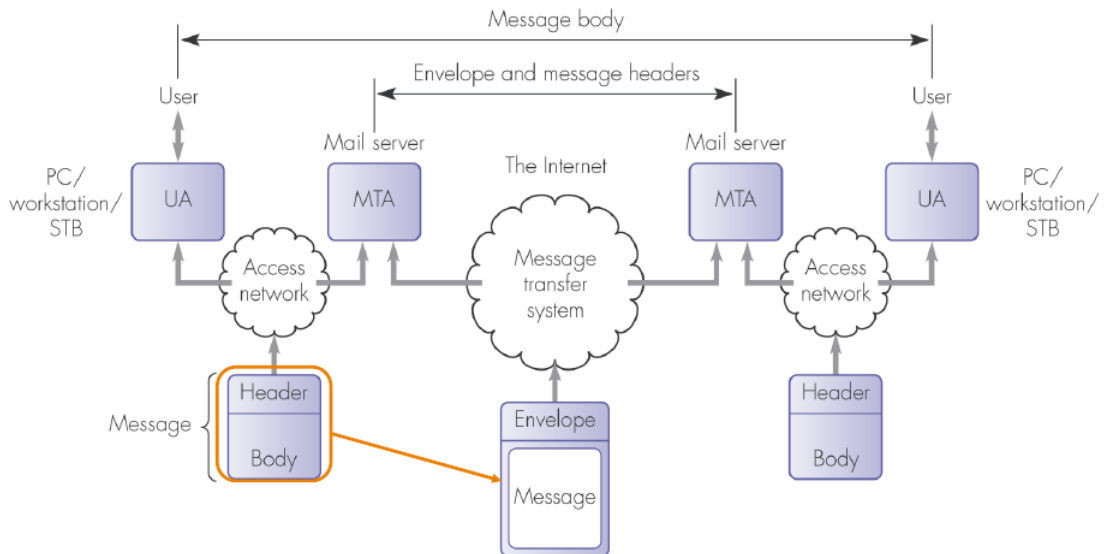


Figure 3.15: Schema di scambio mail

3.8.4 Struttura della mail

Tutto è testo ASCII codificato su 7 bit. Posso far sembrare che il messaggio sia stato mandato da un message transfer agent diverso. La data in cui il messaggio è stato generato, l'identificatore del messaggio, reply-to posso configurarlo per fare arrivare dei messaggi alla posta istituzionale.

3.8.5 SMTP

3 fasi per il trasferimento:

- Handshaking: si crea una connessione TCP
- Transfer of messages: su questa connessione vengono scambiati i messaggi SMTP, i saluti il messaggio e la chiusura della connessione
- Closure: chiusura connessione TCP

Nel momento in cui il mail server contatta un altro mail server manda almeno un messaggio, non chiude subito la connessione ma guarda la coda e pesca i

messaggi che vanno a quel mail server, quando nella coda non c'è più niente per quel mail server chiude la connessione. Nella fase di saluto si autenticano, controllano esistenza del destinatario prima di trasferire il messaggio. Come per HTTP i messaggi sono testo .

Commands (Client MTA → Server MTA)	Descriptions
HELO Mailserver-name	Sends DNS name of the client mail server
MAIL FROM: <e-mail address of sender>	e-mail address of sender
RCPT TO: <e-mail address of recipient>	e-mail address of recipient
DATA	Request to send the message
QUIT	Requests recipient server to close TCP connection
RSET	Abort current mail transfer

Responses (Server MTA → Client MTA)	Descriptions
220	Recipient server is ready
221	Recipient server is closing TCP connection
250	Command carried out successfully
354	Indicates the recipient server is ready to receive message
421	Service request declined
450	Mailbox unavailable
⋮	⋮
551	Addressed user is not here

} Error responses

Figure 3.16: SMTP command response

```

S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection

```

Figure 3.17: Esempio SMTP

3.8.6 Mail access protocols

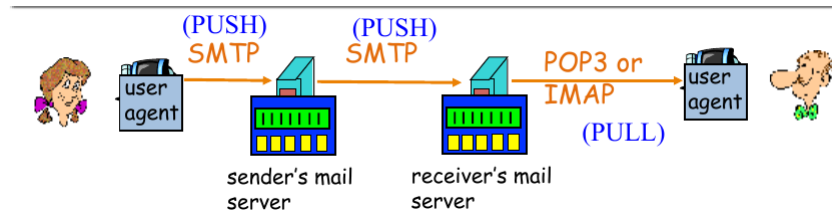


Figure 3.18: Mail Access Protocol

Nell'ultimo tratto da receiver's mail server e l'user agent e' un protocollo di pull, quindi si usano:

- **POP3**: permette di farsi dare la lista di messaggi, farsi passare dal suo mail server un messaggio alla volta e puo' dire se vuole che nella mail box il messaggio venga cancellato o tenuto li', quindi ha due comandi: **download&delete** o **download&keep**
- **IMAP**: permette di gestire nel server cartelle di messaggi e possono essere trasferiti anche parzialmente

POP3 Protocol

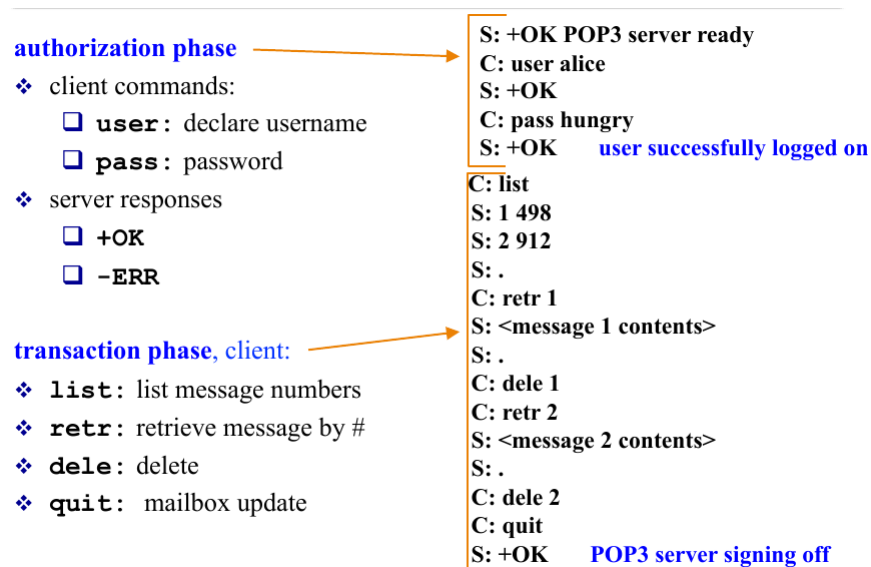


Figure 3.19: POP3 protocol

IMAP

Supporto ad accesso utente da diversi host: non scarica la mail sull'host utente, a differenza di POP3 che lo fa solo se configurato come download&keep. Consente il mantenimento di folder multipli per uno stesso utente perché mantiene informazioni di stato tra sessioni diverse (a differenza di POP3 che è stateless). Permette il download componente del messaggio.

3.8.7 MIME

Non è un protocollo in sé, ma è una cosa che viene aggiunto al protocollo SMTP (estensione) che permette di gestire gli allegati non in formato testo. O anche in formato testo che non sia codificato in caratteri su 7 bit. Multipurpose Internet Mail Extension.

- Trasforma qualunque file che noi vogliamo allegare alla mail in qualunque formato, che sia uno stream di bit o un set di caratteri non riportabile agli ASCII 7 bit, lo ricodifica come caratteri ASCII su 7 bit in modo che possa essere ricodificato alla destinazione ricostruendo il file in allegato.
- Per far capire al destinatario cos'è stato allegato gli dà informazioni aggiungendo campi allo header SMTP. Lo header degli utenti è collezione simile ad HTTP di coppie nome campo - valore. MIME aggiunge delle sue coppie a quelle che abbiamo già.

Se mettiamo più allegati queste coppie vengono generate per ogni allegato.

- Content-Description: descrizione testuale dell'allegato
- Content-id identificatore alfanumerico pseudocasuale attaccato per separare i contenuti della mail
- Content type: Accoppiata tipo sottotipo, può servire al chi riceve di chiamare una helper application per fruire dell'allegato. ci sono dei sottotipi del tipo che vanno ad identificare il tipo dell'allegato:
 - Text: plain, html
 - Image: jpeg, gif
 - Audio: basic
 - Video: mpeg
 - Application: Possono essere applicazioni esterne per cui il sottotipo dà l'idea di quale applicazione bisogna usare per fruire di quel allegato.
- Content-Transfer-Encoding: specifica qual'è la tecnica di codifica utilizzata
- Content-Length: il numero di byte che serve per un controllo quando l'allegato arriva a destinazione. Fa il controllo che se l'allegato decodificato è costituito da un numero di ottetti pari a quello del content-length allora è giusto e puoi utilizzarlo.

3.8.8 Tipi MIME

- Multipart/mixed: per messaggi contenenti più oggetti :
 - MIME header che descrive ogni oggetto
 - Caratteri limite tra oggetti diversi che separano i vari contenuti, vicino c'è l'edintificatore unico pseudocasuale
 - le informazioni ci dicono qual'è l'header application che vogliamo usare per aprire l'allegato.

Ci sono due tipi di codifiche:

- Quoted printable: è meno usata e trasforma testo ascii arbitrario in testo senza caratteri indesiderabili. Stabilisce una connessione tra alfabeti diversi. Toglie i punti tutti soli in una linea nel mezzo del corpo che potrebbero essere interpretati come la fine di trasferimento del corpo. Aggiunge dei caratteri che facciano capire che non è la fine del trasferimento.
- Base64: è la codifica che è usata per qualsiasi cosa che non sia trasformabile in un set di caratteri. Considera 26 maiuscole 26 minuscole 10 cifre sono 64 caratteri. Prende l'allegato che non sia un testo, lo considero come un stream di bit, la rompo in gruppi di 6 bit e guarda il valore numeri rappresentato da questi 6 bit (0 63) è qualcosa di quei 64 caratteri che abbiamo detto prima e trasforma tutto in un sottoinsieme di caratteri ASCII su 6 bit. L'allegato così riscritto è una sequenza di caratteri che per noi non ha senso ma quando arriva allo user agent della destinazione permette di ricostruire il file destinatario, perchè guardando il transfer encoding ripristina il gruppo o di 6 bit ricostruisce l'allegato mettendo tutto in fila.

3.8.9 Esempio MIME

Il primo valore dello header ci dice il content type (di allegati ce ne sono tanti), per ogni contenuto separato dal boundary è il tipo. Attenzione che il boundary è sbagliato nell'esempio, è una stringa incomprensibile nella realtà.

```

From: xyz@abc.com
To: abc@xyz.com
Subject: Happy birthday Irene
MIME-Version: 1.0
Content-Type: Multipart/Alternative; boundary = "TryAgain";

-- TryAgain

Content-Type: Message/External-body;
    name = "Irene.audio";
    directory = "Irene";
    access-type = "anonftp";
    site = "myserver.abc.com";
Content-Type: Audio/Basic;           (Message in audio accessed remotely,
Content-Transfer-Encoding: Base64

-- TryAgain

Content-Type: Text/Richtext;
<B> ***Happy birthday Irene*** </B> (Message in richtext)

-- TryAgain

Content-Type: Text/Plain;
    ***Happy birthday Irene*** (Message in plaintext)

-- TryAgain

```

Figure 3.20: Esempio MIME

3.8.10 email gateway

E' un mail transfer agent che è capace di ragionare sulle mail che deve passare, per cui è qualcuno che apparentemente è il destinatario, ma in realta' quando gli arriva guarda e capisce che è un alias e lo esplode in tutti i destinatari veri, oppure se è una mailing list genera una copia per utente. Traduce messaggi in transito tra sistemi che diversi formati e protocolli:

- mapping tra indirizzi con rappresentazioni diverse
- manipolazione header non presenti o riconosciuti

Procedura:

- messaggio trasferito da mail server a gateway dell'organizzazione
- Il gateway determina su quale rete deve essere inoltrato, in base a indirizzo di destinazione
- il gateway riformatta il messaggio in funzione della natura della rete

La cosa interessante non è tanto cosa fa, ma è la parola gateway, non è sinonimo di router. è più propriamente chiamato come multiprotocol router,

un router normale svolge le sue funzioni di instradamento, mentre un multiprotocol è un oggetto che fa routing però sta tra due rete e non solo è capace di capire i protocolli rete ma anche di protocolli di livello applicazione ed è capace all'interno della rete è capace di guardare dentro i payload (nei dati veri e propri) e tradurli da uno stack di protocolli in una rete ad uno stack in un'altra rete, tradurre la codifica.

3.9 FTP

E' il comando che manda in esecuzione il client stp a cui diciamo chi deve contattare, quello che segue è il nome simbolico del server da contattare. Il client ha preso il nome simbolico ha chiesto al local resolver se aveva un indirizzo di rete, lui è andato a cercare nel domain name system qual'è il l'indirizzo di rete corrispondente e quello che gli ha tornato è che quello che cercavamo era un alias. Spesso si creano degli alias perchè può essere comodo. Per accedere al sistema remoto possiamo o fare il log in se siamo utenti del sistema, oppure possiamo accedere come ospiti tramite la parola *anonymus* che ci porta in un posto particolare del file system del server. Una volta entrato posso fare ad esempio una get di un documento nel server. Il server ci risponde che il comando port va bene e che sta aprendo una connessione file in ASCII mode (che è una stupidaggine ma va bene), poi ci dice che è stato trasferito tutto e ci dice il bitrate, close ci permette di chiudere la connessione ed a questo punto potremmo aprire un'altra connessione con un altro server STP o uccidere con il quit. Quindi FTP trasferisce dei file.

```
$ ftp ftp.cs.purdue.edu
Connected to lucan.cs.purdue.edu
220 lucan.cs.purdue.edu FTP server (Version wu-2.4.2-
VR16(1) ready.
Name (ftp.cs.purdue.edu:users): anonymous
331 Guest login ok, send e-mail address as password
Password: *****
230 Guest login ok, access restrictions apply.
ftp> get pub/comer/tcpbook.tar bookfile
200 PORT command okay. /* creazione port per client */
150 Opening ASCII mode data connection for tcpbook.tar
(9895469 bytes).
226 Transfer complete.
9895469 bytes received in 22.76 seconds (4.3e+02 Kbytes/s)
ftp> close
221 Goodbye.
ftp> quit
```

Figure 3.21: FTP Modalita' D'uso

I comandi non sono moltissimi:

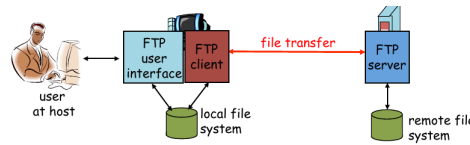


Figure 3.22: FTP

- GET
- PUT: se abbiamo autorizzazione
- possiamo aggiungendo una `l` ai comandi di UNIX per muoversi tra le directory

3.9.1 Control data connections

Abbiamo detto che HTTP è stateless che lavora online (su una connessione TCP possono viaggiare sia i comandi sia i dati). FTP è stato fatto in altro modo, ha due connessioni, controllo e dati, la connessione di controllo viaggiano nome password ed i comandi che diamo, dura per il trasferimento di un file e poi viene chiusa. è un protocollo per cui le informazioni di controllo viaggiano su un canale separata. FTP mantiene uno stato perchè il server si ricorda dove stiamo navigando (`cd relativi`), si ricorda chi noi siamo quindi per ogni spostamento e per ogni file che decidiamo di scaricare o caricare sa le authorization.

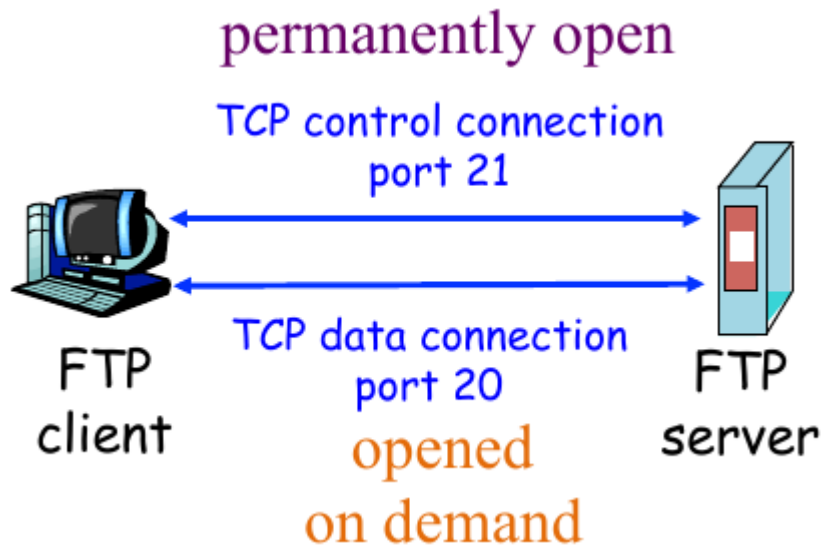


Figure 3.23: Control & data connections

3.9.2 FTP protocol

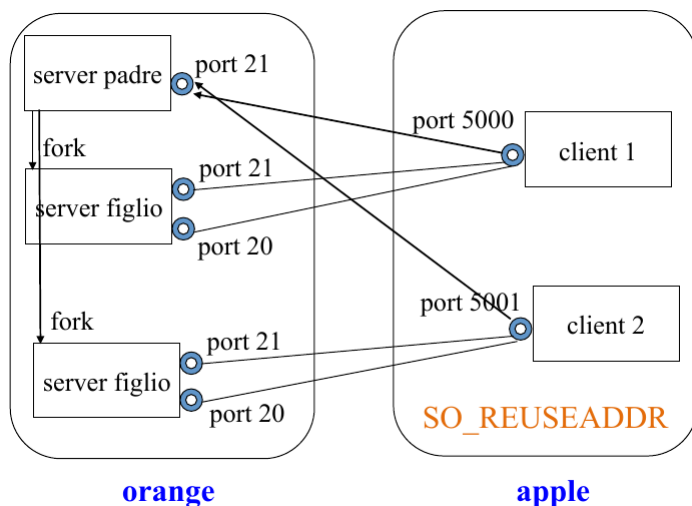


Figure 3.24: FTP protocollo

Io client mi faccio dare una porta effimera dal sistema operativo e faccio partire una richiesta di collegamento con l'FTP server che sta su ORANGE, lui ascolta sulla porta 21. A questo punto il server delega ad un proprio processo figlio (system call di fork con cui crea un clone, il quale può cambiare l'eseguibile e fare altre cose), il processo padre rimane quindi in ascolto sulla porta 21 e non si lega al cliente, il figlio utilizzando sempre la 21 stabilisce la connessione con il cliente, a questo punto ho due connessioni su ORANGE che corrispondono ad associazioni diverse. Per il server padre il protocollo è TCP, indirizzo locale è indirizzo di rete di ORANGE più porta 21, l'indirizzo remoto non c'è (perché padre non si è legato con nessuno); per il server figlio la quintupla è: Protocollo TCP, indirizzo locale = indirizzo di rete di ORANGE più la porta 21, in più è specificato il cliente che sta su macchina con IP APPLE con porta 50000. Un'altro cliente su APPLE chiede di connettersi a ORANGE tramite FTP, il client pesca un'altra porta effimera, chiede una richiesta di connessione al server padre (può farlo perché non è ancora legato), il quale genera un'altro server figlio che si connette con il client2. È importante che le associazioni siano diverse perché TCP sa a chi e come fare demultiplexing. Quando dal server dobbiamo tornare indietro dei file dobbiamo creare la connessione dati, il server figlio fa partire dalla porta 20 la connessione dati sulla porta 50001 del client2, quindi creiamo una connessione diversa. Non c'è confusione su cosa va dove. Nei client FTP c'è una direttiva per dire al sistema operativo che la porta effimera che ha dato per il cliente deve permettere di accettare l'indirizzo per le richieste di connessione (è una socket option).

3.9.3 FTP comandi e risposte

In accordo al fatto che è un protocollo dei primi nati e quindi usavamo comandi su 4 caratteri:

- USER: username
- PASS: password
- LIST: ritorna la lista di file presenti nella directory corrente del FTP server
- RETR: è una get, retrieve di un determinato nome file
- STOR: è la put, metti in magazzino questo nome file sul server FTP

IL server ci ritorna un po' di codici di stato per capire com'è andata la connessione:

- status code and phrase (come in SMTP)
- 331: Username OK, password required
- 125: data connection already open; transfer starting
- 425: Can't open data connection
- 452: Error writing file

3.9.4 FTP vs HTTP vs SMTP

tutti e 3 usano TCP

HTTP è stateless, trasferisce informazioni di controllo in-band, può usare connessioni TCP non permanenti o permanenti.

FTP ha stato, trasferisce informazioni di controllo out-of-band.

SMTP: trasferimento asincrono di file, la mail dev'essere una sequenza di ascii su 7 bit, tutto dev'essere convertito.

3.10 TELENET

È un'applicazione per il login remoto su una macchina su cui non si ha un account (molto vecchio come servizio, terminalNetwork). Anche qui avremmo il client sull'end system utente, gli passiamo come argomento il nome simbolico del server ed un numero di porta, quello tra quadrate è opzionale, di default usa la porta 23 telenet servername-or-ip [port-number (23)] Molto similmente a FTP chiede user name e password e apre un prompt telnet su cui possiamo mettere comandi. Usa TCP come protocollo di trasporto, perché i comandi devono arrivare tutti ed in ordine. È un protocollo multiprotocollo, perché potremmo collegarci su una macchina remoto con un processo che usa un'altra porta e dialogarci purché sappiamo il linguaggio di comandi di quel servizio. Può ancora essere utile perché se un amministratore sospetta che un processo su una

macchina ha un mal comportamento con telnet possiamo provare a collegarci con il sistema come fossimo client mettendolo sotto stress per cercare di capire se funziona bene o ci sono degli errori. Quindi è multiprotocollo: può emulare il client di qualunque protocollo di livello 7 basta usare l'appropriato linguaggio per interagire con il server. Può fare anche marshalling, che è una funzionalità di livello 6 (presentazione) per cui è anche in grado di adattare il formato dei dati trasmessi da un architettura all'altra. Il problema è la sicurezza, perché tutto viaggia in chiaro.

3.11 Secure Socket Layer

E' stato sostituito con la suite di protocolli SSL, dove il secure sta per cifriamo tutto. E' una suite di protocollo perché dentro ne ha tanti: SSH sostituisce telnet, SCP permette il trasferimento di un solo file ed insieme ad SFTP instaura un dialogo simile a FTP ma cifrato sostituisce FTP. Usa servizi TCP e la porta standard 22 ed è diviso in 3 sottolivelli:

- SSH-CONN: ci si connette e si iniziano a scambiare i dati.
- SSH-AUTH: autenticazione, una volta che abbiamo stabilito come far passare le cose il cliente si autentica.
- SSH-TRANS: costruisce un canale di trasporto sicuro. Quando costruiscono questo canale virtuale è una connessione sopra TCP, negoziano la tecnica di crittografia, sulla lunghezza delle chiavi e si scambiano le prime chiavi da usare per la cifratura di tutto il resto.
- TCP: sotto c'è una connessione TCP.

3.12 Domain Name System (DNS)

Visto quando abbiamo ripreso HTTP. E' un protocollo di livello 7 per tradurre nomi simbolici in indirizzi IP, ma usato anche per tirare fuori informazioni riguardo i sistemi che stanno utilizzando, ad esempio gli alias dei server. E' usato a livello applicazione, e soprattutto da protocolli di livello 7 ma può usarlo anche l'utente con il comando di shell `nslookup`. Legge in nomi simbolici da destra verso sinistra, perché va dal grosso (it) allo specifico (il nome della macchina). Il DNS è nato tardi, inizialmente c'era un file con nome simbolico ed indirizzo di ogni macchina in rete e replicavano questo file su tutti gli end system. La rete diventando grossa diventava grossa anche la dimensione di questo file. La richiesta sarebbe di avere un unico nome per ogni end system nella rete, deriva dal fatto che se prendiamo i nomi simbolici ed il DNS li traduce in indirizzo di livello rete che viene usato dai router per capire qual'è la rotta, se ci sono due nomi simbolici uguale il server potrebbe tradurli in un unico indirizzo IP ed i server non sanno più cosa fare.

Il DNS e' realizzato da un insieme di database su alcuni nodi della rete ed usano i nomi simbolici come dati di ricerca degli stessi database per trovare le informazioni associate. Non c'è la lista completa da nessuna parte ma ogni informazione c'è in più database, quindi è un database distribuito e parzialmente replicato.

La deroga che un nome corrisponda ad uno host e' concessa ai grossi enti, ad esempio unimi.it puo' corrispondere a tanti server fisici localizzati in posti diversi o nello stesso posto, ad esempio utilizzando Akamai (un servizio a pagamento) fa iscrivere aziende grossi e monitora i server che stanno rendendo lo stesso servizio, dall'altra parte mi faccio pagare da utenti perche' gli do un pezzo di software da installare come helper application nel browser che fa: le richieste HTTP non vanno al server indicato ma vengono riscritte dal browser (come con un proxy) per essere mandata al nodo akamai piu' vicino a me e nel momento in cui arriva una richiesta di andare ad un server, riscrive la richiesta del client e manda la richiesta ad uno dei tanti server che offrono lo stesso servizio che e' piu' efficiente in quel momento. Un azienda del genere allarga le funzionalita' del DNS per dare servizi agli utenti. Questa funzione e' stata recentemente unita anche direttamente nel DNS: vengono messi i nomi simbolici in un database come chiavi di ricerca, nelle versioni recenti si puo' associare ad un nome simbolico un insieme di uno o piu' indirizzi IP ed il DNS nel momento in cui qualcuno cerca un nome simbolico e la richiesta di risolvere www.unimi.it arriva ad un DNS che sa fare questa cosa, il DNS guarda l'insieme di indirizzi IP associati e per ogni richiesta da un indirizzo diverso dell'insieme associato, una volta finiti gli indirizzi ricomincia da capo.

3.12.1 Gerarchia dei nomi simbolici

L'idea che c'e' alla base e' che i nomi simbolici non sono completamente casuali, gia' sappiamo che i nomi vanno letti da destra a sinistra ottenendo una risoluzione via via piu' fine per capire dove stiamo andando, partendo dal paese, l'organizzazione. Non solo e' costruita in questo modo ma ci sono anche dei vincoli sui nomi, ovvero sulle stringhe componenti il nome simbolico separate dal punto.

I nomi simbolici organizzati in gerarchia ad albero e quello che c'e' in cima e' un root domain che non si scrive mai, poi quello che segue andando da destra a sinistra sono nomi top level domain rigidi, quindi non possiamo inventarci nulla per dire qual'e' il nome piu' generico a destra, alcuni nomi dicono che cosa fa una determinata organizzazione (com = commerciali, edu = education, gov = governativi, mil = militari, org = non a scopo di lucro, net = internet service provider di vari livelli, int = international (grandi organizzazioni internazionali)) poi c'e' un'altro gruppo di top level domain che e' una targa di paese (it) questi sono unici e non possono essere decisi ma e' l'autorita' che controlla la radice che controlla i top level domain. All'interno di un top level domain e' chi gestisce uno di questi che garantisce che tutto cio' che si chiama qualcheduno.edu (ad esempio edu) abbia un nome unico, ad esempio edu ha deciso che c'e' solo un mit (massachusetts institute of technology), a sua volta lui

garantisce che tutti i nomi che terminano con .mit.edu sono unici. Significa che ad ogni livello della gerarchia c'è un controllore che garantisce che i suoi immediati figli abbiano nome diverso. Un nome che termina con punto di questa struttura si chiama nome di dominio completamente definito o fully qualified domain name (koala.csse.monash.edu.au.), possiamo usare anche dei nomi parziali (/etc/networks in unix è usato per il completamento).

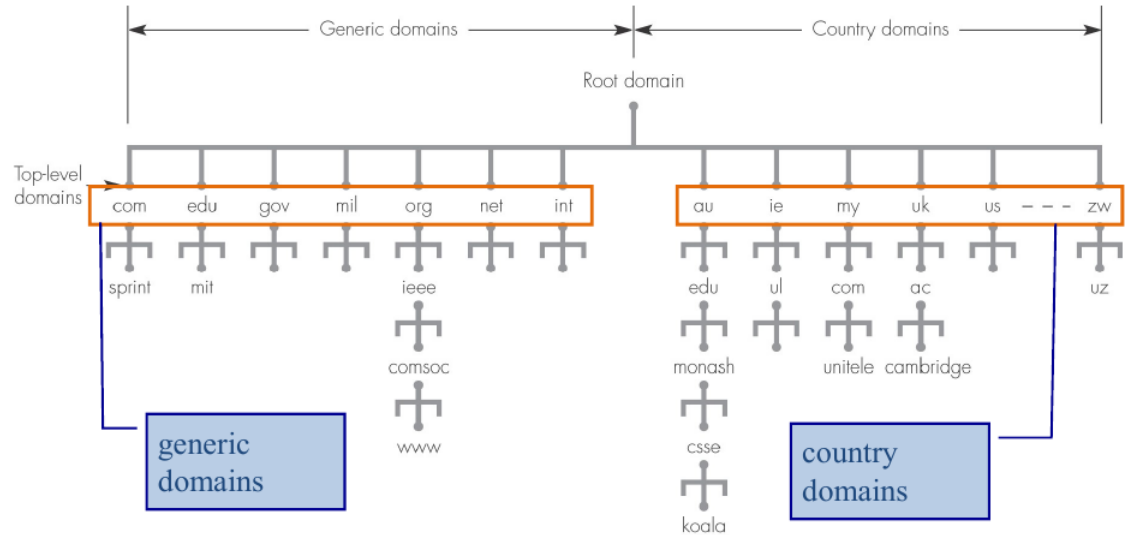


Figure 3.25: Gerarchia dei nomi simbolici

3.12.2 Assegnazione nomi simbolici

Non abbiamo una gestione centralizzata, abbiamo già detto che non c'è una funzione di rete che sia centralizzata perché sarebbe pericolosissimo, la root come organizzazione è responsabile della creazione dei domini top level, ma non è una sola viene replicata, l'autorità di ogni dominio è responsabile dell'assegnazione di un nome unico dei propri figli. Ai nomi non interessano le maiuscole o minuscole, il database è parzialmente replicato perché ogni informazione risiede su più di un server ma nessun server ha tutte le informazioni. Avere delle repliche va sempre nella direzione di non avere nulla di centralizzato, per trade-off robustezza.

3.12.3 Autorità

Ci sono delle organizzazioni che ci permettono di gestire il DNS:

- ICANN: non profit che coordina il DNS, gli indirizzi IP, spazi di indirizzamento, assegnazione dell'identificatore dei protocolli (per gestire il demultiplexing)

- IANA: e' una divisione di ICANN responsabile delle stesse cose di ICANN, e' un braccio operativo che assegna anche indirizzi hardware ai produttori di Network Interface Card, in modo che ogni volta che devono stampare una scheda di rete ci mettono dentro un indirizzo univoco, quindi non ci saranno al mondo due schede di rete con lo stesso indirizzo hardware.

IANA non fa tutto da sola ma usa dei RIR (registry) i quali gestiscono l'allocazione e registrazione di "numeri" Internet per le 5 regioni del mondo:

- AFRNIC: Africa
- ARIN: USA, Canada, parte Caraibi, Antartico
- APNIC: Asia, Australia, Nuova Zelanda
- LACNIC: Latino-Americana, parte Caraibi
- RIPE NCC: Europa, Russia, Medio Oriente, Asia Centrale

I RIR a propria volta non fanno tutto loro, sotto ci sono dei registry che sono delle autorità di paese.

3.12.4 root server

Le root sono importanti perché hanno una visione d'insieme dell'albero, quindi conoscono i server che gestiscono i top level domain, le root prendono il nome che stiamo cercando di risolvere e lo leggono da destra verso sinistra e dato che conosce i top level delega a loro la risposta alla nostra richiesta. L'indirizzo di un paio di root server è caricato durante il boot da un file di configurazione del sistema operativo del client. Sono 13 root server, in realtà ognuno di queste radici non è un unico server ma ad ogni server logico corrisponde una serie di server fisici replicati, le richieste sono mandate con anycast, quindi quello che un DNS fa rivolgendosi al root server tramite anycast, ovvero gli basta che uno qualsiasi dei server fisici che implementano quel server logico gli dia una risposta. Tra loro i nodi fisici devono sempre tenersi consistenti.

3.12.5 Gerarchia name server

All'albero dei nomi corrisponde un'albero di server, ci sono un po' di server che gestiscono le radici, un po' che gestiscono i top level domains. Generalmente questi pool alcuni sono indicati primari ed altri secondari, i primari sono quelli a cui ci si rivolge inizialmente e se non sono disponibili si passa ai secondari.

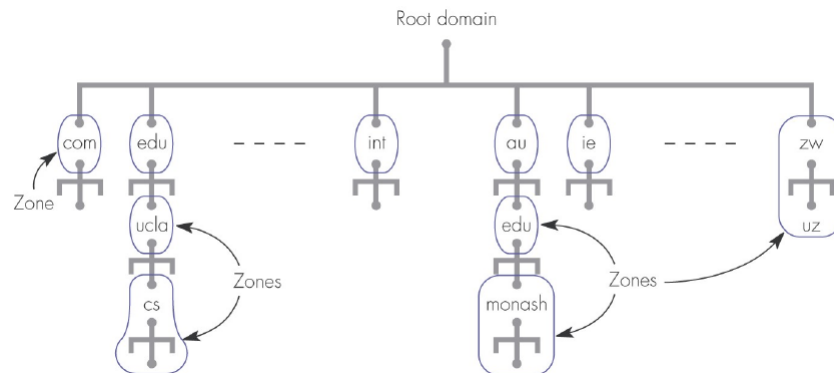


Figure 3.26: Gerarchia dei name server

3.12.6 DNS resource record

E' un record che descrive le nostre risorse in internet. Il nome di dominio e' la chiave di ricerca, la classe e' di 16 bit ma in realta' esiste una classe sola (l'idea era quella di indicare come sono fatte le informazioni associate ad un nome di dominio, i nomi di dominio sono su 4 ottetti di bit ed e' vero per IPV4, mentre IPV6 ha 128 bit, quindi ipotizzando che ci possano essere indirizzamenti differenti ci si e' tenuto delle informazioni perche' magari le informazioni possono non solo essere di tipo IN Internet), il time to live e' un informazione che dice per quanto tempo vale l'informazione contenuta nel resource record, quando qualcuno si procura le informazioni di fatto si procura una copia di resource record e si fa una copia delle stesse perche' ne fa caching. Per quanto tempo mi tengo questa' informazione e' deciso da chi l'ha creata. Sto dicendo in questo discorso che c'e' qualcuno che scrive il resource record perche' e' chi controlla le informazioni, quel record e' quello autoritativo quindi con le informazioni giuste ed aggiornate, le copie cached sono non autoritative e possono anche essere obsolete.

Il numero di componenti del fully qualified domain name puo' essere qualsiasi, non poniamo limiti sulla profondita' dell'albero dei nomi, diciamo che ogni componente puo' essere lungo al massimo 63 byte e che globalmente non deve superare 255 byte. Dentro il record il primo e' un byte (numero grande fino a 255) c'e' un valore che e' il numero di byte da cui e' composto il primo elemento della stringa, il byte successivo a sua volta dice il numero di byte da cui e' composto il secondo elemento.

Il type determina le informazioni mantenute nel record:

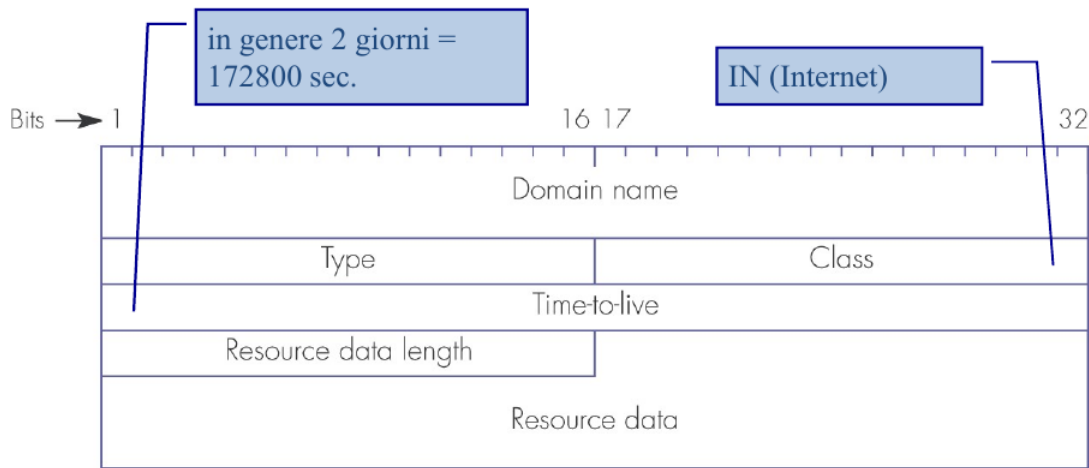


Figure 3.27: DNS resource record

- A / AAAA: al nome simbolico nel domain name e' associato address, A = IPV4, AAAA = IPV6
- NS : name server per il dominio
- PTR: indirizzo IP in notazione decimale puntata (137.204.24.35)
- HINFO: host info, ci da delle informazioni human readable sul sistema (tipo di hardware, sistema operativo, versione del sistema)
- MX: mail server del dominio
- CNAME: alias dei nomi simbolici
- TXT: altre informazioni leggibili su quello che sto cercando

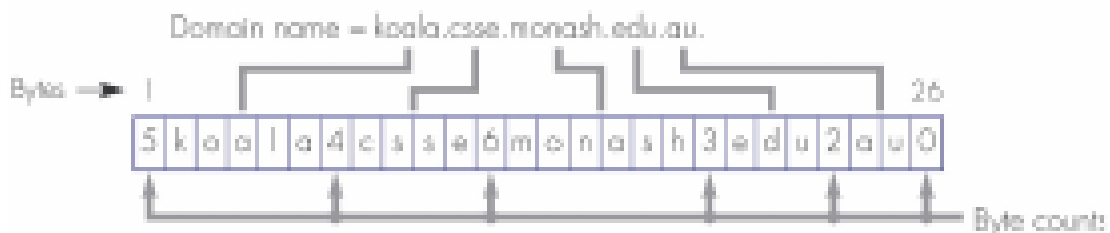


Figure 3.28:

3.12.7 DNS query/reply

Dobbiamo capire qual'è il dialogo tra resolver e DNS locale ed i vari DNS. I vari nodi del DNS comunicano tramite UDP. I primi tre campi sono della richiesta e l'ultimo è la risposta.

- Query Header: 4 ottetti, 32 byte
- Query domain name: Nome di dominio che vogliamo cercare
- Query type: Tipo di query
- Query class: Classe, come devono essere fatte le informazioni che cerchiamo (Internet IPV4 o IPV6)

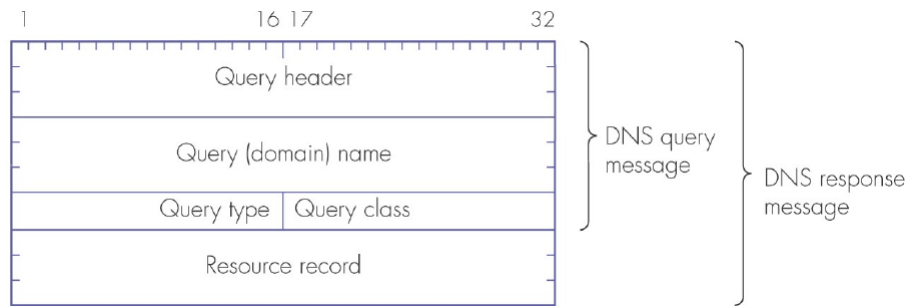


Figure 3.29: DNS query reply

La risposta porta tutte le informazioni della richiesta in più' aggiunge uno o più' resource record per arrivare alla risoluzione della nostra richiesta.

Header DNS

Abbiamo un identificatore della query che serve per accoppiare le risposte con la richiesta, se la risposta replica i tre campi di informazioni e replica anche la query quindi dice a chi si riferisce. I flag indicano se la risposta è autoritativa o meno e dicono nel caso sia una richiesta con quale procedura la richiesta viene processata dai nodi del DNS. Nello header è descritto qual'è il numero di domande fatto ed essendo anche nella risposta c'è il numero di risposte ottenute e si dice di queste quante sono autorevoli e quanti sono resource record aggiuntivi.

Identificatore query	flag
Numero di domande	Numero di risposte
Numero di Resource Record autorevoli	numero di RR aggiuntivi

Table 3.2: Header DNS

3.12.8 Risoluzione Locale

Come facciamo la ricerca? L'applicazione dell'utente (browser) si rivolge ad un processo resolver locale, il quale chiede al DNS locale se sa risolvere il nome simbolico (sulla porta 53) il DNS se ha bisogno ha delle interazioni con tutta l'infrastruttura del DNS (che abbiamo visto prima) finché il mio DNS locale non riesce a recuperare una risposta, allora torna una risposta via UDP al resolver, il quale la restituisce all'applicazione ed a questo punto l'applicazione può contattare l'end system con cui aveva chiesto l'interazione e scambiare cose. Il resolver come prima cosa guarda nelle informazioni locali se ha qualche cosa, in UNIX queste informazioni sono contenute in file/etc/hosts.

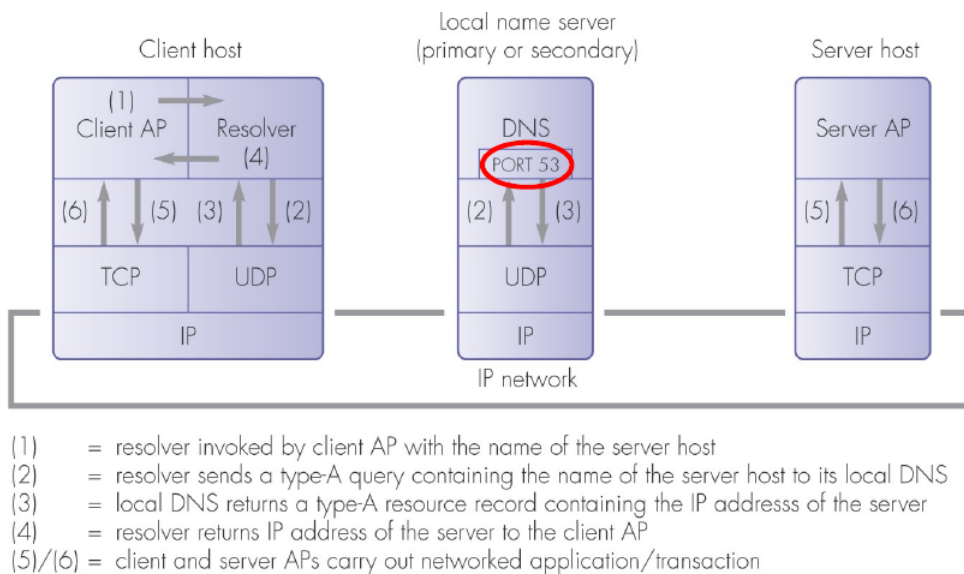


Figure 3.30: Risoluzione locale

3.12.9 Risoluzione ricorsiva pura

Si chiama pura perché nella maggior parte dei casi la soluzione adottata è un ibrido tra ricorsiva ed iterativa. Il local resolver che non è riuscito a trovare le informazioni in locale si rivolge al DNS locale (è quello più vicino a noi) i nostri sistemi hanno sempre due server DNS registrati (primario e secondario, non solo per registrare informazioni ma soprattutto per le interrogazioni, ci serve quindi per robustezza del sistema). Il DNS locale (fisicamente vicino a noi) guarda se possiede le informazioni e può guardare in due posti:

- Se è un server autoritativo per il domain name (quindi lui ha già la risposta)

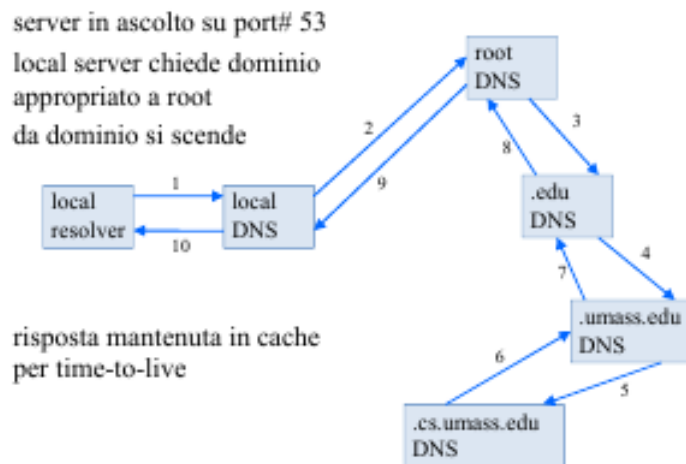


Figure 3.31: Soluzione ricorsiva pura

- Può guardare in cache se qualcuno ha già fatto la richiesta e portato in cache la risposta per un certo periodo (periodo che c'è nel resource record)

Se il DNS locale non trova la risposta va dalla root che ha indicato come primaria (le root sono ridondate loro ed inoltre sono ridondate all'interno del client) a sua volta ha identificato due root logiche (primaria e secondaria), in pratica il DNS locale fa un anycast (vuole avere una risposta da almeno una root fisica che implementa quella root logica). Inoltre alla root la radice la quale a sua volta controlla se è autoritativa per la query oppure risiede in cache. Tendenzialmente la risposta è no perché è autoritativa per i top level domain ma data questa caratteristica conosce il percorso da fare tra i server per reinstradare la query a chi di dovere. Nell'esempio stiamo cercando `www.cs.umass.edu`. L'ultimo server DNS è autoritativo, quindi restituisce la risposta, nel caso di soluzione ricorsiva fa il cammino all'indietro.

Come facciamo a fare il cammino all'inverso? Si ottiene perché abbiamo detto che le richieste DNS hanno un identificatore (generato dal resolver) e quello che succede è che quando il local DNS riceve un interrogazione se non ha risposta si ricorda che ha ricevuto quella richiesta con quell'identificatore da quel signore, così via per tutti i router DNS. Quindi stabilendo dei buffer di query DNS che rimangono in sospeso perché stiamo aspettando la risposta da qualcuno che sta ad un altro livello dell'infrastruttura DNS. Questo può portare delle code molto lunghe nei server che stanno in alto nella gerarchia.

A cosa serve ricordarsi da chi ho ricevuto cosa? Perché quando finalmente un server troviamo la risposta può vedere che la query con quell'identificatore è stata inviata da quel signore lì prende la risposta e glie la rimanda con lo stesso identificatore di query. Chi riceve fa caching della risposta e poi va a guardare nel buffer di query e trova che la risposta che gli è arrivata il cui identificatore

fa match con un identificatore di richiesta che ha ed allora rimanda indietro. Su tutto il cammino vengono seguite indietro i buffer e su tutto il cammino viene lasciata nella cache di ogni nodo la risposta. Il contro è che i nodi devono usare memoria (tanto più tanto sono in alto), devono stare ad aspettare, ma il costo di questo utilizzo di memoria viene ammortizzato dal fatto che quando la risposta arriva lascia tante tracce sul cammino, vuol dire che se qualcun'altro cerca la stessa cosa da qualche altra parte per scoprire qual'è l'indirizzo di rete ha molte possibilità di trovare l'indirizzo prima di arrivare al DNS .cs.umass.edu.

3.12.10 Soluzione iterativa pura

Abbiamo sempre il nostro local resolver che è il processo sulla macchina del cliente, se non trova sul cliente manda l'interazione ed entra nel local DNS e qui c'è la risoluzione iterativa, se il local DNS non ha la risposta guarda chi ha configurato come root e restituisce al local resolver l'identificativo della root, il quale nell'identificativo puro è il local resolver che va a chiedere alla root, la quale se ha la risposta glie la restituisce altrimenti restituisce il dominio di top level che potrebbe avere la risposta, il resolver va a contattare il DNS top level il quale se non ha la risposta, disfacendo l'indirizzo da destra verso sinistra, dà al resolver l'indirizzo per il server di dominio .umass.edu fino a che non si trova la risposta.

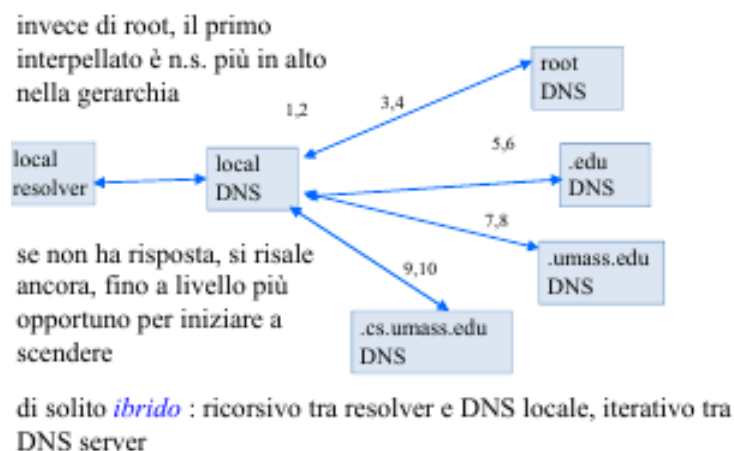


Figure 3.32: Soluzione iterativa pura

I vantaggi sono che nessuno ha bisogno di tenere in memoria niente, però non c'è neanche nessun caching perché la risposta non passa da nessun modo ma viene lasciata in cache nell'end system del cliente. Quindi anche l'iterativa pura non è il massimo, quello che si fa è un ibrido per cui il processing è fatto in questo modo: l'interazione è ricorsiva tra local resolver e local DNS, il resolver delega al proprio server locale l'informazione il quale si ricorda di aver ricevuto la query (quindi ha un buffer di richiesta in sospeso) e a sua volta agisce in modo

iterativo con gli altri DNS, partendo come prima dalla root. Quando la risposta arriva al local DNS lui si ricorda dal buffer da quale end system aveva ricevuto la query e rimanda la risposta a lui, in questo caso e' solo il server locale a mantenere in cache la risposta. Una variante di questa cosa e' non partire dalla root, ma avendo conoscenze a sufficienza, ma percorrere l'albero nell'altro senso, dal DNS locale che ha fatto la richiesta va a cercare in quello superiore (non va direttamente alla root), questo e' fatto perche' potrebbe essere che si cerchino informazioni vicino a noi. Anche in questo caso e' fatto in modo ibrido.

3.12.11 Risoluzione inversa

Magari vogliamo sapere quale nome simbolico corrisponde ad un dato indirizzo di rete. Inversa vuol dire che passiamo al DNS un indirizzo di rete (o 32 bit o 4 ottetti in decimale) quindi serve qualcuno che sappia risalire da questo ai nomi simbolici, le possibili combinazioni sono 2^{32} . C'e' un top level domain che non ci ha fatto vedere prima che si chiama ARPA che ha a che fare con la gestione della rete (quindi anche qui ci sono un po' di DNS server replicati per gestirlo). Il dominio ARPA mantiene un po' di informazioni sugli internet address, quindi c'e' un sottodominio di ARPA che si chiama IN-ADDR, nel momento in cui chiediamo al resolver una risoluzione inversa quello che parte e' una query DNS per un particolare nome simbolico. Noi stiamo cercando l'indirizzo IP 123.113.56.25 e' importante l'ordine di questi ottetti perche' come ci aveva gia' anticipato e suggerito gli indirizzi di rete hanno un significato gerarchico. Da qualche parte nell'indirizzo IP c'e' una divisione (non messa volutamente su un punto perche' non sappiamo quanti bit per network id e quanti per apparato, c'e' una divisione tra network id e device. La genericita' di questi ottetti e' al contrario di quella che avevamo prima, nell' indirizzo ip l'informazione sta piu' a sinistra fino a quella piu' specifica a destra (nel nome simbolico e' al contrario), quindi facciamo partire una query girando l'indirizzo IP: 25.56.113.132.in-addr.arpa, quindi la query parte da ARPA, che a sua volta passa ad in-addr i quali ricordano un albero in cui in primo livello ci sono 256 valori (0-255) noi stiamo cercando 132, quindi in-addr va a guardare le informazioni a 132, 132 ha 256 figli ed a noi interessa 56, anche 56 ha 256 figli ed a noi interessa il 25 arrivati alla fine arriviamo al DNS che ci interessa. Questa roba e' utilizzata ad esempio dai mail server per autenticare da chi arriva una richiesta di connessione.

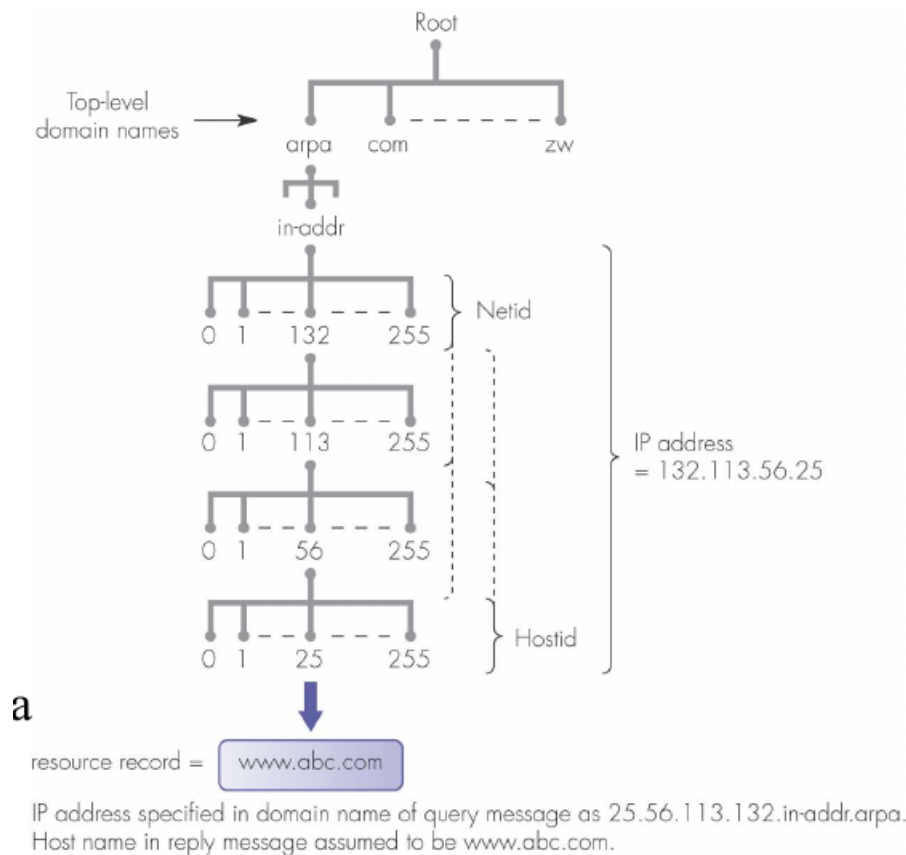


Figure 3.33: Risoluzione inversa

3.12.12 Osservazioni

I DNS moderni possono fare bilanciamento di carico (visto dopo aver parlato di Akamai), possono accettare una lista di indirizzi di rete (legati ad un solo indirizzo simbolico) e restituire a rotazione questi indirizzi a chi vuole quel servizio. Inserimento di record nel DNS: se parto dal basso quello che succede e' che se aggiungiamo un nuovo server, end system, pc siamo noi responsabili a registrare quel computer nel DNS. Se andiamo a livello piu' alto sotto i registry ci sono sparsi nel mondo i registrar, delle autorita' dei vari paesi che possono controllare l'unicita' dei domini, non c'e' un autorita' di dominio .it che genera dei figli ma ci sono dei registrar accreditati (ICANN) per i vari paesi e scegliendo il paese possiamo cercare chi sono le autorita' a chi chiedere se esiste un nuovo indirizzo di rete perche' vogliamo crearne uno nuovo. Il registrar inserisce l'informazione nel server .it determina chi sono i suoi 2 DNS di competenza e con il tempo i server di una zona si aggiornano e i record nuovi si propagano.

3.12.13 Dynamic DNS

Abbiamo detto che il valore *time to leave* e' legato alla stabilita' delle informazioni nel *resource record* ed essendo i server in dipartimento fissi il *time to leave* di questi server DNS e' alto (2 giorni). Ci sono delle situazioni in cui i server sono su apparati mobili, ci ricordiamo che un apparato mobile ha valore a livello di applicazione e non dipende dalla tecnologia che usiamo per andare in rete, dipende dalla posizione diversa nella topologia di rete. Se l'indirizzo di rete ha un indirizzo geografico (i bit piu' a sinistra) un apparato mobile che si sposta dinamicamente nella rete il suo indirizzo IP deve cambiare. Quindi l'host si fa dare un indirizzo dinamico DHCP per il tempo in cui rimane nella rete. Se io non sono folle il nome simbolico che associo all'indirizzo del mio PC non cambia ma l'indirizzo ip legato a questa stringa cambia man mano che mi sposto. Ci sono delle organizzazione che ci mettono a disposizione un servizio di *dynamic DNS*: Quando su un *end system* realizzo un sistema che deve essere identificabile nonostante gli spostamenti mi collego a questo DNS e quando mi collego alla rete mando al gestore e quando mi sposto e mi viene fornito un Ip avviso il *dynamic DNS* qual'e' il mio nuovo indirizzo IP. Quindi se in qualche modo il mio nome simbolico lo lego al *dynamic DNS* posso risolvere il problema del cambio IP.

Chapter 4

Livello di Trasporto

Livello 4. Due servizi standard TCP (connection oriented ed affidabile) e UDP (connectionless ed inaffidabile). Ricordiamo che e' il livello piu' basso di quelli end to end, non si vede la struttura della rete che sta sotto, il che significa che non ci importa nulla della topologia di rete. Lasciamo all'utente la scelta di quale servizio ha bisogno, in questo modo facciamo la rete leggera (perche' deve fare una cosa piu' semplice) e piu' in alto vicino all'applicazione ha senso dire che va bene il servizio di default o uno piu' raffinato che mi dia delle garanzie. Fare servizio affidabile e connection oriented significa che tutto deve arrivare ordinato, per fare questo basta appiccicare ai dati un numero il quale identifica l'ordine in cui sono stati inviati nella sequenza. L'affidabilita' e' piu' tosta perche' sotto abbiamo la rete (puo' essere Internet con diversi apparati e varie affidabilita' con un traffico incontrollabile e per nulla uniformemente distribuito 80/20), visto che questo traffico e' incontrollato l'affidabilita' sulla rete e' data dalle componenti di ritardo, dipende dal tempo di propagazione, dalla velocita' della CPU degli apparati di rete, il carico e le code sia sull'end system (server) sia sugli apparati di rete che stiamo usando. Le code ci provocano anche ritardi nell'inoltro dei dati, che possono cambiare la rotta e quindi arrivare in ritardo, ci danno il problema dei buffer overflow (se nei dispositivi si esaurisce la memoria qualcosa viene perso), tutto questo gioca contro l'affidabilita' (un pacchetto e' stato veramente perso o e' solo in ritardo). Per facilitarci l'affidabilita' (che i dati non vengano persi e non subiscano ritardi che ci anno pensare che siano andati persi) non dobbiamo caricare troppo la rete con i controlli di flusso (orizzontale, non mandare dati troppo velocemente dall'altra parte) verticali senza mandare dati troppo velocemente al livello sotto, inoltre c'e' un controllo degli errori (il discorso dei bit di controllo per capire se i bit ricevuti sono proprio quelli mandati). In piu' comunque indipendentemente da TCP o UDP dobbiamo fare indirizzamento a livello di trasporto, quindi distinguere le socket e questo lo facciamo utilizzando i numeri di porta. I numeri di porta ci servono per fare de-multiplexing (a che applicazione devo mandare i dati?).

4.1 Gestione degli errori

Ricordiamo che stiamo sempre cercando di implementare servizio affidabile e connection oriented. Per fare questa cosa abbiamo degli algoritmi (sono solo delle regole su cosa fare). L'idea con cui lavoriamo: sappiamo fare controllo degli errori ma c'e' riconoscimento degli errori e correzione degli stessi, in questi algoritmi stiamo parlando di rilevazione degli errori. Se rileviamo che una PDU (Protocol Data Unit) contiene degli errori la scartiamo. Gli algoritmi ARQ (algoritmi rilevazione degli errori, Automatic Repeat reQuest) fanno uso di due meccanismi:

- ACK (Acknowledgment): permette alla destinazione di confermare alla sorgente che ha ricevuto qualche cosa (tipo raccomandata A/R). Usato per verificare la corretta ricezione, se il sender non riceve l'ACK entro Δt ritrasmette il messaggio.

- NACK(negative acknowledgment): se succede qualche cosa il destinatario se ne accorge manda un NACK per dire alla sorgente di ritrasmettere il messaggio inviato, quindi se il sender riceve il NAK ritrasmette il messaggio richiesto.

Sono profondamente diversi, l'ACK costringe la destinazione a dire alla sorgente di aver ricevuto ogni singolo dato (pensando all'ordinamento sto assumendo che sto trasmettendo grandi quantita' di dati da mandare e spezzettati), gli ACK sono dei dati che viaggiano all'indietro quindi puo' essere perso anche l'ACK, se la sorgente lo riceve e' tranquilla che il dato e' arrivato ma se la sorgente non riceve l'ACK per un dato x-esimo puo' essere che la destinazione non gli abbia mandato l'ACK per x perche' non l'ha ricevuto, puo' essere nelle code e quindi e' solo in ritardo, puo' essere perso, puo' essere arrivato ma e' l'ACK che ha dei problemi. Quello che e' necessario fare e' che la sorgente quando manda un dato stima un tempo entro cui deve tornare indietro ACK, in qualche modo dev'essere la stima del tempo complessivo della somma dei ritardi su tutti gli hop, ammettendo di essere capaci di stimare questo tempo la sorgente se dopo il tempo non riceve i dati sa che qualcosa e' andato storto, allora mi metto nell'ottica di caso peggiore quindi ritrasmette il dato, per la destinazione non e' un problema perche' butta via x duplicato. Pero' dobbiamo essere capaci di stimare il tempo.

Il NACK sembra piu' leggero finche' tutto va bene nelle reti la destinazione non deve dire niente e noi non usiamo banda di rete se abbiamo ricevuto, la destinazione si lamenta solo se non abbiamo ricevuto qualcosa, ma anche il NACK puo' andare perso e nel caso in cui va perso la sorgente non si accorge di nulla. Ha anche un altro problema si chiama problema della stack, la sorgente deve mandare a destinazione 100 pezzi di dato, riceve fino al 96 e poi non riceve piu' niente, magari la sorgente ha mandato tutto e chiude il file li' mentre la sorgente che ha mandato anche gli altri e non ha ricevuto i NACK per lei va bene cosi'. Sembrava piu' leggero per la rete ma in realta' ci aiuta meno per capire se qualcosa va storto, quindi in ogni caso non possiamo usarli da soli, quindi nei nostri algoritmi useremo gli ACK. Non sono anonimi.

4.2 Protocolli connection oriented

Assunzioni (stiamo parlando di algoritmi e non di protocolli, siamo a livello astratto di descrizione di quello che le due unita' di livello trasporto fanno):

- Dati da Application Layer sempre disponibili
- Non considerato il contenuto dei dati nel il processing per error control, i dati ci arrivano dal livello applicazione quindi in assunzione con l'incapsulamento non ci guardiamo dentro
- Messaggi portano un numero di frequenza che serve per garantire l'ordinamento: #seq in 0,MAX_SEQ

- Taglia payload a cui vengono attaccati lo header di livello trasporto ha una dimensione massima fissata, e giustifica il fatto che se dobbiamo mandare tanti dati sono divisi in protocol data unit di tot dimensione. Taglia payload fissa in MAX_DATA byte.
- Lo header di livello 4 contiene almeno il tipo di messaggio (kind) che e' contenuto in quella protocol data unit, il numero di sequenza e ACK. Lo header dice che tipi di informazioni ci sono nel messaggio a livello trasporto, c'e' dentro un numero di sequenza che e' un informazione legata ad un dato, se c'e' il di informazione che viaggia è un dato ci sarà il campo numero di sequenza invece se il tipo di informazione e' ACK ci sarà uno ACK, la destinazione sta dicendo che ha ricevuto qualcosa, ma lo ACK non e' anonimo, dice che ha ricevuto e cosa ha ricevuto (il numero di sequenza collegato al dato che ha ricevuto). Il message header contiene: kind, #seq, ack. I NAK si possono opzionalmente usare, quindi volendo ci potrebbe essere anche un tipo NACK.
- Il canale fisico e' inaffidabile e' puo' perdere o alterare messaggi, un messaggio alterato viene buttato via e perso. La perdita di dati puo' derivare da buffer overflow (traffico in rete, nodi che sono congestionati che esauriscono la memoria e buttano via).

I tempi devono essere variabili, un timer piccolo puo' provocare dei problemi se i messaggi ack sono anonimi, se non sono anonimi non provoca errori ma delle ritrasmissioni inutili, un timeout grande puo' comunque fare diventare grande tutto il sistema. E' quindi un problema spinoso capire quanto ci mette un dato ad arrivare a destinazione e l'ack a tornare indietro, dobbiamo stimare il round trip time (tempo per fare un viaggio dalla sorgente alla destinazione e ritorno).

4.3 Very silly protocol

Arriviamo agli algoritmi funzionanti con una procedura costruttiva, c'e' una ragione quindi se il diagramma temporale di questo algoritmo e' dichiaratamente molto stupido stupido.

In immagine abbiamo un diagramma temporale, stiamo parlando di interazione tra sorgente e destinazione che devono scambiarsi i dati in modo che arrivino tutti. Sorgente rileva dati da livello applicazione dove ci sono, i dati devono essere il payload del passaggio al livello di trasporto da inviare, prepara lo header con cui incapsulare i dati, manda la sua protocol data unit al livello network. Tutto questo non e' fatto in tempo istantaneo ma ci mette un certo tempo Δt_s . La destinazione aspetta dati da sorgente, quando arriva qualche cosa dal livello sottostante (network) per il processo che implementa l'algoritmo di livello trasporto (qui c'e' del demultiplexing perche' network capisce che deve passare i dati a protocollo di trasporto TCP che realizza il servizio connection oriented). Prende i dati li astrae, controlla che siano corretti, se tutto va bene li passa al livello applicazione e per fare queste cose ci mette il suo tempo Δt_d .

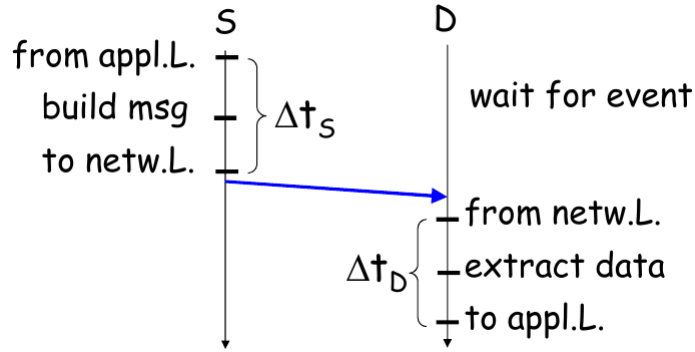


Figure 4.1: Very silly protocol

Dopo di che tutti e due riprendono il loro lavoro, se ci sono altri dati in attesa ricominciano a fare quello che stavano facendo (la sorgente torna a prendere i dati e la destinazione torna in attesa). I tempi (Δt_S e Δt_D) noi non li conosciamo (dipende da quanto tempo il processo ci mette ad avere il controllo della CPU), che cosa succede se un dato mandato dalla sorgente viene perso in rete e non arriva alla destinazione, la sorgente passa a preparare il dato successivo e manderà quel dato alla destinazione, la destinazione continua ad aspettare e se gli arriverà un dato successivo solo in quel momento si sbloccherà dall'attesa preleverà i dati dal livello network e farà le sue operazioni, questo significa che se abbiamo una rete che può perdere dei dati (lossy) l'algoritmo viola il suo contratto di servizio con i livelli superiori (se un dato viene perso sopra nessuno saprà niente) quindi non garantisce affidabilità e neanche l'ordinamento, va bene se siamo certi che la rete non ha nessun problema, ma così non è. Supponiamo anche che la rete sia perfetta e non possono essere persi dati, se la destinazione è lenta a processare rispetto alla sorgente e il livello di trasporto della destinazione non ha un buffer infinito chiaramente la coda della destinazione si allunga e prima o poi si riempirà e la destinazione è costretta a buttare via qualche cosa. Quindi anche se la rete fosse perfetta abbiamo sempre un problema di smarrimento, nel momento in cui la destinazione butta via qualche cosa (ricordando sempre che c'è una coda ai vari livelli) perché il livello network riempie il buffer perché il livello di trasporto è lento passa quello che trova, ma ovviamente si creano dei buchi, è un problema di controllo di flusso orizzontale, tale per cui la sorgente non dovrebbe andare più velocemente della destinazione. Possiamo pensare di fare una cosa, inserire delle attese fisse nella sorgente (se destinazione è lenta la sorgente deve adeguarsi) se è fissa può andare molto male ed essere inutile comunque perché se l'attesa è troppo corta postponiamo il problema ma prima o poi il buffer si riempirà comunque, se d'altra parte l'attesa è lunga abbiamo un altro tipo di problema ci sono dei tempi morti nel sistema in cui nessuno sta facendo niente (perché se imponiamo un'attesa molto lunga alla sorgente lei una volta che invia il pacchetto sta ferma, mentre la destinazione

una volta che ha ricevuto e processato si mette in attesa) e questo e' un uso inefficiente dell'infrastruttura di rete (tempo grande inefficiente tempo piccolo inefficace). Quindi non vanno bene i tempi fissi, dobbiamo in qualche modo capire i tempi. Attenzione che stiamo parlando di end system, non e' che capire da parte della sorgente la destinazione quanto tempo ci mette a fare le sue cose, perchè il tempo cambia perchè dipende dal carico, quindi dovrebbe adeguarsi di volta in volta, altrimenti si ricade o nel caso di tempo troppo piccolo o troppo grosso.

4.4 Silly protocol

E' un aggiunta rispetto a quello che abbiamo visto prima. I dati passano dalla sorgente alla destinazione come prima poi la sorgente si mette in attesa, la destinazione come prima prende i dati se passano il controllo di correttezza li decapsula e differentemente da prima la destinazione manda un ack alla sorgente che quando riceve l'informazione a livello network esce dall'attesa e ricompie il proprio ciclo. Ci sono delle criticita' anche in questo caso sempre riguardo ad efficacia ed efficienza. Qui non vale piu' il problema del controllo di flusso perchè la destinazione può metterci quanto vuole perchè la destinazione aspetta l'ACK, pero' forse non e' la cosa piu' efficiente da fare, perche' i processi lavorano in stretta alternanza (half-duplex), cioe' la sorgente per un po' dopo aver mandato il dato non fa niente fino a che non gli torna l'ack e allo stesso modo la destinazione, quindi il 50% del sistema sempre non fa assolutamente niente perchè è in attesa di qualche cosa. Se siamo in un canale non affidabile può andare perso qualche cosa, può succedere un deadlock, si perde il messaggio, la sorgente rimane in attesa dell'ack e la destinazione rimane in attesa del messaggio che non arriverà mai. La stessa cosa succede se viene perso l'ACK, perchè la sorgente rimane in attesa dell'ACK e la destinazione rimane in attesa del messaggio successivo.

4.5 Positive Acknowledgment with Retransmission (PAR)

E' molto simile a quello che avevamo prima, abbiamo aggiunto la riga seq, start timer, il controllo con un if della sorgente ed un'altro controllo nella destinazione. Inizialmente la sorgente e la destinazione si tengono due variabili: next_msg(fino a che dato ho mandato, serve per decidere il numero di sequenza da appiccicare al pacchetto) e msg_expeceted (fino a che dato ho processato, mi aspetto il dati n+1) entrambe inizializzate a 0. All'inizializzazione la sorgente pesca dei dati dal livello applicazione ci mette il proprio header nello head mette il numero di sequenza passa i dati al livello network e fa partire un timer aspettando che succeda qualche cosa, la destinazione aspetta i dati, se li riceve guarda se passano il controllo di errore, li preleva dal livello network, guarda nello header il numero di sequenza se il messaggio è quello atteso lo manda al livello

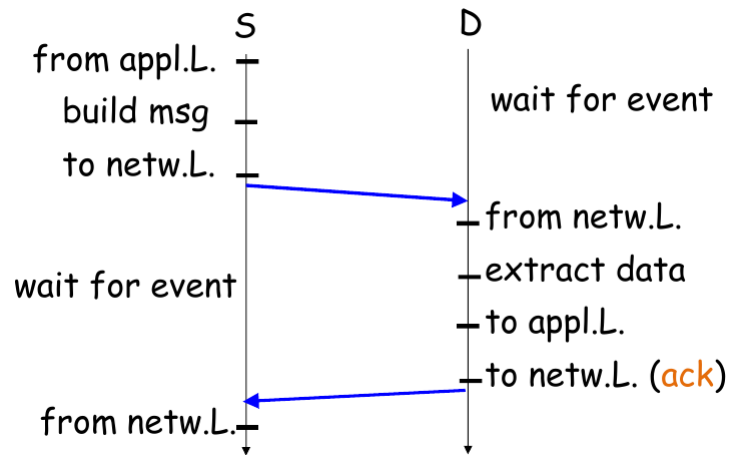


Figure 4.2: Silly protocol

applicazione e aumenta il contatore `mesg_expected` e manda un ack (anche se il messaggio non è quello aspettato) che non dice solo di aver ricevuto i dati ma ha un argomento che è il valore di `mesg_expected`. A questo punto alla sorgente può succedere che o arriva qualcosa dalla rete o scade il timer, se si sblocca perché è arrivato un messaggio corretto aumenta la variabile `next_msg` pesca altri dati da livello applicazione e riprende il ciclo, se invece si sblocca perché è scaduto il timer non entra nell'if perché non viene mai verificato: e la sorgente costruisce un nuovo messaggio con i dati che aveva mandato la volta precedente (non li pesca dal livello applicazione) e non incrementa il valore della variabile `next message` ma appiccica ai dati lo stesso numero di sequenza, fa una ritrasmissione della stessa cosa. A livello destinazione, se il messaggio non è corretto e non passa il controllo butta via il messaggio e manda alla sorgente un ack dicendo di aver ricevuto il messaggio prima, questo serve a rompere il deadlock che avevamo prima, se la destinazione buttasse via qualcosa che non è giusto e non dicesse ack come prima ritornerebbe ad aspettare un messaggio, la sorgente si sbloccherebbe perché c'è il timer ma rimanderebbe un duplicato. Stiamo coprendo forse tutte le possibilità di errori. Il numero di sequenza previene la perdita di duplicati nel caso in cui l'ack dovesse perdersi. Quando si perde l'ack non si creano neanche duplicati perché anche nel caso di continue perdite dell'ack il sender rimanderebbe sempre lo stesso messaggio però non entrerebbe mai nel secondo if del ricevente, perché non è il messaggio expected, quindi non viene mai passato al livello applicazione. Nel caso in cui invece arrivasse un messaggio non corretto, il primo if della destinazione non è verificato, ma la destinazione si fa rimandare il messaggio con l'ack, se anche l'ack non arriva comunque la sorgente prima o poi si sblocca (con il timer) e rimanda lo stesso messaggio. Questo è un algoritmo che funziona.

Con il timer abbiamo risolto il problema del deadlock (come abbiamo già

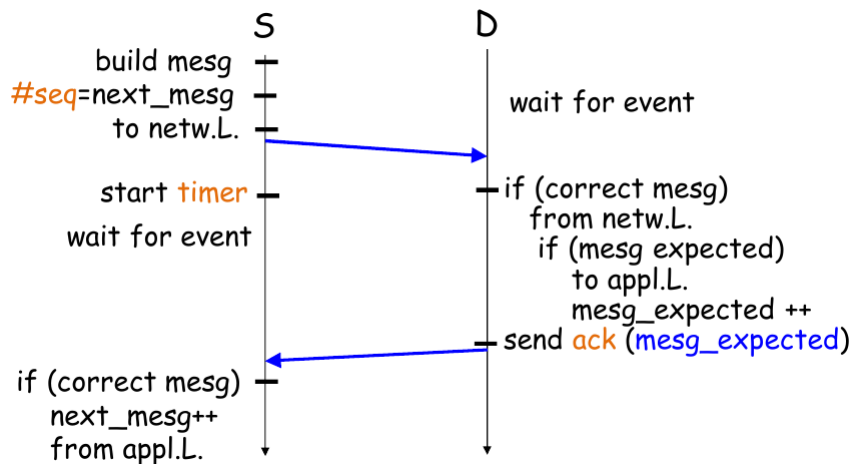


Figure 4.3: PAR

visto), il numero di sequenza previene la perdita di duplicati in caso di perdita dell'ACK, nell'ACK abbiamo aggiunto un'informazione in più, il valore di `message expected`, si chiama ACK cumulativo dice alla sorgente che tutto quello che viene prima dell'`message expected` è a posto per la destinazione.

Adesso proviamo a risolvere un altro aspetto (per passare da algoritmo a protocollo dobbiamo aggiungere lo header), quando abbiamo guardato i protocolli a livello applicazione abbiamo visto i messaggi di trasporto e dobbiamo capire quanto spazio occupano gli header, è importante perché quello che arriva a livello network è lo header di livello 4 che è costituita dall'header di livello 4 ed il payload. In header di liv 4 c'è il numero di sequenza (con cui sto etichettando i dati che mando) che è rappresentato su un numero di bit, fatto salvo che non sappiamo (e non vogliamo sapere) quanti dati dobbiamo passare, il nostro livello di trasporto deve funzionare indipendentemente da cosa gli viene passato dall'alto, ma abbiamo capito che ad un certo punto (liv network) i dati possono essere frammentati, scelgo una dimensione massima e spezzo i dati con quella dimensione e mando quelli.

Posso mettere un limite superiore al campo di sequenza che è una parte dell'header di livello 4? perché mi permetterebbe di avere un header di livello 4 di dimensione massima fissata, c'è dentro il tipo (2 bit), numero di sequenza, il numero di acknowledgment. Nell'header c'è anche il numero di porta della sorgente e della destinazione (sono 16 bit, gli indirizzi di livello trasporto), la variabile quindi è il numero di sequenza, se abbiamo numeri molto grandi può essere un valore molto grande ed avere un numero di bit illimitato o riusciamo a metterci un limite superiore? Ha un massimo che è 1, sorgente può alternare il numero di sequenza 0 e 1 senza problemi. Non c'è il pericolo che si confondano i messaggi marcati tutti uguali perché la sorgente manda alla destinazione il primo messaggio con numero di sequenza 0 e fino a quando la destinazione non

ha ricevuto questo messaggio 0, fa diventare message expected = 1 e torna l'ack = 1 alla sorgente questa manderà sempre 0, quando lo riceverà l'ack manderà 1, quando la destinazione aumenta il suo contatore lo aumenta a 0 (che è il nostro numero successivo). Non si crea ambiguità perché la sorgente non può interpretare 0 come quello precedente, perché la sorgente sa per certo che tutti i messaggi prima dell'ack che mi arriva per la destinazione sono a posto. Quindi 1 bit è sufficiente perché abbiamo una sorgente ostinata, ovvero resta bloccata su quello che sta mandando fino a che non è sicura che la destinazione lo abbia ricevuto e dopo essere sicura che la destinazione lo ha ricevuto passa al messaggio successivo consapevole che quello prima è stato ricevuto.

4.5.1 PAR:ack numerati

È importante che questo bit ci sia (il numero di sequenza attaccato al messaggio o la numerazione degli ACK), dobbiamo considerare i numeri arancioni come se ci fossero ma non ci sono, ci manda un messaggio che noi sappiamo essere lo 0 senza averci appiccicato il numero di sequenza. Il timer scade il messaggio ci ha messo un po' di tempo per mandare il messaggio, la destinazione ci manda l'ack perché l'ha ricevuto senza però appiccicarci il numero di sequenza, scade il timeout della sorgente e fa ripartire il dato 0 perché non ha ricevuto ack, ma in questo momento arriva l'ack della destinazione, la sorgente rimanda il messaggio zero e riceve poco dopo un'ack dalla destinazione che non è numerato ma si riferiva al messaggio 0, quindi è convinta che il messaggio uno sia a posto di conseguenza manda il messaggio 2, si perde quindi il messaggio 1, la destinazione manda ack perché lo manda sempre ed è un casino si sovrappongono. Quindi è necessario che i messaggi e gli ack abbiano il numero a bordo, perché il timer può essere dimensionato male e fare ripartire i messaggi prima che torni l'ack della destinazione. In questo caso il timer era troppo piccolo dato che i messaggi erano anonimi, nel caso in cui i messaggi non fossero anonimi il timer troppo piccolo non causa problemi di funzionamento ma di efficienza.

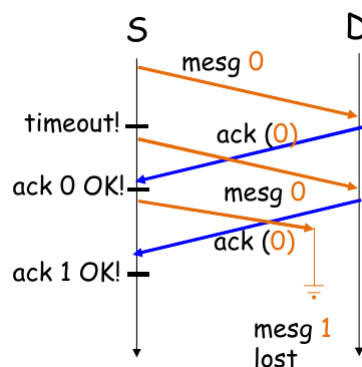


Figure 4.4: Perché gli ACK sono numerati

4.5.2 ildeRQ (Stop & Wait) - sender

E' l'algoritmo vero con cui si implementa il PAR e il piu' semplice, come stop and wait. Vediamo un po' come funziona. Retry counter (problema dei due generali) ci dice quante volte proviamo a reinviare il messaggio se non viene ricevuto.

Definiamo $maxseq = 1$, se il corrente è stato ricevuto significa che è stato ricevuto tutto quello che c'era prima, quindi quando riparto con la sequenza non ho problemi, il valore di $nextmessage = 0$. Metto tutti i dati in un buffer (passo il parametro per riferimento a livello applicazione). Costruisco quello che devo mandare dall'altra parte sta per segmento, il payload sono i dati nel buffer, il numero di sequenza è il valore di $nextmessage$, passo il segmento al livello network e faccio partire un contatore di ritrasmissioni ($retry_counter$), facciamo partire il timer ed entriamo in un ciclo infinito, aspettando un ACK (o un NACK) oppure la sorgente vede scadere il timer. Se riceve l'ACK con il valore = a quello del messaggio che ha mandato aumento il valore del prossimo messaggio, setta = 1 il valore del contatore di ritrasmissioni. Se invece riceve un NACK o vede un timeout incrementa il valore di ritrasmissioni, se abbiamo raggiunto il massimo comunica al livello superiore un errore e ci fermiamo, altrimenti andiamo avanti, il sender costruisce un nuovo segmento con i dati nel buffer, in pratica costruisce un nuovo segmento con i dati ed il numero di sequenza che aveva prima, manda tutto al livello sotto e fa ripartire il timer. Il NACK mi permette di non dover aspettare il timeout, timer lo stimiamo un po' più grande del round trip time.

```
#define MAXSEQ 1
next_mesg=0;
from_appl.L(&buffer)
s.info=buffer;
s.seq=next_mesg;
to_netw.L(&s);
retry_counter = 1;
start_timer(s.seq);
while (True) {
    wait for event; // ack, nak, timeout
    if (r.ack == next_mesg) {
        from_appl.L(&buffer);
        next_mesg++;
        retry_counter = 1;
    } // end if
    if (r.nak == next_mesg or timeout) {
        retry_counter++;
        if (retry_counter > MAXRETRY)
            to_appl.L(error); EXIT;
    } //end if
    s.info=buffer;
    s.seq=next_mesg;
```

```

    to_netw_L(&s);
    start_timer(s.seq);
} //end while

```

4.5.3 idleRQ - reciver

Definisce come massimo numero di sequenza = 1 (tutto il ragionamento che ci basta alternare 0 e 1 come numero di sequenza), inizializza il numero di messaggio atteso a 0 ed entra in un ciclo infinito (astraggono lo scambio di una lunga sequenza di messaggi) attendendo un evento:

- Ricezione di un messaggio corretto
- Ricezione di un messaggio danneggiato

La ricezione di un messaggio danneggiato non porta informazioni in termini di dati (il ricevente non può fare nulla, non possiamo correggerlo), però ci dice che il sender ha cercato di mandarci qualche cosa ma quel messaggio è stato modificato dalla rete, quindi il ricevente può intraprendere azioni. Se arriva un messaggio corretto lo preleva dal livello rete e lo mette in un buffer e ci guarda dentro (nello header) se il numero di sequenza nello header è quello che si aspettava inoltra il messaggio al livello applicazione e manda al livello network del sender l'ack con il numero del messaggio aspettato. Altrimenti ha ricevuto un messaggio corretto con numero di sequenza non atteso, può succedere quando il timeout è troppo piccolo e il sender rimanda il messaggio prima di ricevere l'ack di quel messaggio dalla destinazione, quindi se arriva un duplicato la destinazione lo scarta e poi manda l'ack al sender con il valore associato di message expected. Se è arrivato un messaggio danneggiato lo scartiamo, ma comunque mandiamo un nack alla sorgente con il numero di messaggio aspettato per velocizzare i tempi e non aspettare la ritrasmissione di quel messaggio quando scade il timer della sorgente.

```
#define MAX_SEQ 1
```

```

mesg_expected=0;
while(true) {
    wait for event;
    //correct mesg, damaged mesg
    if(correct mesg arrival) {
        from_netw_L(&r);
        if(r.seq==mesg_expected) {
            to_appl_L(r.info);
            to_netw_L("ack", &mesg_expected);
            mesg_expected++;
        } //end if expected
    } else { //duplicated mesg
        discard duplicate mesg;
    }
}

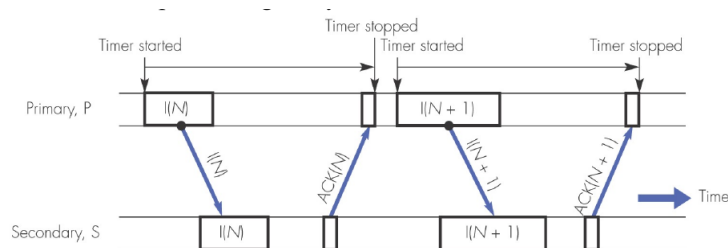
```

```

        to_netw_L("ack", &mesg_expected);
    } // end else duplicate
} // end while

```

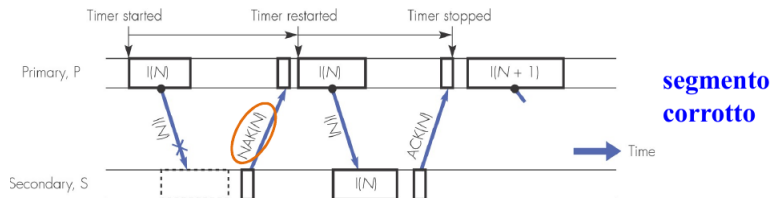
La logica di questo pseudocodice non è corretta perchè il ricevente quando è contento incrementa il valore di message expected, se gli arriva un messaggio danneggiato chiede al livello network di mandare un nack con il valore incrementato, se invece abbiamo ricevuto un messaggio duplicato dobbiamo dire al livello network di mandare al sender un ack con il valore del messaggio precedente a message expected, nel primo else va quindi mandato un ack con valore mesg_expected - 1.



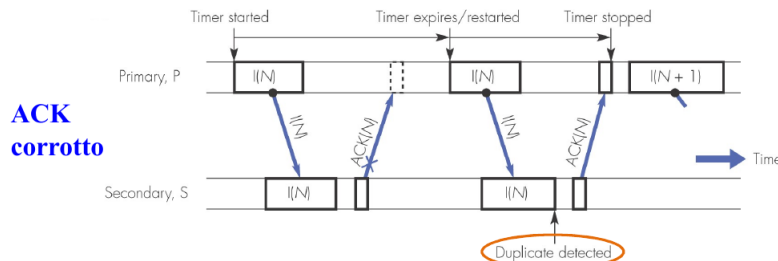
Ad un certo punto vengono fatte cadere le etichette di sorgente e ricevente, perchè possiamo supportare comunicazione bidirezionale (full duplex), allora ricordando che lo header è una tara nel costo delle comunicazioni (il netto è il payload) perchè il lordo è quello che vendono gli internet service provider ma non lo vediamo mai perchè parte della rete la usiamo per la tara, vogliamo quindi usare al meglio il canale minimizzando la tara. Dicevamo che lo header deve contenere almeno una serie di informazioni e sono fissate perchè è un protocollo di livello 4, ciò significa che è stupido mandare dei dati indietro prefissi dallo header e poi uno header da solo, quindi nello header dei dati mando anche le informazioni di controllo che devo mandare, l'ack, questo si chiama piggybacking (cavallina), l'idea è di mettere in groppa ai dati che viaggiano nell'altra direzione anche l'ack, questa cosa fa usare un po' meglio la banda (o la capacità del canale). Il problema è che ci vuole un po' di cura nel codice del ricevente per dire che vogliamo mandare dall'altra parte un ack, però sappiamo che se mandiamo subito l'ack e non stiamo ricevendo dati dai livelli superiori mando header e basta, quindi aspetto un attimo e se mi arrivano i dati carico in groppa allo header l'ack così spedisco tutto insieme, però il ricevente non deve aspettare troppo altrimenti al sender scade il timer e rimanda il messaggio ed a questo punto la ritrasmissione costa troppo. Questo protocollo è sensibile al round trip time (dimensionamento del timer), se il timer è troppo piccolo rischiamo di fare ritrasmissioni inutili. Solo una parte lavora perchè si aspettano gli ack, quindi il canale è inutilizzato.

4.5.4 Situazioni di errore

Segmento corrotto: Quello con numero di sequenza N è mandato dal primario ed il secondario capisce che è sbagliato e manda un nack, appena arriva il primario rimanda il messaggio, l'utilità del nack è che può stringere un po' il tempo. Ricordiamo che il nack da solo non si usa, si può usare insieme all'ack per provocare da parte del sender la ritrasmissione di qualcosa.



ACK corrotto: il primario manda il segmento con numero sequenza n, il secondario manda l'ack n che può essere corrotto o non arrivare proprio quando scade il timer il primario rimanda il numero di sequenza n, il ricevente capisce che è un duplicato quindi lo butta via e rigenera un ack con numero di sequenza n.



4.5.5 Utilizzo canale

Diciamo che la quantità di dati che voglio mandare è l (lunghezza) R è la misura del round trip time b è il bit rate del canale (quello che abbiamo definito come velocità di trasmissione bit per secondo, info da digitale ad analogica), il ritardo di trasmissione è la quantità di dati da mandare diviso il bitrate $T_x = \frac{l}{b}$, il ritardo di propagazione è la lunghezza del cavo tra due dispositivi diviso la velocità di propagazione (velocità luce per wireless o $\frac{2}{3}$ velocità della luce per il rame) $T_p = \frac{R}{2}$. La sorgente impiega $T_x = \frac{l}{b}$ per mettere i dati sul cavo, questi impiegano R per andare dall'altra parte e l'ack della cui taglia non ci occupiamo ci mette l'altra metà del round trip time. Noi stiamo usando la linea per quel $\frac{l}{b}$ (il tempo per trasmettere i dati) e poi non fa niente per un round trip time, quindi misuriamo in questo modo quanto tempo hai usato il mezzo trasmissivo e quanto tempo hai aspettato l'ack? Noi facciamo un rapporto tra il tempo T_x e tutto il tempo in cui il dato è trasmesso e torna l'ack :

$U = \frac{T_x}{T_x + 2T_p} = \frac{\frac{l}{b}}{\frac{l}{b} + R} = \frac{l}{l + bR}$. Si possono fare delle osservazioni su questo tempo di utilizzo, se l'è in questo modo di scrivere l'utilizzo ci dice che è più piccolo del 50%, quindi passo meno della metà del tempo ad aspettare il canale e più della metà del tempo ad aspettare. Però b dipende dall'hardware della scheda di rete, il round trip time dipende dal mezzo trasmissivo e quanto è lontano da noi, ma qualcosa possiamo fare su l la quantità di dati da trasmettere, la dimensione di un segmento posso deciderla dentro l'implementazione dell'algoritmo, possiamo mandare delle non trascurabili quantità di dati per aumentare l e ammortizzare il costo dello header.

Pipelining

C'è un'altra cosa da considerare, non ci piace quando un sistema è in idle (il sistema non fa niente). Supponiamo di avere un sistema con rtt (round trip time) di 500 msec (tipico di una rete satellitare), al taglia dei dati da mandare è di 1 Kb ed il bitrate del canale è di 50 Kbps:

- Transmission delay: $\frac{1Kbit}{50Kbps} = 20msec$
- Propagation delay: $\frac{rtt}{2} = 250msec$
- arrivo a $250 + 20 = 270msec$
- ack ricevuto a $\leq 250 + 270 = 520msec$
- sender aspetta per $500msec$
- utilizzo rete $\frac{20msec}{520msec} = 3.85\%$
- Utilizzo 100% per $\frac{520}{20} = 26$ messaggi outbound

Immaginiamo un nastro trasportatore tra due persone, una ci mette un pacco e invece che stare a fare niente quella persona mette sopra un altro pacco, l'altra lo riceve e mette sul nastro una cartolina che dice che lo abbiamo ricevuto, facciamo così con il pacco successivo, a regime il nastro da una parte è pieno di pacchi e dall'altra parte è pieno di ricevute, abbiamo riempito il canale al 100% ed è proprio quello che vogliamo. Da quello che abbiamo visto mandiamo 25 messaggi alla volta così quando mandiamo il 26esimo riceviamo il primo ack del primo messaggio e mandiamo il 26esimo.

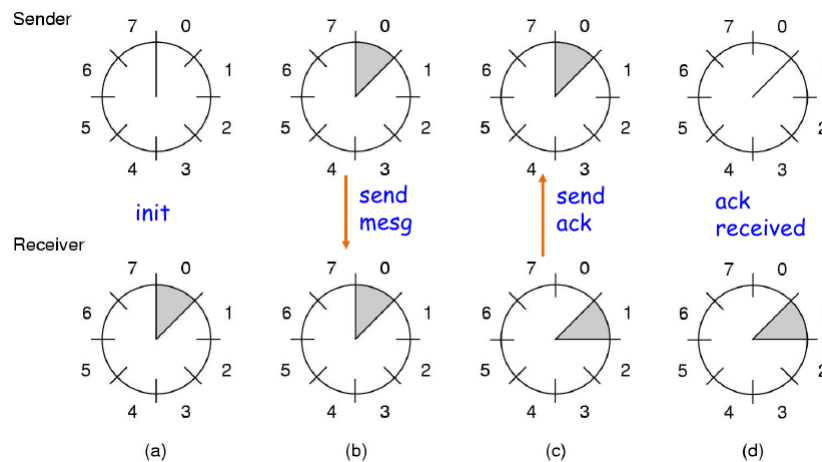
4.6 Sliding window

IDLE RQ va bene quando il round trip time ed il bit rate è piccolo se c'è un basso ritardo di propagazione, quindi reti locali con basse prestazioni. Al contrario se i link non sono bassi ed il bit rate è alto forse idle RQ non è una cosa ideale, meglio avere parallelismo tra messaggi, il sender tiene una finestra in cui tiene i segmenti che ha mandato dall'altra parte in attesa di un ack, nel buffer di tutti i messaggi che ho mandato posso eliminare il più vecchio quando

ricevo un ack, poi posso spostare la finestra mettendo il penultimo come ultimo. Dall'altra parte il ricevente tiene una finestra in cui ha il numero di sequenza dei messaggi che è disponibile a ricevere, nel caso più semplice la finestra del ricevente è grande 1 (si aspetta un solo messaggio), ma possiamo fare qualche cosa di più, possiamo accettare anche messaggi più avanti tenerli lì ed aspettare che arrivino tutti. Abbiamo detto di avere una rete datagram, ogni pacchetto a livello rete è indipendente da tutti gli altri ed i router monitorano la rete, se le condizioni di rete cambiano due pacchetti dalla stessa sorgente a destinazione possono seguire cammini diversi, quindi può essere che il ricevente di permetta di mantenere un po' di spazio di memoria in caso in cui un pacchetto mandato dopo arrivi prima di quello mandato precedentemente. Il parallelismo aumenta l'utilizzo del canale a spese di un overhead della memoria, perchè sorgente e destinazione devono tenere questi buffer (windows).

4.6.1 Esempio sliding window 1-bit

Il sender all'inizio non ha nulla di mandato e si aspetta l'ack 0, quando arriva al ricevente è contento manda l'ack 0 e fa scorrere la finestra. Quando l'ack arriva al sender fa scorrere la finestra che si riaprirà quando invia un altro messaggio ed il receiver si aspetta il messaggio 1.



4.6.2 Codice

```

#define MAX_SEQ 1
next_mesg=mesg_expected=0;
from_appl_L(&buffer);
s.info=buffer;
s.seq=next_mesg;
s.ack=1- mesg_expected;
to_netw_L(&s);
start_timer(s.seq);
while (true) {
    wait for event;
    // correct mesg, damaged mesg, t.o.
    if (correct mesg arrival) {
        from_netw_L(&r);
        if (r.seq==mesg_expected) {
            to_appl_L(r.info);
            mesg_expected++;
        } // end if
        if (r.ack==next_mesg) {
            from_appl_L(&buffer);
            next_mesg++;
        } // end if
    } // end if
    s.info=buffer;
    s.seq=next_mesg;
    s.ack=1- mesg_expected;
    to_netw_L(&s);
    start_timer(s.seq);
} // end while

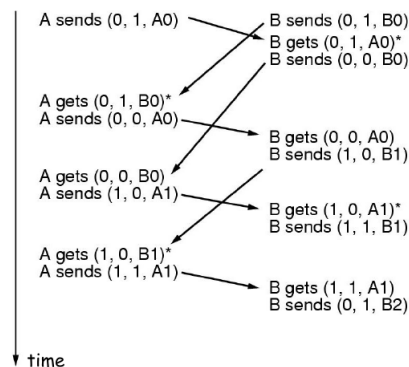
```

In questo codice c'è full duplex (sender e receiver insieme). Stiamo aggiungendo full duplex a stop and wait (sempre 1 bit), il massimo numero di sequenza è 1, se ogni processo è sia sorgente che ricevente allora next message e expected message sono tutti uguali a 0, prendo i dati dal livello applicazione e li metto nel buffer, il numero di sequenza che associo ai dati è il valore di next message, l'ack che carico sul segmento è il complemento del numero di sequenza. Passo a livello network il segmento, faccio partire il timer ed entro nel ciclo infinito aspettando un evento:

- Un messaggio corretto
- Un messaggio corrotto
- Nack o scade il timer

Se arriva un messaggio corretto lo pesco se il numero di sequenza è quello aspettato passo il payload a livello applicazione, incrementa il numero di message expected carico il prossimo. Indipendentemente dal numero di sequenza che mi è arrivato se il messaggio corretto carica in piggyback un ack che è uguale al numero di sequenza del segmento che sto mandando prendo dati nuovi da applicazione e metto nel buffer con nuovo numero di sequenza. Qui trascuriamo che sia arrivato un messaggio danneggiato e che mi sia sbloccato per timeout, quindi per qualche ragione mi sono sbloccato e mando un segmento che ha il contenuto del buffer e come numero di sequenza ha il valore di next message, che resta quello vecchio se mi sono sbloccato senza ack, l'ack che mando è 1 - message expected. E' ancora stop and wait. Se il segmento che ho mandato prima è stato perduto con l'ack del segmento prima, ma il round trip time è più grande e ricarico l'ack sul segmento dopo magari riesco ad evitare ritrasmissioni inutili.

C'è una situazione malata: le due parti iniziano a mandarsi i dati contemporaneamente i due sistemi vanno fuori sincro. Non dobbiamo far partire contemporaneamente i messaggi da tutte e due le parti.



4.7 Continuous RQ

Da IDLE RQ, la parola idle ci ricorda che si alternano nel lavoro sorgente e ricevente, passiamo a continuous RQ per cercare di alzare l'utilizzo della rete usiamo le stazioni sempre. Gli algoritmi sono Go back N e selective repeat, sono tutti e due continous RQ, è permesso mandare una sequenza di messaggi senza ricevere ack, la differenza sta nel comportamento del ricevente. Aumenterà il max seq, in go back n il ricevente ci mette poco lavoro e poca memoria perchè ha una finestra di dimensione 1, in ogni istante il ricevente si aspetta il prossimo messaggio nell'ordine, se arriva qualsiasi altra cosa la butta via, però in go back n vengono buttati via anche messaggi successivi al prossimo atteso (hanno numero di sequenza maggiore). Come può succedere (quando abbiamo parlato di instradamento e routing datagram) abbiamo capito che a livello network i router si costruiscono una visione della topologia di rete e sulla base di questo grafo pacchetto per pacchetto viene inoltrato lungo quello che per il router è la strada migliore, se ci sono cammini migliori e c'è un cambio di percorso (messaggio è il payload del pacchetto di livello 3) i pacchetti dentro cui sono incapsulati i messaggi possono prendere cammini differenti e magari un pacchetto precedente ha preso una strada peggiore e quello successivo supera il predecessore ed arriva prima a destinazione, quindi può essere che messaggi più avanti arrivino prima di quelli attesi, anche questi messaggi, ricevuti fuori ordine, vengono buttati via dalla destinazione. Significa che la destinazione ha un buffer piccolo di 1 messaggio. Il problema è che c'è molto poco uso di memoria da parte del ricevente, c'è molto poco processing da parte del ricevente (perchè si aspetta solo il messaggio con numero di sequenza x e butta via il resto). Quindi è vero che con go back n se il round trip time è grande ho un alto spreco di banda (in idle rq se il round trip time è grande il sender ha un grande tempo di attesa, siamo quindi passati ad un approccio sliding window pensando che in tutto questo tempo invece che fare niente potrebbe inviare altri messaggi riempiendo la pipe), quindi se il round trip time è grande per riempire la pipe ci vogliono tanti messaggi, se succede qualche cosa a qualcuno di questi, ad esempio il messaggio x è quello atteso dal ricevente e non vedendolo arrivare

(è successo qualcosa nella rete) e dietro di lui ci sono altri messaggi, ma in go back n il ricevente sta aspettando il prossimo quindi tutti i successivi messaggi $x + n$ vengono buttati via. Dignifica che se in go back n un messaggio ha un problema e non viene ricevuto dalla destinazione, questa butta via tutti i successivi nella pipeline, se ho un problema ad un messaggio in realtà la sorgente deve ritrasmettere tutti i successivi che ha nella finestra di invio. Questa è la ragione dell'alto spreco di banda.

Alternativa è l'approccio selective repeat, la destinazione è in grado, entro determinati limiti, di mantenere in memoria messaggi fuori ordine, quindi la finestra non è 1 ma $k \geq 1$. Di conseguenza può tenere in memoria $k-1$ messaggi, aspettando che arrivi il messaggio con numero x e poi mandare tutto a livello rete. Richiediamo maggiore gestione e occupazione di memoria da parte della destinazione (la dimensione è un parametro che dipende da fattori), abbiamo maggiore processing da parte di destinazione perchè deve:

- Controllare che il numero di sequenza del messaggio stia nel buffer e metterlo lì
- Vedere se ho il messaggio successivo, se c'è l'ho lo mando al livello applicazione e controllo nel buffer se ho altri messaggi successivi da consegnare al livello applicazione

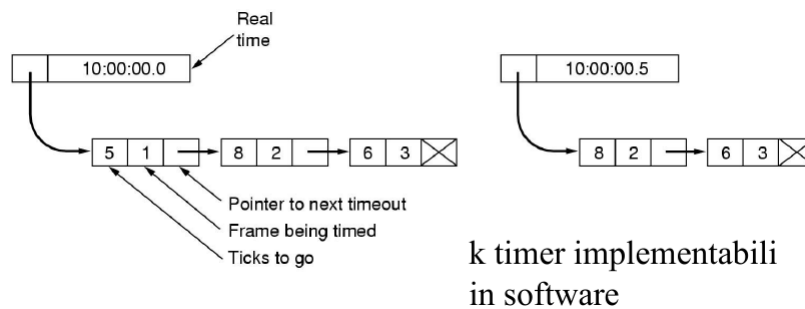
L'idea è quella di risparmiare sulle ritrasmissioni: se la destinazione sta aspettando 5 e nel buffer può mettere fino a 9 (5 elementi), se arriva 6 è fuori ordine ma lo mette nel buffer e nel momento in cui arriva 6 genera nack dicendo che sta aspettando 5, oppure può essere che decidiamo di non usare nack e nel frattempo la destinazione si mette via i dati, fino a che scadendo il time out il sender che ritrasmette 5, quando 5 arriva a destinazione lo passa al livello superiore e poi essendosi messa da parte 6 7 8 li manda al livello applicazione e poi passando ack dice che gli manca 9. Questa cosa può verificare che la sorgente rimandi 678 anche se la destinazione ce li aveva già da parte, l'idea di selective repeat riesce a limitare le ritrasmissioni inutili. In realtà non è proprio così e dipende dai ritardi di trasmissione, ma in ogni caso riesce a limitare lo spreco di banda e ritrasmissioni inutili.

In entrambi i casi il sender deve ricordarsi k messaggi inviati (riempie la sua pipeline di messaggi inviati) e deve aspettare k ack facendo partire k timer. E' una cosa intorno a cui giriamo in modo ingegnoso.

4.7.1 Dettagli implementativi

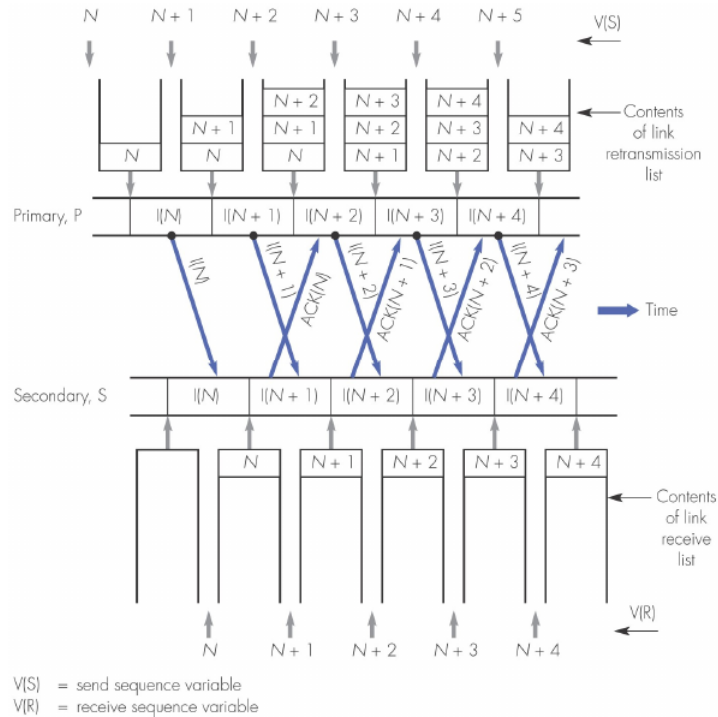
Non vengono realmente mantenuti k timer, il mantenimento di un timer da parte di OS vuol dire che ad ogni tic di clock del sistema l'OS deve andare a guardare tutti i timer che stava mantenendo e decrementare il valore di tutti quanti e per ogni timer il cui valore diventa 0 deve fare partire una trap per dire al processo che il timer è scaduto. L'idea è di mantenere un timer soltanto con questa struttura: realtime è il valore attuale del clock, associato a questo tempo viene mantenuta una lista di cui il primo elemento (5,1) ci dice che rispetto al

tempo reale tra 5 tic di clock questo timer scade, se scade quello che il nostro sender deve fare è ritrasmettere il messaggio con numero di sequenza 1, in questa struttura dati c'è anche un puntatore all'elemento successivo della lista (8,2), la semantica di questo elemento successivo è: il secondo timer scade ad 8 unità di tempo dopo la scadenza del precedente, all'interno c'è un altro puntatore al successivo elemento della lista. Con una struttura di questo genere succede: non arriva un ack, ad ogni tic di clock l'os decrementa solo il 5, quando 5 diventa 0 vuol dire che siamo ormai arrivati al tempo 15, il primo elemento della lista viene eliminato ed il secondo diventa il nuovo primo elemento e andiamo avanti così in cui l'os si mette a lavorare solo sul primo elemento della lista. Supponiamo invece che l'ack arrivi (arriva a tempo 13) quando non abbiamo ancora decrementato completamente il valore iniziale, ci arriva l'ack per il messaggio 1 quindi il primo elemento non lo dobbiamo più usare e modifichiamo il tempo dell'elemento successivo sommando il tempo rimanente a quello del primo elemento che abbiamo eliminato. Per cui k timer vengono implementati in software con una lista. Nel caso in cui dev'essere fatta una ritrasmissione l'ultimo puntatore della lista non avrà puntatore nullo ma punterà all'elemento 1 e rimettiamo in coda l'elemento 1.



4.7.2 Schema generale

Senza errori di trasmissione go back n e selective repeat funzionano allo stesso modo. Non guardiamo sender e receiver ma ricordiamoci che il flusso può essere bidirezionale e full duplex ma per semplicità guardiamo cosa funziona in un senso, il primario manda i dati al secondario.



In alto ci sono i numeri di sequenza che vengono appiccicati dati che arrivano dal livello superiore (applicazione) che vengono numerati e pian piano infilati nella finestra di invio, questo fino a quando la finestra di invio non è piena, il primario avendo dati nella finestra di invio manda il messaggio con numero di sequenza N. Il secondario sta aspettando il messaggio con numero di sequenza N, la pendenza della linea ci ricorda che ci sono dei ritardi non nulli in rete, N arriva al secondario che fa controllo di correttezza, toglie lo header del suo livello 4, mette il payload nel buffer e manda i dati al livello superiore, aumenta il numero di messaggio atteso e fa partire l'ack per un valore N (sto interpretando ack come ack cumulativi, quando il secondario manda ack n vuol dire al primario che ha ricevuto tutto e consegnato tutti i messaggi minore = n). Intanto il primario riempie la pipe, quando ha finito di trasmettere N comincia a trasmettere N+1, mentre manda N+2 lo raggiunge l'ack N del secondario quindi lo processa e cancella dalla finestra di invio tutti i dati minore = N, perchè sa che è arrivato e non dovrà più ritrasmetterlo e così via. Tutti e due stanno lavorando al 100% come l'utilizzo del canale e siamo in un mondo ideale.

4.8 Go Back N

Vediamo ora come funziona quando ci sono dei problemi. Formato dei dati: L'idea è che il formato dei messaggi contiene i dati, non sto più a guardare il tipo perchè so che nello header di livello 4 (trasporto) c'è spazio per contenere tutto, la porzione dati può essere non vuota, il campo numero di sequenza ha valore valido e poi può esserci l'ack, può essere messo in piggy back ed è cumulativo (come dicevamo prima).

Inizializzazione: scambio di dati full duplex, entrambi i processi ricordano che se l'ack che stanno aspettando è 0 allora anche il messaggio da mandare è 0, mentre per il ricevente se il messaggio atteso è 0 il messaggio in window è 0.

Interfaccia tra sender e livello applicazione: prima abbiamo assunto che il livello applicazione abbia sempre dei dati da fornire al livello di trasporto, ma c'è un controllo di congestione, quindi ci si è occupati del controllo di congestione nel senso che può esserci un dialogo tra livello trasporto e applicazione dicendo di non inviare più dati, all'inizializzazione finchè c'è spaziosa nella finestra di invio della sorgente il livello 7 può mandare i dati, altrimenti il livello 4 può disabilitare il livello 7. Questo può anche essere collegato a quello che abbiamo detto parlando di socket, l'OS può bloccare l'applicazione sulla write perchè lo spazio sull'OS è pieno. Se il livello applicazione è abilitato e ha dati da mandare, il livello di trasporto prende i dati e appiccica il numero di sequenza e fa il suo lavoro.

La finestra della sorgente è gestita come un array circolare: C'è dentro il controllo di flusso, nel momento in cui arriva un nuovo dato posso sovrascrivere uno vecchio perchè tanto ho già ricevuto l'ack. Dobbiamo dimensionare questo array circolare, è legato al max seq che non è più 1.

Procedura invio messaggio: Prende i dati dal livello applicazione riempio i campi dello header del nuovo messaggio, mette il numero di sequenza, sempre e comunque manda un ack fin dove è arrivato e passo al livello network. Quando invio il messaggio setto il timer che rileva la perdita del messaggio o dell'ack.

Schema funzionamento ciclo infinito: Il livello trasporto aspetta che succeda qualche cosa e quando succede fa quello che deve. Ci arriva un nuovo dato dal livello applicazione:

- Messaggio danneggiato: dato che non sto considerando nack non faccio niente, se è danneggiato lo butto via e non lo richiedo, si aspetta che scada il timer.
- Messaggio corretto: ci può essere dentro un dato, un vecchio duplicato, qualcosa troppo avanti da buttare via. Guardiamo lo header, se il numero di sequenza associato ai dati è quello del messaggio che volevo passo il payload dei dati al livello applicazione e aggiorno il numero di messaggio atteso, se non è questo allora lo butto via. Se il messaggio ha in piggyback un ack, se l'ack si riferisce ad un numero di sequenza che sta nella mia finestra di invio posso ripulire la mia finestra di invio (tolgo tutto quello che è minore uguale di ack) resetto tutti i timer e sistemo la lista dei timer. Avendo liberato la finestra posso fare entrare nuovi messaggi da inviare.

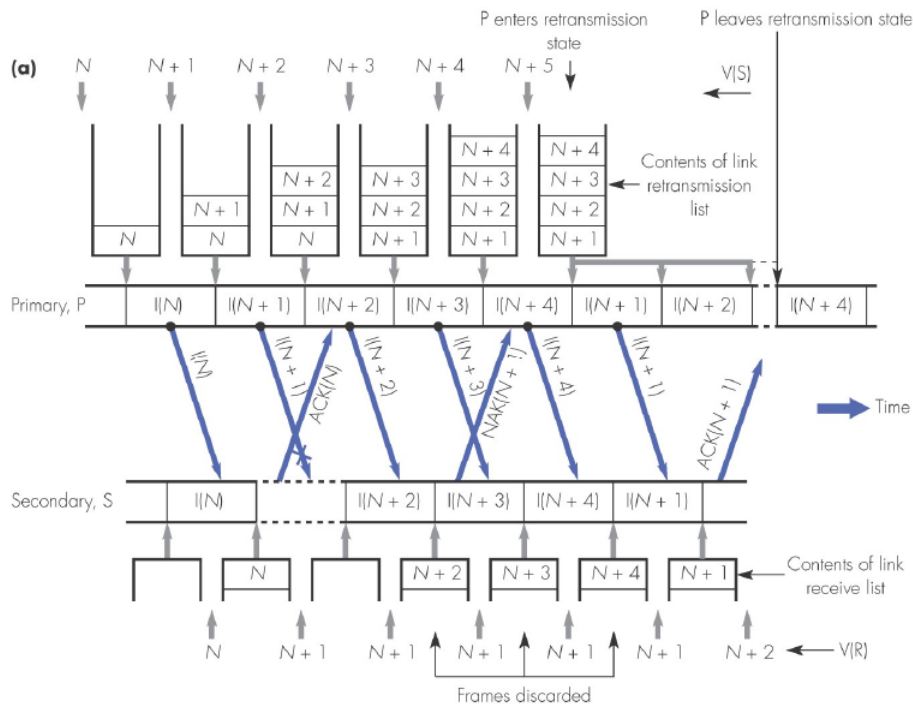
Aggiorno anche il valore di ack atteso che è uguale al numero di sequenza del messaggio più vecchio nella finestra di invio.

- Time Out: se il sender vede scadere un timeout devo ritrasmettere quel messaggio, invio anche tutti i messaggi successivi, quindi se scade il timeout per un messaggio x vuol dire che o non è arrivato oppure che è arrivato e si è perso l'ack, il problema è che per la sorgente sono indistinguibili, ha l'obbligo di lavorare per rispettare i termini di un contratto di scambio dati affidabile e connection oriented (lavora nel worst case). Visto che il protocollo che stiamo eseguendo usa go back n , la sorgente sa che non è arrivato ack per x , visto che la finestra della destinazione 1 sta buttando via tutti quelli inviati dopo x , quindi devo ritrasmettere sia x sia tutti quelli che ho mandato dopo x e rifaccio partire tutti i relativi time out con la gestione che abbiamo visto prima. Notiamo che quando scade il timer il livello superiore si blocca e non ci invia più dati.

Ricordiamo che stiamo considerando messaggi full duplex (che viaggiano in entrambe le direzioni).

Lo ack cumulativo visto che abbiamo detto che stimiamo il valore del round trip time in base al traffico in rete ha il vantaggio che se perdo un ack quello successivo viaggiando con successo nasconde il fallimento di ack precedenti facendo sì che il primario possa capire comunque che i messaggi vecchi sono arrivati senza fare ritrasmissioni.

4.8.1 Messaggio corrotto



Consideriamo sempre l'invio da p ad s. Il primario manda messaggio con numero N , il secondario sta aspettando N , che è quello atteso, passa il dato al livello applicazione e svuota il buffer, il secondario manda l'ack N al primario il quale manda $N+1$, ma ad $N+1$ il messaggio non arriva (o arriva danneggiato), il primario comincia a mandare $N+2$:

- I nack sono abilitati: arriva $N+2$ al secondario, che capisce che non è quello atteso, quindi butta via e aspetta $N+1$, quindi il secondario manda un nack per $n+1$
- I nack non sono abilitati: il primario manda i dati fino a $N+4$ ed a questo punto succede che al primario scade il time out per il messaggio più vecchio che ha nella finestra ($N+1$) e a questo punto deve rimandare $N+1$ e tutti i successivi che ha mandato, perchè il secondario ha ricevuto tutto fino a $N+4$ ma ha scartato tutto perchè non erano i messaggi che si aspettava.

l'ack in piggyback, se non mi arrivano i dati dal livello superiore mando solo l'ack. Questo timer che è circa la metà del round trip time dev'essere piccolo a sufficienza per fare arrivare l'ack prima che dall'altra parte scada il timeout e parta una ritrasmissione. Utilizziamo anche in nack per velocizzare il recupero che sono utili se il timer del sender è sufficientemente grande (quindi i nack mi evitano ritrasmissioni) se è piccolo i nack non servono a molto. La finestra di ricezione non è più 1 ma qualche valore k .

Formato del messaggio: Il segmento (messaggio) contiene i dati ed il numero di sequenza, l'ack messo in piggyback ai dati oppure da solo, oppure un nack da solo MAI in piggyback. L'ack è cumulativo ed il nack è sempre puntuale.

Inizializzazione: E' come per go back n in più c'è la gestione dell'array circolare. Ci ricordiamo qual'è il messaggio più vecchio in attesa di timeout perchè il sender rimanda solo quello quando scade il timer. Nel ruolo di ricevente ci ricordiamo una lista dei messaggi che sono arrivati corretti ma fuori ordine per passarli al livello superiore.

Interfaccia tra livello applicazione e livello trasporto: E' uguale al quella di go back n.

Procedura di invio messaggio: E' come in go back n. L'ack è in piggyback, se è un messaggio dati faccio partire un timer in attesa dell'ack.

Ciclo infinito:

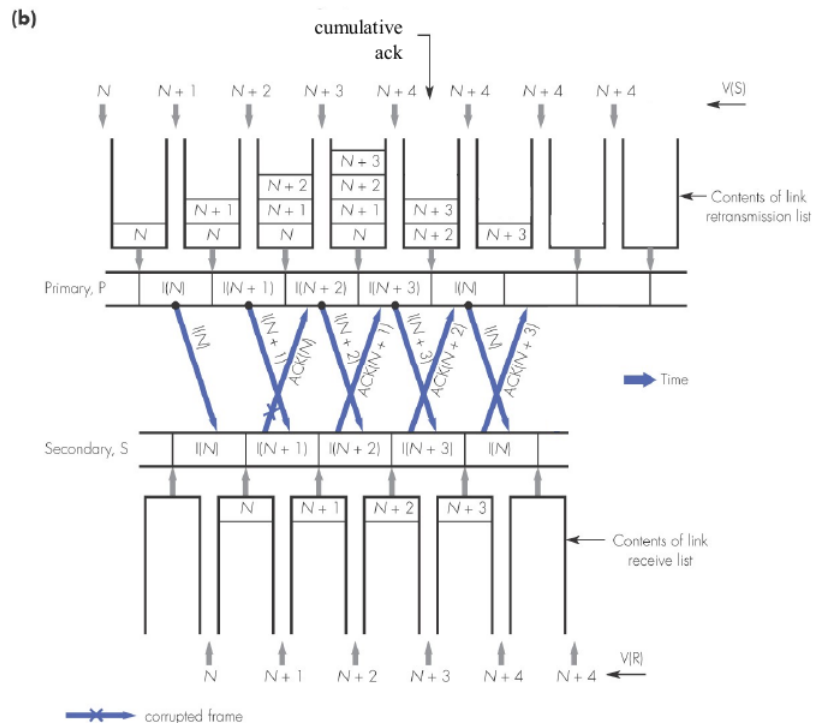
Eventi possibili:

- Nuovo dati da livello applicazione, come go back n
- Arrivo messaggio danneggiato: potrebbe essere quello che mi stavo aspettando, chiamo la procedura di invio messaggio per fare partire un nack con numero di sequenza uguale al messaggio atteso.
- Arrivo messaggio corretto: se è un messaggio dati e non è nella finestra di duplicati me lo tengo nella finestra di ricezione, tutto quello che posso passare al livello applicazione in ordine dalla finestra di ricezione lo mando, se ho passato qualcosa al livello applicazione faccio scorrere la finestra di ricezione ed aggiorno la memoria dei messaggi arrivati ed in attesa di consegna. Se ho consegnato almeno un messaggio non devo mandare alcun nack però mi è arrivato qualcosa che mi ha fatto procedere, quindi devo dire alla sorgente che fino a qui ho ricevuto tutto, faccio partire il timer per mandare l'ack alla sorgente. Se sono dati ma non è il messaggio atteso e non ho un nack già schedulato da inviare, vuol dire che è qualcosa che arriva dopo, posso inviare il nack per quello che stavo aspettando. Se quello che ricevo è un nack per la mia finestra di invio allora glie lo mando. Se il messaggio che ricevo ha in piggyback un ack per un messaggio nella mia finestra di invio (dato che sto usando ack cumulativi) tutto quello che ha valore minore uguale all'ack posso togliere i messaggi dalla finestra di invio, azzerando i timer e facendo scorrere la finestra e successivamente aumento il valore dell'ack atteso.
- Timeout per messaggio in finestra sorgente: non ho ricevuto l'ack e quindi lo ritrasmetto, invio messaggio per il messaggio più vecchio di cui stavo

buffer e insieme a lui consegna al livello applicazione $n+2$, 3 e 4. Aggiorna il suo numero di messaggio atteso ($N+5$) e manda un ack cumulativo $N+4$ al primario che quando arriva puo' eliminare tutto quello che e' gia' stato ricevuto e che ha tenuto nel buffer.

L'idea è che questo algoritmo fa una ritrasmissione chirurgica evitando ritrasmissioni inutili in realta' non e' del tutto vero. Abbiamo detto che il nak e' un meccanismo opzionale, supponiamo che non sia abilitato nella nostra trasmissione ed il diagramma temporale cambia poco, perche' puo' succedere che scatti il timer per il dato perso e ritrasmetterebbe $n+1$. Potrebbe capitare che il primario ritrasmetta $n+1$, quello che puo' succedere per cui selective repeat fallisca e' che alla fine della ritrasmissione di $n+1$ il primario potrebbe vedere un timeout di $n+2$ e ritrasmetterlo, semplicemente perche' ci vuole un po' di tempo perche' $n+1$ arrivi al secondario e che generi l'ack per $n+4$ e che lo stesso arrivi al primario. Quindi puo' essere che ogni tanto sfora. Il gioco è tutto su quanto fare più grande il timer rispetto al round trip time.

Ack corrotto:



Siamo nello stesso caso del precedente, al primario non importa se l'ack e' corrotto perche' continua a mandare fintato che gli arriva un ack o riempie il buffer e poi quando gli arriva l'ack per il messaggio successivo quell'ack fa eliminare dalla finestra del sender anche tutti i messaggi precedenti. Sempre con ack cumulativi, con quelli puntuali sarebbe piu' complicato, non solo dal

punto di vista dell'affidabilità anche dal punto di vista della costruzione del codice, perché chi è sorgente di dati deve andare a spunta per capire cos'è arrivato e cosa no.

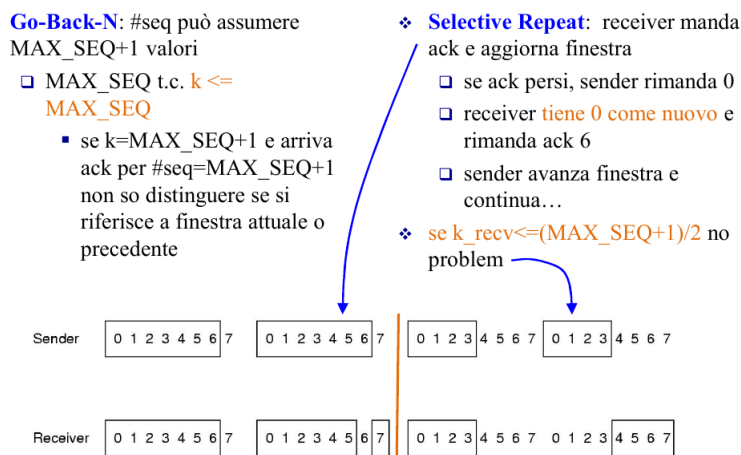
4.10 Range numeri sequenza

Avevamo capito che con idleRQ avevamo un numero di sequenza che poteva essere rappresentato con 1 bit ed il buffer aveva dimensione 1. Qui le cose sono un po' diverse. Ci sono dei limiti al massimo numero di sequenza che non sono più 0 o 1 e che possiamo ricavare e motivare per assurdo. In go back n la finestra di invio dev'essere di dimensione al più uguale del massimo numero di sequenza utilizzato, perché dobbiamo mettere tutti i numeri di sequenza nella pipe, quanti sono i messaggi e quant'è lunga la pipe possiamo stimarlo. Chi fa gli standard ragiona sulla rete terrestre, reti wired, la velocità delle reti che ci sono, tant'è vero che questi protocolli non sono scolpiti nella pietra, i protocolli che usiamo quotidianamente, come TCP (quello che sta sotto HTTP), viene continuamente mantenuto. Si fanno due conti sulla base delle reti che ci sono (quali sono, quanto veloce vanno, qual'è il traffico di rete) e capiscono quanto dev'essere lunga la pipeline. Come abbiamo già visto il protocollo ci dice quanto devono essere lunghi i messaggi, definisce com'è fatto lo header e quanti bit utilizzare per rappresentare un'informazione, da questo numero di bit viene fuori il massimo numero di sequenza, perché se lo standard dice che dobbiamo usare 4 bit sappiamo che il massimo numero rappresentabile è 15, poi dobbiamo riavvolgere. Il lavoro quindi è al rovescio, sapendo il massimo numero di bit per rappresentare il massimo numero di sequenza ad esempio 15, se questo è il massimo valore del numero di sequenza in base a questo abbiamo delle limitazioni sulla finestra di invio in modo che non ci sia ambiguità quando devo rimandare un nuovo messaggio con lo stesso numero di sequenza di un precedente (ad esempio un nuovo messaggio con numero di sequenza 0). Quello che ci viene detto è che, ad esempio se uso 4 bit con i numeri di sequenza da 0 a 15, max seq va da 0 a 15 ma la finestra di invio in caso di go back n non può essere grande 16, dev'essere al più 15. Dov'è il problema (3 bit per numero di seq, quindi da 0 a 7): supponiamo che il sender si tenga un buffer di invio grande 8 (per assurdo, può mandare quindi messaggi da 0 a 7) per tenere tutti i numeri di seq distinti. Riempie la pipe manda tutti i messaggi da 0 a 7 e poi aspetta l'ack per il messaggio più vecchio (lo 0), il ricevente riceve tutti questi messaggi, ogni volta che riceve il messaggio manda l'ack per quello ricevuto, ma tutti gli ack vanno persi (worst case), il ricevente si aspetta un nuovo messaggio con numero di seq 0 (perché più di 7 con 3 bit non possiamo rappresentare quindi ricomincia), ma il sender non avendo ricevuto nessun ack quando scade il time out rimanda il vecchio 0 ed il destinatario è contento perché si aspetta il numero di seq 0 e manda l'ack al sorgente, il sorgente rimanda anche tutti gli altri messaggi dall'1 al 7. E così succede il casino, perché duplichiamo l'informazione. Stiamo violando il contratto di consegna ordinata.

Supponiamo ora che la finestra di invio è grande non più di 8 (il massimo

valore che può assumere il numero di sequenza): il sender si prende una seq grande 7 che quindi può contenere i messaggi da 0 a 6, il ricevente aspetta e come prima riceve tutto e restituisce ack per tutto e si mette ad aspettare il messaggio con num seq 7, come prima tutti gli ack sono persi, quindi scadendo il timer per il primo messaggio il sender si presta a re-inviare 0, che torna al ricevente ma che non ha nella finestra di ricezione (perchè in questo momento è lì bloccato in attesa di 7) quindi manda un ack dicendo che e' arrivato fino a 6 e sto aspettando 7. Questo ack quando arriva al sender gli fa capire che può far scivolare la finestra di invio, che andrà da 7 a 5 ed il receiver continuerà ad aspettarsi esclusivamente il prossimo.

Con go back n posso riempire la pipe con maxseq messaggi ma lo stesso non vale per selective repeat, in questo algoritmo siamo piu' stringenti, supponiamo di usare lo stesso limite che usiamo per gobackn: 3 bit per rappresentare i numeri di seq (0-7) e usiamo una finestra di invio grande 7. Questa volta anche il ricevente e' piu' grande di 1 (è ragionevole dire che la finestra del ricevente è grande come quella del sender, quindi 7), all'inizio della trasmissione il sender si permette di mandare i primi 7 messaggi e li' si ferma ad aspettare, il ricevente riceve tutti e manda indietro ack per tutti, fa scivolare in avanti la sua finestra di ricezione e si mette ad aspettare, ma ora la finestra non si aspetta solo 7 ma la finestra scala quindi ora si aspetta 7012345. Gli ack come prima vengono tutti persi quindi il sender vede il timeout per 0 comincia a rimandare il vecchio 0, che viene tenuto dal ricevente perche' e' sulla sua finestra, e nonostante non sia il successivo lo mette nel suo buffer aspettando che arrivi il messaggio 7 che sta aspettando. Un'altra volta stiamo violando il contratto di consegna ordinata. Quindi una finestra che soddisfa la disuguaglianza per go back n, per selective repeat non va bene, dobbiamo restringere parecchio: se usiamo 3 bit per rappresentare il numero di seq (e rappresentare i numeri di sequenza da 0 a 7), la finestra di invio e ricezione non devono essere piu' grandi di $(maxseq+1)/2 = \frac{7+1}{2} = 4$, con questa configurazione evitiamo di confonderci su qual'è il nuovo od il vecchio messaggio.



4.11 Utilizzo del canale

Avevamo capito cosa succede nel caso di idleRQ e come calcoliamo l'utilizzo a numeratore il tempo per cui il sender ha lavorato diviso il tempo che intercorre tra l'invio del primo bit del messaggio ed il ritorno dell'ack per quel messaggio, ed avevamo cercato di tirare l'utilizzo al 100% facendo mandare un'altro messaggio al sender e non lasciarlo lì a fare niente. Se la finestra di invio è grande K il sender trasmette k messaggi prima di aspettare ricevere l'ack vuol dire che il lavoro fatto dal sender è k volte il tempo di trasmissione di un messaggio, quindi l'utilizzo di continuous RQ è k volte l'utilizzo di idle RQ se k è la dimensione della finestra di invio: $U = k \cdot T_x / (T_x + 2T_p)$.

Possiamo lavorare su questa formula cercando di massimizzare l'utilizzo, avevamo già osservato che il bitrate è una caratteristica dell'hardware del sistema (scheda di rete del sender) quindi non ci si può fare niente, così come il round trip time che è una caratteristica fisica della rete. Invece con la massima dimensione del messaggio possiamo giocare ed anche sulla massima dimensione del messaggio, il k (legato al valore di maxseq) ed abbiamo capito che dipende dallo standard che ci dice quanti bit utilizziamo per rappresentare il numero di seq. La dimensione della finestra varia dinamicamente durante lo scambio dati. La formula dell'utilizzo è ideale, considera il caso migliore, se nulla va storto il massimo utilizzo che riesco ad ottenere è quello, se tutto va bene è indifferente parlare di continuousRQ nelle due modalità, perché i due si differenziano solo quando qualcosa va storto: $k \cdot T_x \rightarrow T_x + 2T_p$.

4.12 Controllo di flusso

Go back n non ha bisogno di controllo di flusso perché butta tutto quello che non gli serve, non ha memoria il ricevente, non ha difficoltà di processing. Per Selective Repeat oltre alle considerazioni fatte prima, c'è un limite massimo anche sulla finestra di ricezione, e non è detto che sia soddisfatto e potrebbe generare problema di flusso, perché il ricevente non è detto che riesca ad avere la massima finestra di ricezione in accordo all'upper bound e quindi il sender deve adeguarsi alla finestra di ricezione del receiver.

Chapter 5

TCP

Fino ad ora abbiamo visto gli algoritmi incastonati nel livello 4. Ora possiamo cominciare a vedere come TCP implementa questi algoritmi.

5.1 TCP, Transport Control Protocol

Protocollo di trasporto full duplex (dati in entrambe le direzioni) orientato alla connessione ed affidabile (garantisce che i dati vengano consegnati tutti, una ed una sola, volta in ordine), e' un protocollo byte-oriented, fino ad ora ci ha un po' ingannati perche' i numeri di sequenza visti nei diagrammi sembrano essere progressivi, ma in realta' TCP controlla l'arrivo di ogni singolo byte e lo enumera, vuol dire che se il primo messaggio ha num seq 0 e contiene 100 byte il secondo messaggio ha numero di seq 100 perche' il primo byte del secondo messaggio è il 100 pezzo nella ricomposizione, il primo messaggio contiene da 0 a 99, il secondo parte dal 100. Perche' l'hanno fatto cosi' attento al singolo byte, quando abbiamo parlato di socket abbiamo detto che la rete e questi protocolli visti nel corso sono nati in ambito di ricerca e nate queste cose in ambiente accademico il OS utilizzato era UNIX ed i comandi sono lunghi, le prima applicazioni erano telnet (collegamento su una macchina remota) e vuol dire che potevo lavorare su una macchina remota e mandare dei comandi alla macchina, era importante in questo mondo che i comandi dati alla macchina remota arrivassero nell'ordine in cui li ho scritti, quindi hanno progettato TCP orientato al byte, si considera il trasferimento come uno stream di byte. I dati sono trasferiti in segmenti, stiamo istanziando gli algoritmi nel protocollo utilizzato in internet e la protocol data unit di TCP si chiama segmento, perche' sono pezzi di uno stream di dati scambiati tra sorgente e destinazione. Quanto sono lunghi i segmenti, c'e' una dimensione massima ed anche in questo caso dobbiamo mettere insieme cose che abbiamo detto, abbiamo visto che a livello 3 (il livello piu' alto dove si vede la rete) per facilitare la gestione della memoria nei router si possono frammentare i dati in modo che se dalle applicazioni arrivano delle grosse quantita' di dati li facciamo viaggiare nei router in blocchi non molto grandi (memoria paginata vs segmentata). A livello 3 la protocol data unit di liv 4 potrebbe essere frammentata e questo e' un disastro, noi abbiamo il nostro segmento di livello 4 a cui viene preposto lo header (poi c'e' il payload) tutto deve diventare payload del protocol data unit di livello 3 ma se 'e grande potrebbe essere rotto in frammenti, lo header di liv 4 contiene anche il numero di sequenza da attaccare ai dati, se frammentiamo a liv 3 ci troviamo con lo header del 3 + header 4 + dati ed un altro pezzo con header 3 + solo i dati. Dobbiamo quindi stare attenti che il secondo pezzo in cui non è presente lo header di livello 4 è collegato al primo, inoltre abbiamo un problema perche' se un frammento si perde a livello 3 non importa niente ma a livello 4 arriva il payload di un pacchetto che pero' e' monco. L'idea del TCP e' che e' un casino frammentare, quindi non facciamolo, ma io TCP fornisco servizio connection oriented ed affidabile, quindi ci vogliono 3 fasi (costruzione connessione, trasferimento dati, chiusura connessione) e' stato paragonato nella sua spiegazione al servizio telefonico, nella fase di costruzione della connessione ci diciamo se uti-

lizziamo dei segmenti di una taglia massima (MSS maximum segment size) tale che siamo sicuri che a livello sotto la frammentazione non c'è, lo standard dice che sorgente e destinazione in fase di apertura connessione negoziano e la sorgente propone una dimensione del segmento (più è grande più in percentuale è piccola la dimensione dello header) che cercherà di ammortizzare il costo dello header (lo header deve essere piccolo rispetto al payload), se nella fase di apertura connessione ci sono segnali dalla rete che questa taglia di segmento è troppo grande nello standard TCP ci si allinea con maximum segment size di 536 byte, che è poco, ma per standard Internet ogni cosa che parla con i servizi standard (TCP, UDP, IP) deve supportare una taglia di segmento non inferiore a 536 byte, altrimenti stai fuori da Internet. Stante così le cose da applicazione mi arrivano dei dati e quello che TCP fa è: qualunque cosa che mi chiede di mandare lo taglio in pezzi non più grandi di maximum segment size così sono sicuro che sotto non ci saranno segmentazioni e non ho problemi. Se è tanta roba lo taglio in pezzi senza stare a guardare ma se quello che mi arriva sono dati piccoli per norma TCP cerca di aggregarli per mandare un segmento di taglia massima così gli costa meno attaccarci davanti 20 byte di header, ma questo potrebbe mandare in confusione il ricevente su cosa gli arriva, l'idea è che di default non sono mantenuti i confini del messaggio dell'applicazione.

Ci sono due situazioni in cui non si obbliga TCP a creare un segmento di taglia massima:

- PUSH: l'applicazione passa delle informazioni al TCP chiedendo che vengano spinte dall'altra parte anche se il contenuto del segmento è più piccolo della MSS. Ad esempio succedeva quando usavamo Telnet, le socket (che ci danno accesso a TCP) nate nel mondo universitario accademico dove ancora oggi si predilige software open source (UNIX). L'idea del PUSH viene perché i comandi di UNIX sono molto corti, ma è importante per chi sta usando un'applicazione remota che i comandi arrivino nell'ordine giusto. In applicazioni di questo genere il push serve per far funzionare l'applicazione, non possiamo assumere che l'utente che sta usando l'applicazione possa mandar più comandi senza guardare l'output di questi comandi, non possiamo far aspettare TCP che raggiunga il MSS anche se noi dobbiamo mandare pochi byte di messaggio, dobbiamo avere il modo di dire al TCP di mandare i comandi che abbiamo scritto mentre noi aspettiamo per scegliere il prossimo senza aspettare che il maximum segment sia pieno. Manda ora i dati, non aspettare di costruire il MSS.
- URGENT: un push può contenere qualunque dimensione di dati, l'applicazione sa quando dire di mandare via la cosa in push. Ci può essere al più un carattere urgent per segmento e se TCP sta cercando di costruire un segmento di dimensione massima quando arriva il carattere urgent viene inserito nel segmento in costruzione, si segnala all'interno dello header di TCP dov'è il carattere urgent e quel segmento viene mandato via così com'è e viene segnalato all'utente che è arrivato un carattere urgent. È stata pensata per toglierci dai guai, è come il ctrl+c in unix, magari c'è un bug nel software e noi vogliamo fare terminare un processo.

TCP cerca di non intralciarsi da solo, quindi include controllo di congestione, verticale, TCP si accerta che la rete non è una situazione di congestione, altrimenti si autolimita, se ha percezione che i router che stanno tra la sorgente e destinazione sono congestionati sul traffico altrui la sorgente non può fare nulla ma cerca di allentare la congestione rallentando buttando nella rete meno pacchetti, e controllo di flusso (orizzontale, non mandare dati troppo velocemente ad entità di pari livello). Tutti e due questi processi sono dinamici, la velocità con cui il processo destinatario elabora i dati dipende dal carico della CPU che elabora i dati, quindi varia dinamicamente, quindi la sorgente deve variare dinamicamente il rate con cui la sorgente deve mandare i dati in rete, così come cambia dinamicamente il traffico in rete.

Quando abbiamo parlato di socket avevamo detto che un'entità TCP (sorgente e destinatario) deve mantenersi un po' di informazioni, come minimo una quintupla che è l'associazione che descrive la connessione, protocollo utilizzato, indirizzo IP e porta dell'indirizzo locale, indirizzo IP e numero di porta del processo remoto con cui stiamo parlando. Chi sono i dialoganti e che protocollo stanno utilizzando. Oltre i buffer in cui TCP mette i dati che arrivano dal livello superiore per spedirli dall'altra parte e mette anche i dati che arrivano dal livello network per la sua entità locale, c'è anche una porzione di info che riguardano la connessione e si ricordano l'identificatore della connessione (catena di puntatori che ci fa arrivare al processo), si ricorda il numero di sequenza iniziale, il primo numero di sequenza non è zero è troppo facile se tutti i dati trasmessi iniziano per 0, perché è troppo rischioso. Per cercare di contrastare attacchi di replication (replico una connessione fatta da altri) quando x parla con y, x i suoi dati non comincia a numerare i dati da 0, ma è pseudocasuale ed è ricavato dai bit meno significativi del suo clock di sistema, allo stesso modo y non enumera di dati da zero ma fa la stessa cosa di x. Questo legame viene mantenuto per tutta la vita di TCP, ci si rende conto quindi anche quanto tempo passa tra l'invio di ogni dato.

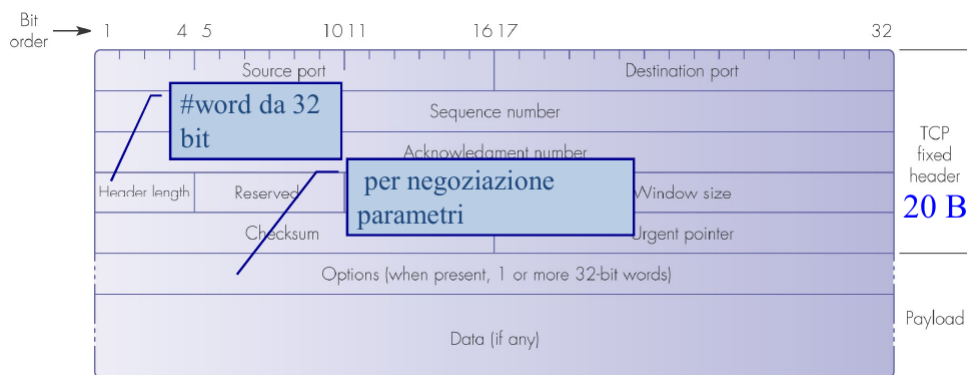
Nel connection record c'è:

- ID connessione
- il numero di sequenza iniziale
- la dimensione della finestra: dimensione della finestra di ricezione (1 MSS nel caso di approccio go back n come nel caso di stop and wait e più grande di 1 nel caso di selective repeat). Non necessariamente la finestra è di questa dimensione per il problema dell'upper bound in selective repeat, è uno spazio di memoria dedicato dal sistema operativo per gestire le connessioni TCP, se, sul ricevente, ci sono tante connessioni che devono dividersi la disponibilità di spazio, ognuna di loro avrà buffer di ricezioni più piccoli. Sono quindi parametri che cambiano dinamicamente, il ricevente ricorda in funzione di quello che ha ricevuto la dimensione massima della finestra ancora disponibile rispetto al massimo possibile e la rimanda indietro dicendo che non possiamo mandargli più roba di così

altrimenti vado in buffer overflow. Quindi se vogliamo fare controllo di flusso non mandarmi più dati di questi

- lo stato della connessione, se ci stiamo connettendo, se stiamo chiudendo, se la connessione è pronta per lo scambio di dati
- MSS (maximum segment size)

5.2 Formato del segmento



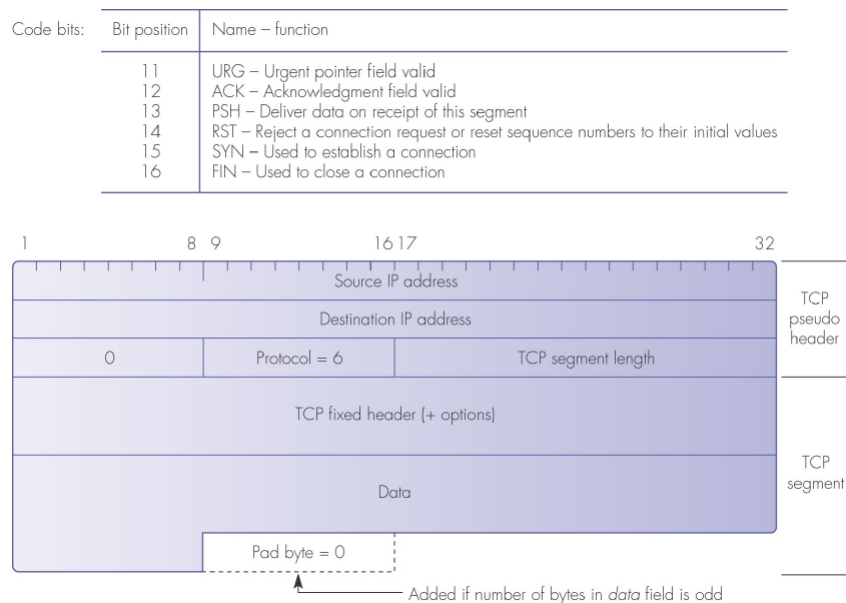
TCP è di livello 4, quindi si occupa degli indirizzi di livello 4 (numeri di porta) che sono unsigned short,

- quindi i primi 32 numeri di porta sono 16 bit per la nostra porta + 16 bit per la porta del destinatario, non ci sono gli indirizzi di rete perché siamo in un livello 4.
- Numero di sequenza su 32 bit, quindi il problema di riavvolgimento è solo per file veramente grossi.
- Il numero di ack, TCP usa solo ack cumulativi, deve rappresentare il numero di sequenza dell'ultimo byte ricevuto. Quindi il numero di ack occupa tanti bit quanto il numero di sequenza.
- Lunghezza dello header (descrivono il numero di bit in numero di parole, 4 ottetti), perché è vero che è lungo 20 byte di TCP, ma c'è la possibilità che vengano aggiunte delle opzioni, che non sappiamo quanto spazio occupano, ad esempio che algoritmo usare (selective repeat o go back n) oppure dire qual'è la MSS proposta. Quindi questo campo dice alla destinazione che deve fare parsing per qualcosa di più di 20 byte.
- bit reserved: sono 8 bit che al momento non sono utilizzati, ma sono lì per usi futuri, supponiamo che con l'evolvere della rete potremmo aver bisogno di altre opzioni per fare funzionare TCP e dobbiamo aumentare la

dimensione dello header ed avremo bisogno di più bit per rappresentarlo, così siamo già pronti per un'evoluzione di TCP.

- **Code bits:** sono dei flag che ci danno delle informazioni sullo stato della connessione TCP.
- **Window Size:** dimensione della finestra di ricezione, ogni volta che mando un segmento ricordo a chi mi deve mandare i dati quanto spazio libero c'è nella mia finestra di ricezione. Piggyback, così com'è piggyback il numero di ACK.
- **Cehcksum:** vengono aggiunti dei bit di ridondanza (bit di parità) per permettere a TCP di fare rilevamento di errori (non correzione), sono 16 bit che controllano se il segmento è ok, dobbiamo ricordare gli if danneggiato dello pseudocodice, allora lo butto via, se il cehcksum mi torna allora con alta probabilita' quello che ho ricevuto e' corretto, altrimenti butto via il segmento.
- **Urgent Pointer:** se c'è un carattere urgent nel file diciamo dov'è
- **Opzioni:**
- **Dati:** finite le opzioni partono i dati, è quindi informazioni di livello superiore al 4

5.2.1 TCP - code bit e pseudo header



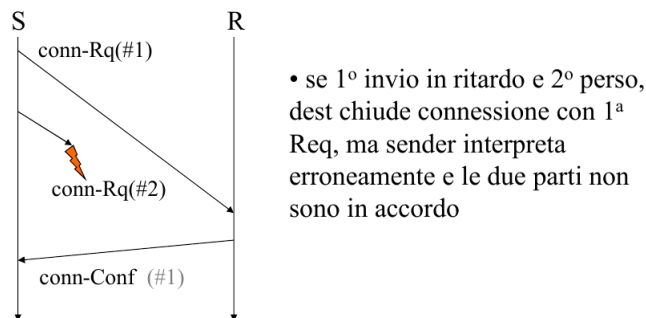
Il checksum è calcolato sullo pseudo-header, il protocollo del livello rete è best effort (non abbiamo la garanzia che i dati siano corretti), quindi il checksum non controlla solo il contenuto del segmento ma controlla anche delle informazioni che sono solo nel livello rete. TCP nel suo lavoro di garantire affidabilità controlla anche le informazioni di livello 3, ad esempio che non siano stati alterati gli indirizzi di rete locale e remoto. Quindi il checksum lavora su header TCP + info parte dati + info header livello network + info che non stanno da nessuna parte ma che si ricava.

C'è un ottetto di bit 0, che possiamo usarlo per allargare la dimensione di un altro campo. Poi c'è un ottetto che è il numero di protocollo, serve per fare demultiplexing (a che processo passo i dati?) ed è un campo dello header di livello network. C'è una lunghezza totale del segmento TCP (non c'è scritto nello header TCP), così TCP controlla che il segmento tirato fuori dal livello inferiore sia tutto. I checksum sono calcolati su header TCP + tutti i dati + byte di imbottitura (tutti 0) che è aggiunto allo pseudo header se il numero di byte è dispari.

I code bit sono 6 e dicono se nella parte di payload c'è un valore urgente e dicono in che posizione è. Nei code bit possiamo anche dire che c'è un ack in piggyback ed in che posizione è. Il flag PSH sta per push (fai partire il segmento senza aspettare di aver raggiunto il segmento di taglia massima). RST è un reset, ci siamo accorti che qualche cosa non va, per cui mandiamo dall'altra parte una segnalazione di errore dicendo che non riusciamo a far funzionar ela connessione e resettiamola. SIN e FIN (sincronize e finalize): sincronizziamoci su come costruiamo la connessione prima di negoziare i dati e diciamoci che chiudiamo la connessione. Finalize controlla che tutti i dati mandati dall'altra parte siano stati ricevuti nell'ordine e quindi possiamo chiudere la connessione soddisfatti.

5.3 Apertura connessione

L'apertura di connessione in TCP è volutamente complicata perchè si devono dire un po' di cose sulle regole per parlarsi che devono essere comune intesa tra le due parti. Dobbiamo complicare un po' la cosa per non fare verificare ambiguità, tipo: la sorgente manda una richiesta di connessione proponendo dei parametri per parlarsi (ack, go back n o selective repete...), non sappiamo niente sullo stato di connessione della rete, può essere che il nostro primario veda un timeout e rimanda la richiesta di connessione e magari cambia un po' le opzioni. Può succedere che la seconda non vada a buon fine e la prima arrivi al corrispondente remoto che mi manda indietro una conferma di apertura, ma quali parametri usiamo? quelli della prima richiesta o quelli della seconda? è una situazione di ambiguità. Questo ci porta a realizzare una apertura di connessione realizzata tramite un algoritmo.



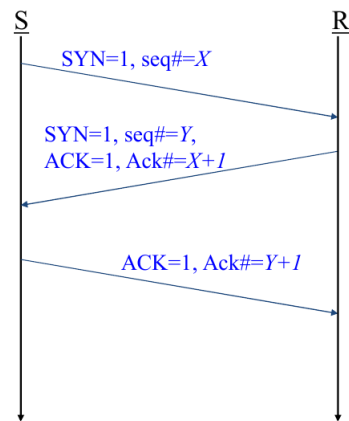
5.3.1 Three-way handshake

Dove handshake sta per stretta di mano, le due parti comunicanti stabiliscono un contratto con un dialogo a tre vie (perchè 3 i messaggi scambiati). S e R può non avere tanto senso perchè i dati possono viaggiare full duplex, diciamo primario S e secondare R. S manda ad R un segmento con il bit di sincronizzazione alzato, segnala che il primo segmento di una conversazione, inoltre manda anche il suo numero di sequenza iniziale, che abbiamo detto essere un numero calcolato partendo dai bit meno significativi dell'orologio di S. R riceve questo segmento dove non c'è nulla di dati (solo header + opzioni TCP), rimanda indietro il suo segmento con il sync bit alzato, il numero di sequenza iniziale y (anche lui calcolato dai bit meno significativi dell'orologio di R), in più in questo segmento alza l'ack bit a 1 (è valido il valore del campo ack number) e questo campo ha valore x+1, ha ricevuto il segmento iniziale con numero di sequenza x e sta aspettando il messaggio con valore x+1. Il terzo messaggio è mandato da S, ha alzato il valore dell'ack e manda un ack per Y+1 dicendo che anche lui ha ricevuto il messaggio e si aspetta il prossimo. (SYN, SYNACK, ACK). Nel terzo segmento S può iniziare a mettere del payload, ci può anche essere un sequence number maggiore di x.

C'è un meccanismo per tenere legati i valori degli orologi con il numero di sequenza appiccicati ai dati, perchè il valore del numero di sequenza deve essere incrementato ogni 4/8 micro secondi.

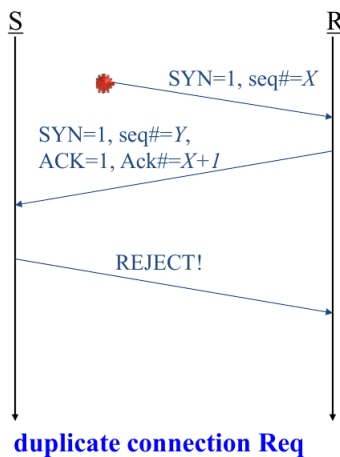
Proiettandolo sulle socket è quello che succede quando c'è la connect del client, fa dire alla applicazione al suo TCP sottostante di far partire il SYN. E la accept del server accetta il segmento SYN ed inizia ad eseguire con quel client il three way and shake.

Questa procedura funziona sempre correttamente anche in presenza di delay and duplicate, è un'aspetto legato alla sicurezza nel caso di replication, mi arriva una richiesta di connessione apparentemente da parte di una persona che aveva fatto poco fa la stessa richiesta, è una duplicazione, non va così la connessione.

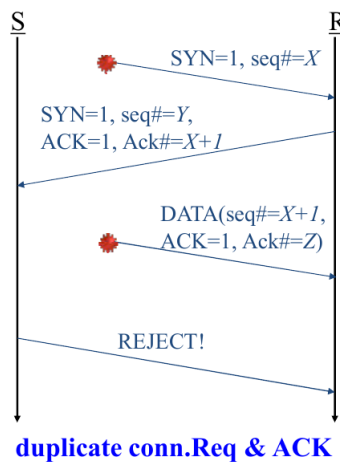


Errori

Esempio 1: Sbuca dalla rete una richiesta di connessione che apparentemente arriva da S per R, quindi R vede un segmento con SYN bit alzato con un numero di sequenza x che è proposto apparentemente da S, R risponde dicendo che accetta la richiesta di connessione. S vede questa risposta, la vecchia richiesta di connessione non l'aveva mandata lui (o era vecchia) e la respinge, perchè l'ack $x+1$ non centra niente con il valore di clock, quindi respinge la connessione.



Esempio 2: Si possono replicare anche più messaggi, prima arriva ad R x e poi $x+1$, però gli ack hanno sempre in piggyback un ack, quindi apparte il segmento di SYN tutti gli altri segmenti hanno l'ack bit alzato ed un valore valido per ack number. Quindi in questo caso ad R non torna che il valor di ack sia Z (non centra niente con il suo clock), quindi non risponde niente ad S, quindi troncano la conversazione.



5.4 Es. sessione TCP (tcpdump)

C'è un esempio di connessione realistico, tcpdump è uno strumento per fare sniffing di pacchetti.

```
rtsg.1023 > csam.login: S 768512:768512(0) win 4096 <mss 1024>
  □proc. su port# 1023 invia richiesta apertura connessione e propone
    parametri receiver window e mss. No dati trasferiti

csam.login > rtsg.1023: S 947648:947648(0) ack 768513 win 4096 <mss 1024>
  □ricevente replica con ack e propri parametri

rtsg.1023 > csam.login: . ack 1 win 4096
  □ack a ricevente: tcpdump lo stampa come differenza da precedente

rtsg.1023 > csam.login: P 1:2(1) ack 1 win 4096      /* send 1B dati */
csam.login > rtsg.1023: . ack 2 win 4096             /* acknowledged */
rtsg.1023 > csam.login: P 2:21(19) ack 1 win 4096    /* send 19B dati */
csam.login > rtsg.1023: P 1:2(1) ack 21 win 4077     /* acknowledged */
  □dati sono ancora in buffer; inoltre, 1B dati da csam a rtsg

csam.login > rtsg.1023: P 2:3(1) ack 21 win 4077 urg 1
csam.login > rtsg.1023: P 3:4(1) ack 21 win 4077 urg 1
```

5.5 Trasferimento dati

Windows size advertisement lo abbiamo già detto, nel formato dello header c'è sempre questo campo che è valido, serve per il controllo di flusso. Quando abbiamo parlato di idle RQ, go back n e selective repete, abbiamo visto il caso in cui vogliamo riempire la pipeline tra sorgente e destinazione (quindi massimizzare l'uso della rete), non sempre è possibile, ed un caso è quello delle applicazioni interattive, può capitare di svolgere delle operazioni dal proprio lato e non vedere del feedback e poi ad un certo punto succede tutto di botto, in queste applicazioni cerchiamo di interagire mandando dei messaggi (caratteri, movimenti del mouse), quello che mandiamo non vengono immediatamente replicati sullo schermo, ma sono dati che vengono messi in segmenti TCP, viaggiano dall'altra parte e quando noi riceviamo l'ack per questi dati la nostra applicazione ci replica le nostre azioni sul nostro schermo. Questo effetto di rallentamento si ha quando TCP decide di costruire sempre dei segmenti di MSS, quindi fino a che non abbiamo disegnato sufficientemente a lungo per riempire un segmento di taglia massima non parte, quando abbiamo riempito un segmento di taglia massima il segmento parte e noi vediamo quello che abbiamo fatto sullo schermo. Quindi le applicazioni interattive che hanno bisogno di un feedback funzionano male se TCP vuole costruire segmenti di taglia massima, dobbiamo impedirglielo, possiamo mandare pochi dati ogni volta alzando il push bit, ma può essere inefficiente. Oppure abbiamo una possibilità intermedia, l'algoritmo di NAGLE che è opzionale e chiuso dentro TCP, diciamo a TCP usa questa cosa, funziona in questo modo: la sorgente comincia a mandare pochi byte di

dati, fino a quando non riceviamo un ack immagazziniamo i dati che arrivano dall'applicazione, nel momento in cui arriva l'ack diamo all'utente che quello fatto prima è stato ricevuto e tutti i dati messi da parte li inviamo in un segmento nuovo, questo di fatto vuol dire fare funzionare TCP come idle RQ, perchè per un periodo di tempo aspetta.

5.6 Errore di scambio dati

Rispetto agli algoritmi visti dobbiamo entrare di più nell'implementazione e specificare cose, in particolare quant'è il retransmission timeout. Sappiamo che TCP fa controllo di flusso con la variabile window size nello header, ma non abbiamo detto nulla riguardo il controllo di congestione.

5.6.1 Timer ritrasmissione

Come facciamo a stimare il retransmission timeout. L'idea è di stimare il round trip time e stimare il timeout un po' più grande di questo tempo. Prima dobbiamo stimare il round trip time medio, stiamo ragionando su un protocollo connection oriented ed affidabile che è stato pensato per trasmettere una sequenza di segmenti, quindi fare la media algebrica potrebbe essere un problema perchè i campioni sono tanti e dipendono dalle dimensioni del file. Si preferisce usare una funzione di media smorzata dove la freccia ci segna qual'è il nuovo valore di una variabile, gli assegniamo il vecchio valore del round trip time moltiplicato per un parametro alfa e sommiamo 1 - alfa moltiplicato per M, dove M è una misura fatta sul tempo di arrivo dell'ack dell'ultimo segmento inviato.

$$RTT \leftarrow \alpha \cdot RTT + (1 - \alpha)M$$

Il valore di alfa è circa 0,9 oppure $\frac{7}{8}$, determinato per via empirica. Dobbiamo notare che diamo un peso molto alto alla vecchia storia ed un peso molto basso alla misura, perchè la misura riguarda un singolo segmento, potremmo essere stati fortunati o particolarmente sfortunati, noi non vogliamo cambiare la stima di RTT per un campione particolare. La misura M si fa solo con la prima trasmissione di un segmento, perchè ho un problema, già visto, il primario manda i dati al secondario entro il time out il secondario non vede arrivare l'ack e in questo momento vedere arrivare l'ack, in questa situazione il primario non sa misurare il tempo perchè l'ack potrebbe essere arrivato alla prima o alla seconda trasmissione del segmento. Quindi se abbiamo fatto delle ritrasmissioni ed alla fine l'ack ci arriva non aggiorniamo il RTT ma lasciamo il valore vecchio, ma dopo aver fatto la ritrasmissione usiamo un valore di timeout che è il doppio di quello precedente per non continuare ad incorrere nello stesso errore. Nel momento in cui ho superato il problema di ritrasmissione e inviando un nuovo segmento ricevo subito l'ack faccio nuovamente la stima del RTT.

Come facciamo a calcolare il retransmission timeout? sappiamo che è un pochino più grande del RTT. Calcoliamo una deviazione standard D sempre con una media pesata, assegniamo quindi alla nuova deviazione standard D la

vecchia D moltiplicata per un parametro α più 1 meno α per la misura in valore assoluto tra il RTT meno la Misura M che avevo fatto:

$$D \leftarrow \alpha \cdot D + (1 - \alpha)|RTT - M|$$

Alcune implementazioni usano l'alfa di prima ed altre implementazioni usano una beta che è uguale a $\frac{3}{4}$, il problema che questo non fa parte del protocollo di comunicazione, è una caratteristica dell'algoritmo, di nuovo il peso più grande viene dato alla storia passata (la vecchia D) ed il peso minore sta nell'apporto dato al calcolo della nuova deviazione.

Quindi il valore del RTO è dato da il valore della nuova $RTT + 4$ per la nuova deviazione standard:

$$RTO \leftarrow RTT + 4 \cdot D$$

Ricordiamo che RTO è calcolata dopo aver calcolato l'aggiornamento di RTT.

5.6.2 Controllo di congestione

Dipende dalla versione di TCP, il meccanismo utilizzato per capire se c'è congestione in rete è fatto basandosi sugli ack. Può essere fatto in vari modi:

- Se non arriva un ack allora può essere successo qualche cosa o al segmento dati oppure all'ack stesso. Se è andato perso uno dei due è perché ha trovato congestione all'interno della rete (bassa probabilità che si sia modificato), ovvero che un router in mezzo alla comunicazione aveva il buffer pieno ed ha buttato via il nostro ack.
- Arrivano ack duplicati, quello che succede è che gli ack cumulativi vengono costantemente mandati indietro prima che scadano i timeout in piggyback. Posso dedurre che dall'altra parte qualche cosa è arrivato, però non sta andando tutto bene altrimenti riceverei un ack nuovo. Questa è considerata una situazione di congestione lieve.

In qualche modo chiediamo a TCP di reagire a queste congestioni, lui alleggerisce il carico di lavoro sulla rete, fa questa cosa tramite degli algoritmi che ora andiamo a vedere.

Congestion window

E' in realtà un numero. E' un valore tale che un processo mantiene la dimensione della send window come il valore minimo tra la finestra di ricezione (quantità di dati che l'altra parte può memorizzare senza buttare via niente) e la congestion window (è una stima che facciamo su quello che possiamo mandare senza congestionare la rete). La congestion window è un numero che calcoliamo tramite una funzione, quando due stazioni iniziano a parlarsi il mittente parte considerando la dimensione della finestra di invio pari ad una maximum segment size, viene

ricevuto un ack per quel segmento ed incrementiamo la dimensione della finestra di 1, quindi la dimensione della finestra è raddoppiata e manda il doppio dei segmenti. Quindi in questa prima fase di slow start una volta che vengono ricevuti tutti gli ack nella finestra di invio la finestra raddoppia, fino ad un valore massimo. Poniamo una soglia SST di 32 e al raggiungimento della soglia noi passiamo da una crescita esponenziale a lineare, ogni volta che arriva un ack incremento la dimensione della finestra della finestra $+ \frac{1}{cwnd}$, quindi ogni volta che ho ricevuto tutti gli ack nella finestra aumento la dimensione della stessa di 1. Si cresce fino ad una seconda soglia in cui non si permette più di crescere. Questa seconda fase si chiama congestion avoidance. Questo vale se tutto va bene e ricevo tutti gli ack.

Situazione di errore

Se qualcosa va storto dipende dalla versione di TCP, alcuni distinguono tra congestione lieve e grave, altri applicano la stessa strategia. TCP Tahoe: In ascisse che il tempo in ordinate la dimensione della finestra. TCP Tahoe è drastico se scade il timeout (quindi congestione grave) cerca immediatamente di ridurre il più possibile il traffico in rete, la dimensione della finestra di congestione diventa 1 MSS ed inoltre diminuisco la mia crescita, setto la prima soglia (ssthresh) dimezzandola rispetto a dove mi sono fermato prima. Cosa succede se riceve degli ack duplicati, se riceve lo stesso ack per 3 volte fa la medesima cosa, riporta la dimensione della congestion window ad 1 MSS e come prima la nuova dimensione della soglia è la metà di dove mi sono fermato prima.

TCP Reno: Parte la connessione e come prima la ssthresh è 16, arrivato a poter allocare 8 segmenti di taglia massima e scade un timeout, dato che è una congestione grave fa come prima, riparte da 1 con la ssthresh che è la metà di dove mi sono fermato. Alla seconda interruzione ad una dimensione della congestion window pari a 12 riceviamo il terzo ack uguale, dato che la congestione non è grave la nuova soglia è il valore attuale della finestra di congestione mezzi (nel nostro caso 6), il nuovo valore di congestion window è il nuovo valore di soglia più 3 valori di MSS, quindi 9 nel nostro caso. Possono succedere tre cose:

- C'è un timeout, quindi mi comporto come prima
- Mi arrivano 3 ack duplicati, e rifaccio la stessa cosa che ho appena fatto
- arriva un ack nuovo, succede una cosa controintuitiva, proviamo a ripartire dalla soglia con una crescita lineare

5.7 Chiusura della connessione

Non è banale, abbiamo già notato che esiste un code bit FIN. E' simile al passo e chiudo con le radioline. Abbiamo un algoritmo che fa di tutto per garantire che i dati arrivino dall'altra parte in modo ordinato, quando chiudiamo dobbiamo essere sicuri che il nostro lavoro lo abbiamo davvero fatto, ovvero che l'altra

parte ha ricevuto tutti i dati che abbiamo mandato. Quindi la chiusura implica che ci sia incastrato dentro questo problema. Nella figura vediamo un problema che dev'essere risolto, quello che succede che R chiude la socket senza mandare l'ack per il messaggio $x+1$, chiudere il canale in modo drastico significa per OS di R cancellare il connection record, dimenticarsi tutto di quella socket, rilasciare il numero di porta rilasciato in locale che può essere adesso usata da qualcun'altro. TCP quando chiudiamo non si ricorda più nulla. Quindi se viene fatta una close da R e dopo arrivano dati da S sempre sullo stesso canale TCP, R cosa se ne fa? Serve un accordo sulla disconnessione.

5.7.1 Chiusura simmetrica

Primario e secondario (il traffico può essere full duplex tra questi due, quindi vogliano essere certi che l'altra parte ha ricevuto tutto). Quando il primario decide che non deve mandare più niente (close nell'applicazione) manda un segmento con il FIN bit alzato e con un numero di seq che è pari alla quantità di dati mandati da P ad S più uno, gli stiamo comunicando quanti dati gli abbiamo mandato. Se il secondario non ha nulla da mandare manda anche lui un FIN bit alzato, più il numero di sequenza che ha valore numero di byte mandati da S a P più uno, inoltre manda un ACK con valore dato dal primario come numero di sequenza $+1$. Come terza via P manda ad S un ack con numero $\max \text{sent } S + 2$.

Se imponiamo che la chiusura è simmetrica abbiamo un problema (il solito sincronismo tra applicazione e quello che fa TCP), se S ha ancora dati da mandare possiamo scegliere di buttare i dati da mandare oppure è che i dati vengano flushed, ovvero che rifiuti i dati nuovi dall'applicazione ma i dati che già avevo li mando tutti dall'altra parte prima della finalize e poi manda la finalize.

5.7.2 Chiusura asimmetrica

E' il caso più generale. Il primario come prima dice che la sua applicazione non deve più mandare, manda il segmento di finalize con FIN bit alzato con numero di sequenza numero di bit mandati $+1$. S manda un ack in cui l'ack number riscontra tutti i byte che hai mandato $+1$, però S non chiude ancora, se il secondario ha altri dati da spedire li spedisce aspettando gli ack fino a che anche nel codice dell'applicazione del secondario c'è una close ed a quel punto anche lui manda il segmento di finalize di cui si aspetta un ack da P e poi si chiude effettivamente la connessione. I segmenti di chiusura possono essere persi, dobbiamo avere un sistema per non lasciare i processi pendenti.

5.7.3 Rilascio connessione

Dopo qualche tentativo di invio del segmento di finalize si chiude la connessione. A questo punto c'è un problema dall'altra parte, la cosa peggiore è che ci sono stati dei problemi in rete per cui l'altro processo esiste ma non riceve più niente, quindi sta appeso a ricevere dei dati senza ricevere né dati né finalize. Anche

questa è una situazione che va risolta (è in starvation, sta in attesa di un evento che non si verificherà mai). Quindi ogni volta che un TCP su una connessione fa partire un timer, quando quel timer scade si chiude la connessione per sospetto crash dell'altra parte. In realtà questo secondo timer ci introduce un nuovo problema, S potrebbe non ricevere dati da P non perchè P ha chiuso o non c'è un cammino, ma perchè l'applicazione stessa di P è lenta a generare i dati. Per evitare questo abbiamo il keep alive timer, S non sta ricevendo dati da P da un certo tempo, arriva al punto in cui è passato tando dall'ultimo dato che ha ricevuto, per sicurezza fa un controllo rapido, se non ho ricevuto dati per un keep alive timer (2 ore) mando dall'altra parte dei segmenti (normalmente l'ultimo che è stato ack) solo per vedere se riesco a raggiungere l'altro e ricevere un ack, se lui mi manda un ack la socket di P è attiva e il cammino esiste vuol dire che lui semplicemente non ha niente da mandare, altrimenti, dopo 10 tentativi S chiude perchè deduce che non riesce più a parlare con P.

Dobbiamo comunque risolvere il problema dei delayed duplicates, il processo chiude la socket, il processo rilascia la porta e l'OS da quella porta effimera ad un altro processo. A questo punto arrivano dalla rete dei segmenti della connessione prima. Anche quando la procedura TCP è riuscita a terminare l'OS non mette a disposizione quella porta ad un altro processo, ma aspetta due volte il maximum segment lifetime, che è il tempo massimo che un segmento può stare in un router, appunto per evitare che arrivino dei nuovi dati una volta chiusa la connessione.

5.8 Altre caratteristiche

Persistence timer, questo è legato alla window update che inviano nello header TCP, che è il meccanismo che ci serve per fare controllo di flusso. Ci può causare un problema, supponiamo che una parte abbia esaurito il suo buffer di ricezione, può essere che se abbiamo un ricevente in cui l'applicazione destinataria dei dati è lenta può essere che il buffer si riempia e la notifica mandata è che il buffer ha spazio 0. Può essere pericoloso perchè la notifica che riabilita dall'altra parte a mandare i dati viene persa, si crea quindi una situazione di DEADLOCK. Per evitare questa situazione si usa un persistence timer che funziona in questo modo: quando il ricevente segnala che la sua finestra di ricezione = 0 e blocca la sorgente, la sorgente fa partire un persistence timer, se il sender non riceve nulla quando scade il timer il sender manda un segmento tentativo per vedere cosa viene detto, viene mandato l'ultimo segmento che è stato ack perchè non crea problemi. Nel momento in cui il ricevente scarta il segmento duplicato lo scarta, manda un ack e nell'ack manda anche la window size, se è maggiore di 0 il sender può riprendere a mandare. Un altro aspetto legato al controllo del flusso è quello della sindrome della silly window: ho un ricevente che lavora lentamente (consuma pochi dati per volta e tende a lasciare riempire il buffer di ricezione di TCP), il rischio è che quando quel ricevente preleva qualcosa dal buffer di ricezione avrei la tentazione di dire all'altra parte di mandare i dati, ma potrebbe essere che ci vengano mandati una quantità di dati irrisoria che ci fa fare molto lavoro per niente, succede che andiamo avanti continuando a

bloccare e sbloccare il sender per pochi dati alla volta. Clark (ricercatore che ha studiato gli algoritmi) si e' inventato l'algoritmo di Clark per ricevitori lenti: quando io ho bloccato l'altra parte (mandato window size = 0), non manda un update maggiore di 0 fino a quando lo spazio libero nel suo buffer di ricezione e' il minimo tra il MSS (maximum segment size) e la meta' del buffer di ricezione. Manda il window update quando puo' ricevere una quantita' di dati rilevante.

5.9 UDP - User Datagram Protocol

E' un protocollo di trasporto per un servizio a livello superiore connectionless ed inaffidabile (non garantisce una mazza), da lo stesso servizio best effort di IP. Cosa aggiunge ad IP? L'unica cosa che aggiunge e' l'indirizzamento di livello 4 (il numero di porta per fare demultiplexing) e un po di controllo dell'errore, il checksum come nello header del segmento TCP. Non ingloba ne controlli di flusso ne di congestione, significa che con TCP portare attacchi di delay of service (per buttare via dati e saturare la CPU rendendo indisponibile il servizio) non si puo' fare perche' riduce la finestra di congestione, ma con UDP si puo' fare benissimo, tant'e' vero che molti amministratori bloccano il traffico UDP in ingresso, a meno di situazioni particolari. UDP va bene in particolare per traffico multicast (broadcast, tutte le destinazioni sulla rete, oppure un sottoinsieme di destinazione per un certo traffico) e multimedia (sottoinsieme di destinazione per un certo traffico, possono stare ovunque). TCP non va bene per multicast perche': pensiamo al meccanismo di trasmissione e ritrasmissione e per stimare il round trip time e di conseguenza il retransmission timeout, se noi parliamo con piu' destinazioni cosa succede se non tutte ci mandano un ACK per ogni segmento? il segmento lo ritrasmettiamo a tutti? o soltanto a chi non ci ha mandato l'ack? diventa un caos starci dietro. Così come TCP non va bene per il multimedia perche' pretende di fare arrivare a destinazione proprio tutto, ad esempio se manda un fotogramma TCP si blocca, che vuol dire che la qualità percepita di chi guarda il video è pessima. Come qualità percepita per fruizione di multimedia è preferibile saltare qualche cosa e non frizzarsi. A differenza di TCP, UDP permette la frammentazione, TCP numera i singoli byte del segmento e viene decisa la misura del MSS per non frammentare, perche' sarebbe problematica rinumerare tutti i sotto-segmenti. UDP se ne frega e fa i messaggi della dimensione che vuole tanto non è affidabile. UDP preserva i confini dei messaggi dell'applicazione, TCP cercava di rendere grande ogni segmento di MSS per ammortizzare il costo dello header.

Dal punto di vista della programmazione siamo messi come TCP, dobbiamo crearci una socket, dobbiamo legarci un indirizzo con la bind, quando abbiamo finito di parlare dobbiamo chiudere la socket con la close. La differenza che una socket intesa come punto terminale di una comunicazione UDP può essere usata per parlare con più destinazioni differenti perche' ogni volta che mando un messaggio devo specificare a chi va quel messaggio. Quindi non esistono primitive di listen, connect, accept per svolgere 3 way handshake, sono diverse le primitive per i messaggi perche' dobbiamo specificare chi è il destinatario ed il mittente.

La socket non è connessa ad una socket remota (connectionless). Dato che la socket non è connessa nel buffer di ricezione possono arrivare messaggi da più utenti, l'ordine di estrazione è sempre FIFO, ma quando lo estraiamo dobbiamo dire all'applicazione anche chi è il mittente che ha inviato quel messaggio (header livello 3 e 4)

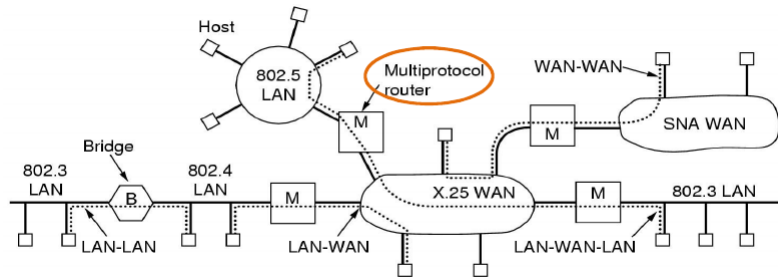
5.9.1 Header UDP

E' di 8 byte, contiene soltanto la porta sorgente e la porta di destinazione, la lunghezza del messaggio UDP e contiene un minimo di aiuto di applicazione perchè contiene il checksum, calcolato esattamente come TCO, sullo pseudo-header. L'identificatore di protocollo usato a livello 3 è 17. La taglia massima di un messaggio UDP è 64Kb, in cui ci dev'essere spazio anche per gli 8 bit di header UDP e 20 byte di header livello network.

Chapter 6

Livello Rete

Il livello rete nello stack OSI è il più alto dei livelli end to end. Per riuscire a connettere due end system abbiamo bisogno di router che implementino solo protocolli TCP/IP, sono router multi-protocollo, generalmente chiamati gateway, perchè non capiscono solo il formato dei pacchetti ma capiscono anche protocolli di livello più alto.



Funzionalità del livello:

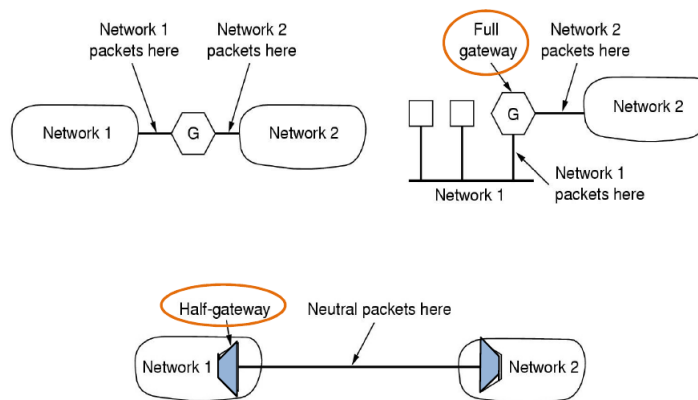
- Routing: dobbiamo fare instradamento, creare grafi tra router. Scoprire topologia e trovare cammini migliori tra ogni sorgente ed ogni destinazione
- Addressing: si utilizzano gli indirizzi di livello rete (172.34.87.205) per differenziare i vari end system ed i router. Può anche essere necessario tradurre tra indirizzi di livello network (IPv4 o 6, sono dati in base alla posizione geografica) e indirizzi di livello 2, i MAC address. Gli indirizzi MAC non hanno connotazione geografica, quindi sono degli indirizzi di 48 bit (anche wireless) scritti nel firmware della scheda di rete. Ci può servire tradurre da indirizzo di rete ad indirizzo di livello 2 (che facciamo corrispondere alle local area network, quelle per forza broadcast) perchè nelle reti LAN non si fa instradamento, supponiamo che ci sia un Sorgente per un Destinatario in due LAN diverse, quello che possiamo fare è trovare l'indirizzo IP, il sorgente scrive quindi nei pacchetti di livello 3 l'IP address del destinatario ed i router usano questo indirizzo per guidare il messaggio, fino al router che si interfaccia alla LAN, che non si fa più niente dell'indirizzo IP perchè non può fare instradamento, deve quindi guardare nello header di livello 2 per trovare il MAC address del destinatario. Tutti i router al confine con le LAN devono quindi fare questa traduzione tra indirizzi IP e MAC.
- Fragmentation: il livello rete è quello che frammenta per limitare la dimensione massima delle protocol data unit. Quando passo da una rete all'altra che supportano dimensione dei pacchetti diversi.
- Monitoring e gestione errori: si occupa anche di verificare che la rete funzioni, esiste un protocollo ICMP per fare questa cosa.
- Traduzione tra reti diverse

- Diversi formati indirizzi
- Diverso formato header
- Supporto a sicurezza

6.1 Gateway

E' un router multiprotocollo intelligente che conosce più stack di protocolli che generalmente conosce anche protocolli di livello superiore. Sta tra reti differenti che usano protocolli differenti. Due tipi:

- Full gateway: è un router che fa la traduzione tra formati di protocolli di tutte le reti che ha sulle sue interfacce. Un gateway sa tradurre dal formato di una qualunque delle reti che è collegato al formato di una delle altre due (6 combinazioni possibili), se ne aggiungiamo una 4 abbiamo 12 combinazioni possibili. La situazione che si comporta meglio è quella di non usare dei full gateway.
- Half gateway: è un router che sta al confine della rete che traduce dal formato dei pacchetti (protocolli) di quella rete in una sorta di linguaggio universale e viceversa nel linguaggio universale al linguaggio della sua rete per il traffico entrante. Qualunque siano le connessioni deve sapere solo due linguaggi. Il linguaggio universale è quello che usa il protocollo di Internet (TCP, UDP, IP)

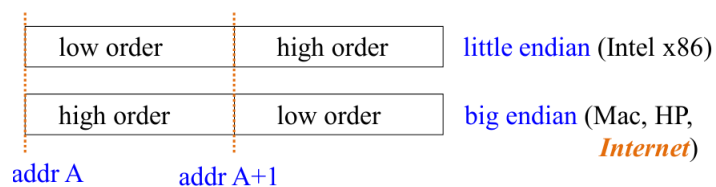


6.2 Internet Protocol (IP)

Il protocollo di Internet di livello 3. IP porta solo le informazioni da qui al prossimo hop che mi sembra buono, però non fa altro, è un banale trasportatore ed usa dei protocolli compagni per fare tutte le operazioni che non siano di trasporto.

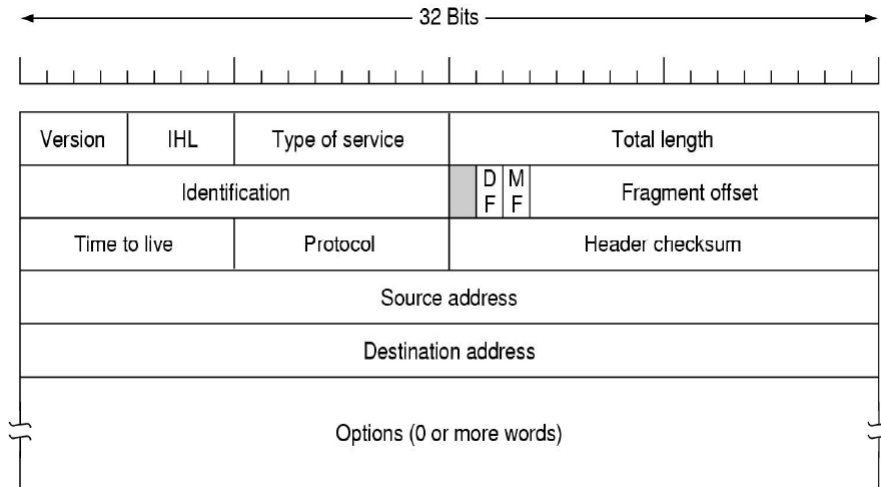
IP fornisce un servizio best effort per niente garantito. Fornisce un servizio di trasmissione datagram su reti interconnesse, datagram che possono avere dimensione massima di 64Kb.

In funzione del linguaggio con cui lavoriamo per sviluppare applicazione che usano la rete può essere necessario fare delle traduzioni tra un formato dati all'altro, tra little endian e big endian. Ad esempio se usiamo un PC con architettura little endian e passiamo i dati su Internet che usa architettura big endian andiamo da tutta un'altra parte, è quindi fondamentale che IP riordini i byte all'interno degli indirizzi in modo che le informazioni vadano nella direzione giusta.



6.2.1 Header IPv4

E' lunga 20 byte, in più ci possono essere delle opzioni.



- Version: Il primo campo riporta il numero di versione
- IHL: la header lenght perchè ogni apparato successivo di Internet devono guardare lo header per spedire il pacchetto e questo campo dice se devono guardare anche delle opzioni
- Type of service serve per fare qualità di servizio è un ottetto, 3 bit per la priorità (da 0 a 7). Su ogni interfaccia i router hanno due buffer, uno per i pacchetti in input ed un buffer di output. La priorità è associata al buffer di output che di default è gestito come FIFO tranne se abbiamo qualcosa di prioritario. Possiamo che tipo di servizio quel pacchetto vuole (alta affidabilità, grosso throughput, costo se usiamo rete di provider diversi).
- Lunghezza totale del pacchetto, 2 ottetti
- Flag inutilizzato
- DF, flag di dont fragment, tutti i segmenti TCP partono e vengono incapsulati in pacchetti IP che hanno il dont fragment bit alzato
- Identification: numero pseudocasuale su 16 bit, che identifica tutti i pezzi del segmento spezzato
- Fragment offset: spiazamento, 0 nel primo frammento utilizzato e poi la posizione in cui questo pacchetto deve iniziare rispetto a tutto quello che lo precede.

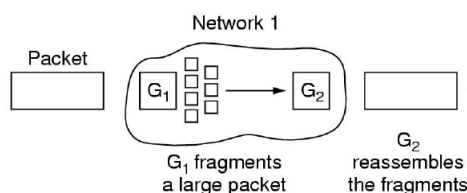
- Flag MF, more fragment è 1 tutte le volte che il frammento non è l'ultimo del puzzle ed è 0 per l'ultimo frammento.
- Time to live: serve per dire quanto a lungo può stare in circolazione un pacchetto nella rete. L'instradamento viene fatto in modo autonomo da ogni router, può essere che le informazioni su cui ragionano i router non siano tutti uguali (vero soprattutto nei cambi di topologia), significa che la visione inconsistente della struttura di rete potrebbe fare prendere dei cammini inconsistenti ai pacchetti (magari anche dei loop). E' il numero massimo di hop che un pacchetto può fare per raggiungere la destinazione, viene decrementato ogni router visitato, quando arriva a 0 il pacchetto viene buttato via.
- Protocol: TCP fa il calcolo sullo pseudo header considerando che nel campo IP del numero di protocollo ci sia il suo numero di protocollo. Serve ad IP a fare demultiplexing
- Header checksum: è un controllo di correttezza fatto solo sullo header di livello 3 e deve essere ricalcolato ad ogni hop.
- 32 bit per l'indirizzo sorgente
- 32 bit per l'indirizzo destinazione
- Opzioni: riguardano principalmente l'instradamento, 1Byte di codice + 1Byte per la lunghezza + i dati, le opzioni sono:
 - Sicurezza, dovrebbe specificare quanto è riservato il datagram, ma in realtà non sono riservati
 - Strict source routing: non sono i router che decidono la rotta di un pacchetto, ma è lo header del pacchetto che tramite questa opzione indica quale cammino dev'essere fatto. Può avere due usi, siamo in territorio ostile, non ci fidiamo della rete in cui siamo per cui facciamo passare il traffico dato solo per apparati di cui ci fidiamo. Può anche avere un uso malevolo, per attaccare alcuni dispositivi facciamo passare una valanga di traffico per i dispositivi che vogliamo mandare in sovraccarico in modo che non funzionino. Spesso questa impostazione è disabilitata.
 - Loose source routing: nello strict c'è tutta la sequenza di dispositivi che devono essere collegati, nel caso di loose metto dei flag ogni tanto, non sono adiacenti due router in questa lista.
- Record route: ping, ha accesso ad ICMP (protocollo di livello 3) ed ha tradotto l'alias in un server. Abbiamo inviato tot pacchetti e facciamo delle statistiche sul round trip time. Permettono di ricordare la rotta, perchè posso ricordare tutti i router intermedi attraversati tra la sorgente e la destinazione, perchè il router aggiunge il suo indirizzo IP al pacchetto.
- Timestimp: permette ai router attraversati anche il tempo in cui il pacchetto passa dal router.

6.3 Fragmentation

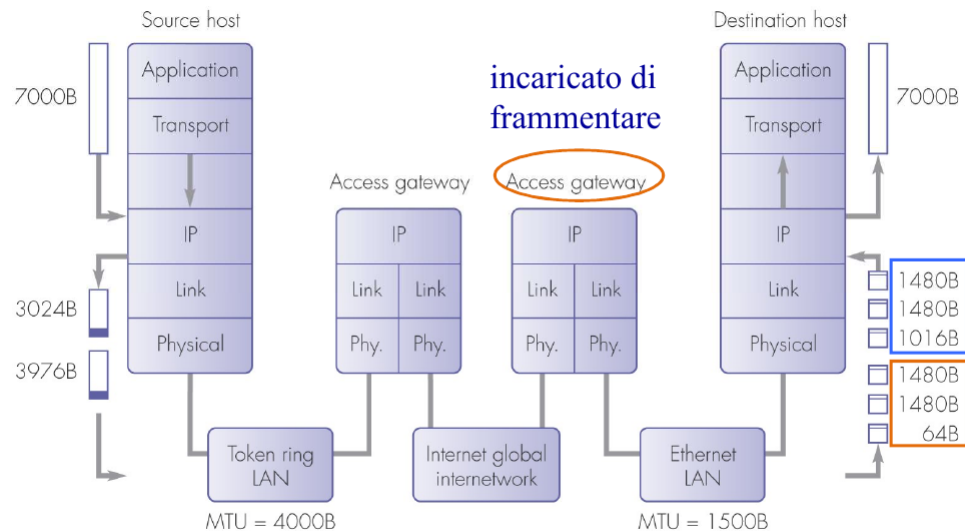
Nelle funzionalità del livello network c'è anche la frammentazione. Ne abbiamo anche parlato nelle lezioni introduttive per limitare la dimensione dei pacchetti per facilitare la gestione della memoria nei router. Le reti non sono tutte uguali, per ciò che è Internet sappiamo che ci sono dei vincoli (segmenti in TCP con MSS maggiore di 536 byte). Allo stesso tempo in Internet non tutto è conforme alle limitazioni dello stack, ci sono anche delle reti che permettono di far girare dei jumbo packet (pacchetti maggiori di 64 kb). Quindi può essere che un pacchetto debba passare in una rete che non supporta pacchetti così grossi, quindi il gateway che si vede arrivare questo pacchetto capisce che non può gestire questo pacchetto e deve frammentarlo, con i problemi già visti, un frammento è una porzione di quello che sta arrivando ed ogni frammento deve avere il suo header (quindi stiamo replicando le informazioni di controllo per ogni frammento), più o meno perchè cambia il valore di fragment offset nello header IP. Lungo il cammino i frammenti potrebbero uscire dalla rete passando per un nuovo gateway ed essere ricompolti. Esiste la frammentazione:

- Trasparente: perchè la sorgente e destinazione non si accorgono che durante il cammino i pacchetti che si stavano scambiando sono stati decomposti, fanno la decomposizione e ricomposizione i gateway in automatico
- Non trasparente: dopo che un gateway ha frammentato il pacchetto, i frammenti arrivano così come sono alle destinazione che dev'essere lei a ricomporre i vari frammenti prima di passarli ai livelli superiori

In Internet è più comune la frammentazione trasparente.



Dobbiamo anche fare un conto di quanto fare piccoli i frammenti. Uno host deve mandare 7kb di dati ed è in una rete lan che supporta pacchetti di 4kb, quindi spezza i dati in due pacchetti (ocio che non possiamo avere 4kb esatti per pacchetto, perchè c'è lo header di 20byte). I dati viaggiano ed arrivano alle soglie della rete dello host di destinazione, che usa un protocollo Ethernet che supporta pacchetti di 1500B, dobbiamo quindi frammentare, lo facciamo tenendo conto che per ogni frammento deve esserci una replicazione dello header, che sappiamo essere di 20 Byte.



Note: All values shown are the amounts of user data in each packet/frame in bytes

6.4 IP addressing

Parliamo dell'indirizzamento. Ci sono tanti schemi di indirizzamento, nella versione di protocollo IPv4 gli indirizzi di rete sono unsigned long di 32 bit, che per essere più manipolabili da noi umani sono 4 ottetti rappresentati in notazione decimale puntata. Quindi la trascrizione di ogni ottetto può avere una rappresentazione in decimale tra 0 e 256. Gli indirizzi devono essere globalmente unici, vedremo che non sono totalmente piatti ma hanno una componente (più a sinistra) che rappresenta una rete e la parte più a destra che rappresenta uno host all'interno della rete. L'unicità globale serve per permettere l'instradamento, non ci possono essere due apparati che hanno lo stesso indirizzo perchè non sappiamo dove mandare geograficamente il pacchetto. E' fatto dall'organizzazione ICANN (già visto per il DNS) che si occupano di assegnare nomi e numeri.

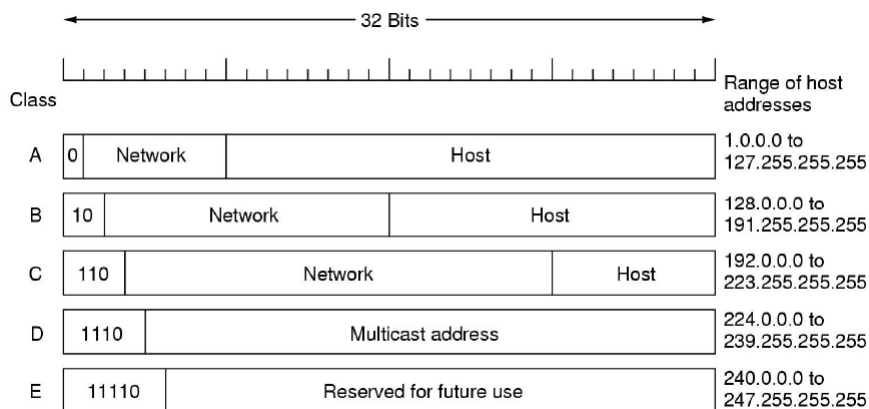
Gli indirizzi IPv4 sono in esaurimento (in realtà sono esauriti), quindi ad un certo punto si è standardizzata la versione 6, che usa indirizzi di 128 bit, è lento lo switch perchè bisogna cambiare tutti gli apparati di rete che usano IPv4. Concentriamoci su IPv4 e guardiamo i suoi schemi di indirizzamento:

- class-based addressing
- Subnetting
- Classless addressing (CIDR)
- Network address translation (NAT)

6.4.1 Class-based addressing

Lo schema più vecchio era l'indirizzamento basato su classi, avevano deciso che c'erano 5 classi di indirizzi:

- A: il primo ottetto per identificare la rete (la parte più significativa), il primo bit è 0 per fare capire ai router che hanno a che fare con un indirizzo di classe A. Inoltre hanno 3 ottetti per identificare i dispositivi in rete
- B: i primi due bit del primo ottetto sono 10, significa che usano i primi due byte come identificare di rete e i secondi due come identificatori di host.
- C: hanno i primi 3 bit che sono 110, che usa i primi 3 ottetti per il network ID e l'ultimo come Host ID
- D: è utilizzata per il multicast, ovvero serve per identificare due dispositivi che fanno parte dello stesso gruppo ma possono essere ovunque nel mondo. I primi 4 bit sono 1110.
- E: non è utilizzata



Come vengono utilizzate le classe A, B, C. La divisione serve per permetterci di essere più efficienti nella funzione di instradamento. Una tabella di instradamento ha per ogni riga queste informazioni:

- L'indirizzo di destinazione
- Qual'è il next hop
- La metrica, quanto manca per arrivare a destinazione (numero di hop)
- Flag, che servono per ragioni amministrative (se il cammino esiste, se è visibile per PING o trace route)

Se fosse stato così per indirizzi IPv4 dovremmo avere in ogni router una tabella con tutti gli indirizzi IPv4 possibili per poter instradare correttamente il pacchetto. Ovviamente cercare qualche cosa in una tabella disordinata ci costa tantissimo. Per evitare di fare tutti questi confronti, i router capiscono dai primi bit della rappresentazione (159) capiscono che è un indirizzo di classe B (ad esempio), quindi guardano i primi due ottetti e cercano una strada per arrivare ad un apparato a caso nella rete di destinazione. Quindi per ogni rete (ad esempio la rete di UNIMI) in tutti i router nel mondo c'è una sola riga nella tabella di instradamento, quando poi il pacchetto arriva nella rete di UNIMI si arrangiano quegli apparati. E' fatto con la netmask. Quando un router capisce a quale classe appartiene l'indirizzo di destinazione del pacchetto, mette tutto l'indirizzo di destinazione (che c'è nello header IP), in and bit a bit con una maschera costituita da tutti 1 dove c'è un bit che identifica la rete e tutti bit 0 nello spazio che rappresenta l'host ID in una rete (per la classe B 16 bit 1 + 16 bit 0). Serve per estrarre l'identificatore di rete e portare a 0 l'identificatore di host.

Questo fa sì che abbiamo indirizzi particolari che non possono essere usati da nessuno:

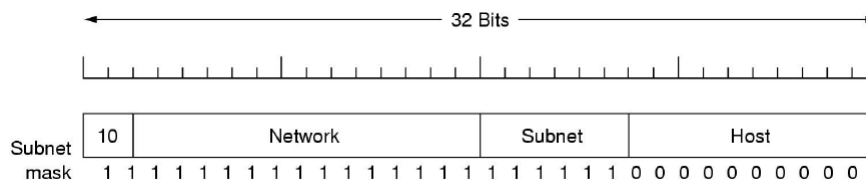
- Se la parte di host ID è tutta a zero (159.128.0.0) è l'indirizzo base della rete, che non può essere assegnata a nessun dispositivo.
- Se la parte di network ID è uguale a zero stiamo dicendo che dobbiamo trovare uno host che è nella stessa rete del sorgente
- Se tutti e 32 i bit sono 1, vogliamo mandare un broadcast sulla rete della sorgente
- Host ID è tutta 1, la destinazione è tutti i dispositivi sulla rete destinataria identificata da network ID
- 32 bit 0, è un qualunque indirizzo. E' utilizzato a livello di applicazione per dire che sono un router che accetto comunicazioni su qualunque interfaccia di rete se ho più interfacce di rete attive.

Ho due problemi:

- Molta distanza tra una classe e l'altra, da oltre 16 milioni per A a 65k apparati per B a 254 per classe C. Sono salti grossi e creano problemi, perchè ogni amministratore vuole prendere indirizzi grossi nel caso in cui l'azienda cresce, quindi molte organizzazioni potevano benissimo starci in classe C, ma hanno portato via indirizzi a qualcun altro. Viene risolto facendo saltare il sistema delle classi passando ad uno schema senza classi che ci permette una granularità più fine.
- Per reti molto grandi (A e B) è difficile capire dove sono i dispositivi, perchè sono sparpagliati sul territorio. Un tentativo di individuare un particolare dispositivo può essere un'idea migliore dividere gli indirizzi assegnandogli dei valori (ad esempio per UNIMI per reparti). Risolto con l'idea del subnetting.

6.4.2 Subnetting

Abbiamo una rete grossa ma vogliamo dare un senso agli indirizzi che assegniamo, ad esempio di classe B che può assegnare indirizzi per 65k apparati, esclusivamente all'interno della rete possiamo capire che la nostra organizzazione ha un numero k di divisioni (o edifici o quello che vogliamo), individuiamo queste componenti con un identificatore di subnet per tutti gli host che stanno in quella componente, ed i restanti identificano l'host in quella sottrete.



Nell'esempio si è usato 6 bit per la sottorete ed i rimanenti 10 per l'identificatore di apparato. Per tutti gli apparati all'esterno dell'organizzazione non devono conoscere nulla della nostra organizzazione in sottoreti, quindi applicano una maschera di 16 bit 1 seguiti da 16 bit 0. Se invece siamo all'interno della rete, viene estesa la netmask perchè sappiamo che la rete prevede di utilizzare i 6 bit più significativi dello host ID per rappresentare il subnet ID, quindi usiamo una maschera fatta da 16 bit 1 per controllare che siamo nella nostra rete + altri 6 bit 1 per tirare fuori l'indirizzo di subnet e scoprire in che sottorete sta l'apparato da raggiungere. Ovviamente un router sa raggiungere tutti i dispositivi nella sua sottorete, gli viene passato quindi solo l'indirizzo generale della sottorete.



6.4.3 Classless Inter-Domain Routing

Abbiamo tolto le classi permettendo che il punto di divisione tra network ID e Host ID sia dove vogliamo, è molto comodo per dare il giusto numero di indirizzi ad ogni organizzazione. Noi dobbiamo trovare la potenza di 2 più piccola tra le più grandi che vogliamo indirizzare, se dobbiamo indirizzare 1000 dispositivi la potenza di 2 più piccola tra le più grandi di 1000 è 1024.

L'indirizzo di una rete è rappresentato in notazione decimale puntata $x.y.w.z$, dove n dice quanti sono i bit più significativi usati per la network ID, ci dice automaticamente come dev'essere fatta la netmask per tirare fuori la network ID.

Hanno anche assegnato gruppi di reti di classe C, per esaurirle, a continenti, in pratica abbiamo assegnato ad un continente 131072 reti. L'idea è

di riuscire comunque ad assegnare dei gruppi di reti contigue di classe C ad una organizzazione, in modo che riusciamo a far corrispondere a quella organizzazione soltanto un indirizzo di base, nascondendo tramite la netmask i bit che differiscono.

6.4.4 CIDR e routing

Il router che analizza il pacchetto analizza in ordine le righe della routing table, per ogni riga calcola la AND bit a bit tra l'indirizzo del destinatario e la mask che c'è in quella riga, confronta il risultato con l'indirizzo base di quella riga, se l'indirizzo corrisponde inoltra il pacchetto verso l'indirizzo indicato in next hop altrimenti guarda la riga successiva. A questo punto la maschera è lunga n bit a 1. La n può avere valore qualunque tra 1 e 30, le reti 30 sono sensate, perchè identificano una rete tra due router (ci sono solo due host identificati).

Cambridge: 194.24.0._ a 194.24.7._ ovvero da
 11000010.00011000.00000000._ a 11000010.00011000.00000111._
 □ sono $256 \times 8 = 2048$ indirizzi su 8 reti $\rightarrow 256-8=248$
 □ mask: $255.255.248.0 = [1].[1].11111000.[0]$
Oxford: 194.24.16._ a 194.24.31._ ovvero 16 reti per $(256 \times 16=)$
 4096 indirizzi, da 11000010.00011000.00010000._ a
 11000010.00011000.00011111._
 □ mask: $255.255.240.0 = [1].[1].11110000.[0]$
Edinburgo: 194.24.8._ a 194.24.11._ ovvero 4 reti per $(256 \times 4=)$
 1024 indirizzi, da 11000010.00011000.00001000._ a
 11000010.00011000.00001011._
 □ mask: $255.255.252.0 = [1].[1].11111100.[0]$

ES: Edinburgo 194.24.8._ a 194.24.11._ (mask
 11111111.11111111.11111100.00000000)

□_.8	11000010.00011000.000010	00
□_.9	11000010.00011000.000010	01
□_.10	11000010.00011000.000010	10
□_.11	11000010.00011000.000010	11

in tutti e 4 i casi lo AND con mask produce
 11000010.00011000.00001000 che è l'indirizzo base
 ricordato nella routing table

indirizzo base è 194.24.8.0/22

la cardinalità dei gruppi è una potenza di 2

6.5 IP routing

Il problema dell'instradamento consiste nel trovare un cammino buono per qualche criterio per collegare una sorgente e destinazione ed avevamo già parlato di diverse metriche. Devono essere soddisfatte alcune proprietà:

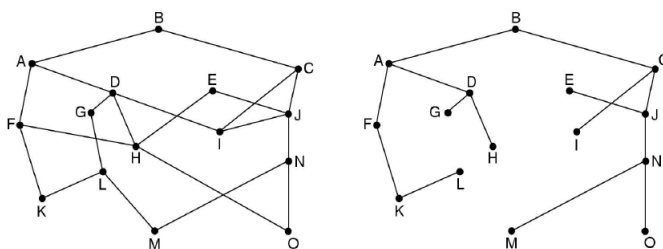
- Correttezza: la politica di instradamento deve trovare una rotta che esista, in cui tutte le componenti del cammino funzionino e sia priva di cicli. Abbiamo già visto che il campo TTL quando arriva a 0 buttiamo via il pacchetto.
- Semplicità: i router devono poter usare la massima parte delle risorse di computazione per smistare il traffico degli utenti.
- Robustezza: a fronte di cambiamenti di topologia, se si verifica un cambiamento di topologia (vengono aggiunti o tolti dispositivi dalla rete), l'algoritmo deve capire il cambiamento e trovare nuove rotte.
- Stabilità: l'algoritmo deve fornire dei cammini stabili, non deve oscillare. Il cammino tra sorgente e destinazione non deve cambiare in continuazione altrimenti tutti i pacchetti che stanno viaggiando, se le scelte di cammino cambiano, vengono smistati tra varie rotte creiamo una grande entropia nell'insieme di pacchetti che viaggiano verso la destinazione aumentando che qualcosa vada storto durante la trasmissione.
- Fairness vs. ottimalità: i cammini devono essere scelti tenendo d'occhio sia criteri di ottimalità sia di equità. Supponiamo di scegliere l'algoritmo sulla metrica della latenza, quindi il cammino più veloce è quello migliore che sarà formato da router con maggiore velocità di trasmissione e da canali con maggiore bitrate per cui i dati viaggiano più velocemente. Se il criterio è solo quello dell'ottimalità significa che tutto il traffico va a scegliere solo le rotte veloci e tutto il traffico si imbottiglia sulle medesime tratte di mezzo trasmissivo, mentre tutto il resto della rete non sta lavorando. E' una buona idea trovare anche un bilanciamento tra equità, ovvero che tutti i dispositivi di rete lavorano e nessuno risulta particolarmente sovraccarico ed il traffico fluisce in modo migliore.
- Algoritmi non adattivi (statici) vs. adattivi: due strategie, politica statica non è una politica, l'amministratore di rete conosce la topologia della rete e configura manualmente sugli apparati qual'è la via di uscita per il traffico ad una determinata destinazione, scelta che è configurata all'interno del router e non più cambiata da niente all'interno dell'apparato. Al contrario nel caso degli algoritmi adattivi i router sono autonomi e monitorano la topologia della rete, il carico e cambiano dinamicamente la rotta per instradare i dati per comportarsi al meglio in ogni istante.

6.6 Principio di ottimalità

L'idea è che una rete opera come un sistema peer to peer, ovvero tutti i router sono capaci di svolgere le stesse funzioni ed ognuno di loro opera indipendentemente dagli altri sulla base delle conoscenze che ha sulla rete, in modo che le decisioni siano prese in modo distribuito. Non c'è un'entità centralizzata che prende le decisioni per tutta la rete, sarebbe un collo di bottiglia, sarebbe single point of failure perchè se viene meno nessuno sa fare niente e diventa anche un singolo punto di attacco della rete. Quindi a meno che non stiamo utilizzando quello visto nelle opzioni dello header IP specificando su quali dispositivi deve passare il traffico (source routing), noi stiamo chiedendo ai dispositivi di fare instradamento in questo modo: un pacchetto arriva ad un router ed è il router che guarda l'indirizzo di destinazione del pacchetto ed in base alle informazioni che ha nella tabella di instradamento capisce qual'è il percorso migliore per il pacchetto. La tabella di instradamento contiene una riga per ogni destinazione, come riempire la tabella?

Il problema è come si deve comportare la globalità dei router per fare sì che un pacchetto arrivi alla destinazione, ad esempio se ci sono dati contrastanti in diverse tabelle di instradamento un pacchetto potrebbe arrivare in un loop e non arrivare mai a destinazione. Quindi quello che vorremmo è che ogni router si comporti in modo autonomo per inoltrare i pacchetti ma che globalmente le tabelle di instradamento dovrebbero contribuire a creare dei cammini che abbiano senso. Questo è riassunto dal principio di ottimalità, se il router J è sul cammino ottimale tra S e D, allora il cammino ottimale tra J e D si sovrappone a quello da S a D. Ottimale significa che è il meglio che è riuscito a trovare ma non è detto che sia il migliore in assoluto. Ovvero le decisioni che diversi router prendono riguardo il cammino migliore verso una destinazione sono coerenti, tutti prendono le stesse decisioni.

Nella figura di destra sono tutti i cammini che prendono i router per arrivare al B, i nodi sono concordi nel prendere determinate decisioni di instradamento. Il principio di ottimalità costruisce una serie di cammini migliori che è una struttura ad albero che ha come radice B (nell'esempio). Se questo albero lo vediamo al rovescio otteniamo un sink tree per la destinazione B. Il sink tree è una struttura ad albero (non ci sono cicli), sink (lavandino in inglese) raccoglie un insieme di rami e li convoglia verso un punto comune che è la destinazione. Il principio di ottimalità e algoritmi che trovano rotte rispettando questo principio trovano rotte che formano una struttura ad albero che trova il sink tree per quella destinazione. Quindi il problema dell'instradamento è quella di trovare un sink tree per ogni nodo nella rete.



6.6.1 Rooting Statico e Flooding

Routing statico, come dicevamo prima gli amministratori di rete configurano manualmente il percorso che fa un pacchetto per arrivare ad una destinazione. Ci vuole una certa cura per configurare le rotte statiche, perchè dobbiamo ricordarci le varie interfacce di uscite dei router per non creare dei cicli. Se si verifica un problema sulla rete il traffico che doveva passare per la rotta che avevamo configurato non ci passa più, quindi perdiamo i pacchetti. Se la rete è enorme dobbiamo ridurre la probabilità che una componente si guasti ed è impossibile se la rete è grande, quindi il routing statico va bene per reti piccole dove c'è una bassa probabilità di guasto.

Un'altra strategia è quella del flooding (alluvione), questo approccio di instradamento è un approccio, che come l'acqua in un allagamento, fa andare i pacchetti dappertutto, se un router genera un pacchetto lo manda su tutte le interfacce di uscita, se ha ricevuto un pacchetto lo manda su tutte le interfacce di uscita (tranne quella in cui ha ricevuto il pacchetto). Quindi l'idea è che non ha bisogno di nessuna conoscenza sulla topologia di rete, perchè provo tutte le strade. Abbiamo dei problemi:

- Terminazione: può essere che un pacchetto ritorni sui propri passi e generi dei cicli. Il numero dei pacchetti diventa un O grande del numero dei router al quadrato. Per buttare un pacchetto dobbiamo aspettare che il suo time to live arrivi a 0.
- I router tengono una storia dei pacchetti già inoltrati, ma dobbiamo capire cosa significa la storia (potrebbero ad esempio tenere lo header) e se tengono il pacchetto per una seconda volta lo buttano via. Inoltre quanto diventa grossa questa storia? ad un certo punto diventa molto costoso confrontare gli header passati con quello presente, inoltre se facciamo pulizia della storia passata potremmo non trovare il duplicato.
- Possiamo usare il selective flooding, per cui in pacchetto viene inoltrato su output interface che più o meno vanno dalla parte giusta, questo implica di configurare i router in modo che conoscano più o meno la parte giusta, quindi tanto vale che calcoliamo delle rotte.

Nelle reti wireless e mobili dove i dispositivi cambiano posizione uno rispetto all'altro il flooding è una politica adottata. Inoltre è il meglio che possiamo fare

se non vogliamo costruire delle rotte statiche nella fase iniziale della rete quando non conosciamo nessuna rotta. E' un approccio che sicuramente costa in banda e capacità dei canali, però dall'altra parte non richiede di mantenere info nei router, è robusta perchè finchè c'è almeno una rotta che congiunge sorgente e destinazione la troviamo, inoltre sicuramente trova anche il cammino ottimo (perchè li prova tutti).

6.6.2 Distance Vector, Bellman-Ford

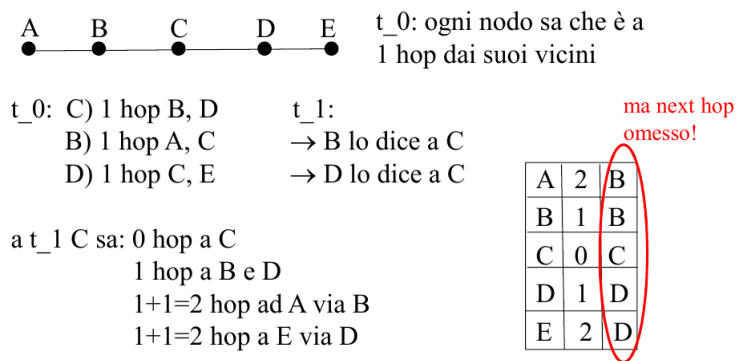
L'algoritmo di Bellman Ford dice che: dato un grafo (una rete è un grafo, i nodi sono i router e gli archi sono il mezzo trasmissivo che connette i router), se due nodi nel grafo sono direttamente connessi da un arco allora la distanza tra i due nodi è uguale al costo dell'arco che collega i due router, il costo è la metrica che utilizziamo (latenza, hop...). Se invece i due router non sono vicini, la distanza tra x e y è il minimo del costo per andare da x ad un vicino i più il costo per andare dal vicino i alla destinazione y. E passo dopo passo ci allarghiamo aumentando il numero di hop, alla prima iterazione abbiamo trovato cammini di 1 hop, alla seconda visitando i vicini del nodo sorgente troviamo cammini di 2 hop... fino ad arrivare alla destinazione.

$$D_{xy} = \min_{\forall i \in \text{vicinato}(x)} \{c_{xi} + D_{iy}\}$$

E' quello che è utilizzato da tutti i protocolli che usano un instradamento secondo un approccio distance vector, tra cui il primo algoritmo usato che è il routing information protocol (RIP) (RIP, quello che ci spiega per il distance vector è RIP). Ricordiamoci che la entry di una tabella di instradamento ha formato destintaion, netmask, output interface, metric, flag. Il router vede arrivare un pacchetto, prende l'indirizzo destinazione e lo mette in and bit a bit con la netmask, se il risultato è uguale a quello che c'è nella tabella di instrdamento allora il pacchetto viene inoltrato su quella network interface, con una serie di flag che ci danno caratteristiche su quel cammino. Questo è vero sempre, sia che si parli di rotte statiche, sia qualunque protocollo distance vector. Ogni router prende le decisioni di instradamento sulla propria tabella di instradamento costruita sulle base delle informazioni locali sulla rete. E' vero solo per distance vector che non è nota la topologia completa della rete, quindi i router prendono delle decisioni su informazioni parziali, quindi trovano dei cammini sub-ottimi (ottimali ma non ottimi). Il vantaggio è che gli approcci distance vector hanno un basso overhead (o/h, sovraccarico), caricano poco sia di computazione, di informazioni che circolano sia di quantità di memoria i router e la rete.

Esempio

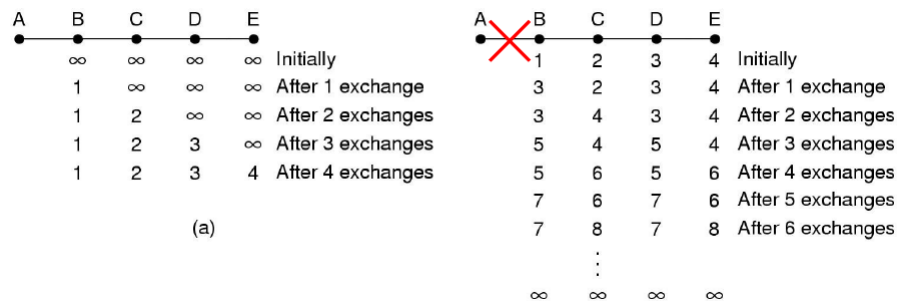
Supponiamo di avere una topologia di rete a bus, cosa conoscono e come fanno ad imparare le rotte. Al tempo zero i router mandano i segnali di portante sulle interfacce attive e ricevono il segnale portante dai vicini accesi, quindi in prima battuta riescono a scoprire chi sono i vicini e come si chiamano. Al tempo 1 ogni nodo dice ai suoi vicini chi ha vicino, C ha imparato che ha due vicini B e D ad 1 hop, esiste A che è a due hop via B, così come E via D. Quindi può costruire la tabella di instradamento in cui ha i nodi che conosce, la distanza, e dove mandare (output interface) il pacchetto per quel nodo. C quando manda i suoi vettori di distanza manda un estratto della sua tabella di instradamento, solo le prime due colonne, il next hop è omesso perché allora la banda dei canali era bassa, quindi si cercava di mandare meno informazione di controllo possibile. Quindi il vettore di distanza è un sottoinsieme della tabella di instradamento, è composto da destinazione e distanza.



Ad ogni passo di scambio di vettori di distanza i nodi imparano qualcosa che sta ad 1 hop di distanza.

Il problema è che reagisce male in caso di guasti Supponiamo che si rompa il collegamento tra A e B, B non riceve più vettore di distanza da A, ma riceve un vettore di distanza da C che gli dice che sa arrivare ad A in 2 hop (così come tutti gli altri nodi), il problema è che C (e tutti gli altri nodi) sanno arrivare ad A passando per B. Ad ogni scambio di vettori di distanza, la distanza stimata da A cresce di 1 fino ad un valore infinito, che non è veramente infinito ma è 256 se usiamo il numero di hop come metrica (che è il numero massimo rappresentabile nel campo TTL dello header IP). I vettori di distanza vengono scambiati ogni tanto (30 secondi, o quando ci sono novità), quindi è lungo arrivare a 256 aumentando di 1 ogni 30 secondi. Inoltre tutti i router sono convinti di avere un percorso per A, ma si continuano a scambiare i pacchetti tra di loro (loop) ed usano risorse per niente. Questo problema si chiama count to infinity.

Ci siamo inventati delle soluzioni per ovviare a questo problema.



Split Horizon

Un nodo invia al vicino X le informazioni di rotta nella propria routing table relative a destination per cui non è usato X come next hop. C quando parla con B non gli dice niente delle destinazioni in cui usa B, per evitare che B pensi che C sa arrivare verso A.

Non è vero che non do informazioni ad X perchè le conosce già, bensì perchè X non sa che lui è usato come next hop per quelle destinazioni, quindi non deve pensare di poter tornare indietro per raggiungerle in caso avesse perso il cammino.

t_0) C:

A	B	C	D	E
B	B	C	D	D
2	1	0	1	2

C→B

C	D	E
0	1	2

C→D

A	B	C
2	1	0

D→C

D	E
0	1

Poisoned Reverse

Una strategia alternativa, io dico ad un nodo che il cammino che ho verso un informazione è infinito, in modo che quel nodo pensi che io non ho la rotta a quella destinazione e non pensi di poter usare me.

La differenza è che con lo split orizon non ricevendo informazioni di aggiornamento un'informazione di rotta può essere rimossa, mentre con il poisoned

reverse si ricorda che c'è una rotta ma ha costo infinito, quindi si buttano via i pacchetti.

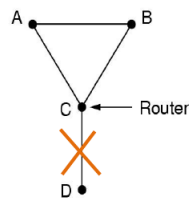
Se si rompe il canale tra A e B, C dice a B che lui ha una rotta che costa infinito (diciamo infinito in modo che ad ogni iterazione di bellman-Ford il nodo può cercare di trovare una rotta che costa meno di infinito) per arrivare ad A, B capisce di avere distanza infinita da A e a propria volta propaga questa informazione. Non funzionano sempre queste strategie, come nel caso in cui la topologia contenga dei loop.

t_0) C:	A	B	C	D	E
	B	B	C	D	D
	2	1	0	1	2
C→B	A	B	C	D	E
	∞	1	0	1	2
C→D	A	B	C	D	E
	2	1	0	1	∞
D→C	A	B	C	D	E
	∞	∞	1	0	1



al passo successivo C ha dist. ∞ da A, ecc...

quando AB cade, B sa di avere dist. ∞ da A e propaga



ma entrambe le strategie possono fallire...

6.6.3 Approccio Link State

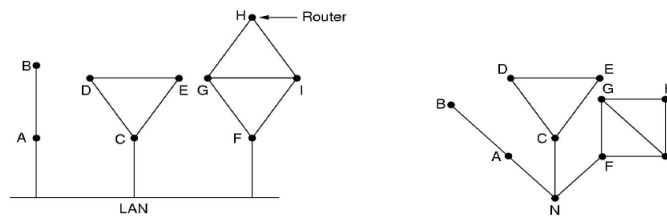
Tutti gli algoritmi che usano l'approccio Distance Vector soffrono del problema di count to infinity. Possono considerare una metrica alla volta, in funzione della metrica può convergere lentamente la banda dei canali, può trovare cammini sub ottimi. Ogni nodo dice ai suoi vicini le informazioni su tutti.

Un'alternativa è usare algoritmi con approccio link state, viene trovato il cammino ottimo e può usare più metriche ed utilizza questo approccio:

- Ogni nodo scopre informazioni sui propri vicini
- Ogni nodo dice a tutti le informazioni sui propri vicini. Mentre distance vector dice ai suoi vicini le informazioni sulle destinazioni che conosce. Ogni nodo sa quali sono i vicini di ogni altro nodo.
- Ogni nodo impara l'intera topologia
- Ogni nodo calcola shortest path a ogni altro, viene calcolato il cammino ottimo, usando Dijkstra

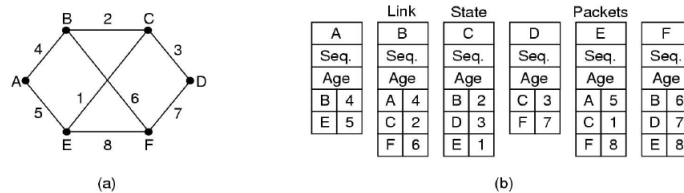
Step 1

Al boot ogni router manda un pacchetto di Hello su ogni linea punto punto con gli altri router indicando il proprio indirizzo di route, chi riceve questo pacchetto risponde con il proprio indirizzo IP (in questo momento ogni router sta imparando gli indirizzi di tutti i suoi vicini), se ci sono più router attaccati alla stessa LAN, quella LAN viene considerata come un nodo perciò ogni nodo scopre i propri vicini. Per completare il primo passo i nodi devono scoprire i costi per arrivare ai nodi vicini, (1 hop) con dei pacchetti di ping (echo ICMP) scoprono delle stime di ritardo per arrivare e tornare da quelli vicini, quindi ho delle metriche tra me e i miei vicini (bit error rate).



Step 2

Ogni nodo dice tutto quello che sa sui suoi vicini a tutti. Un nodo manda ai suoi vicini un pacchetto contenente: l'indirizzo della sorgente (chi è lui), un numero di sequenza (che aggiorna ogni volta che cambia le informazioni sul proprio vicinato, servono per fare sì che un nodo che riceve più pacchetti di link state di un nodo tiene solo le più recenti), un età che dice per quanto tempo mantenere le informazioni, un insieme di coppie identificatore del vicino e metrica di costo con il vicino. Questi pacchetti di link state sono scambiati periodicamente (30 sec) oppure quando cambia qualche cosa, in particolare se si verificano dei guasti, vengono scambiati in flooding, ovvero ogni nodo dice a tutti i suoi vicini quello che sa lui sul suo vicinato, va benissimo perchè il flooding possiamo usarlo anche prima di avere qualsiasi informazione sulla rete e mandiamo le nostre informazioni a tutti, utilizzando tutte le possibili rotte. Questi pacchetti sono scambiati in modo affidabile, ci aspettiamo un ack dal vicino, a sua volta il vicino ridissemina a tutti i suoi vicini il nostro pacchetto di link state e lui si aspetta gli ack, siamo sicuri quindi che tutti i router hanno costruito la stessa topologia e tutti costruiscono le stesse rotte ottime tramite l'algoritmo di Dajikstra.



E' possibile che intanto che questi pacchetti vengano disseminati i nodi abbiano una vista inconsistente ed in base a questa vista calcolino Dajikstra e calcolino delle rotte differenti e la somma di queste rotte differenti trovino dei loop, ma se la topologia resta stabile per un certo tempo i nodi convergono ad una topologia unica ed una visione consistente delle rotte. Ogni nodo ricorda i pacchetti ricevuti e scarta i duplicati e versioni obsolete dei pacchetti. Il numero di sequenza è rappresentato su 32 bit. La age è rappresentata su 16 bit, decrementato per ogni hop mentre i pacchetti vengono diffusi con il flooding, mentre quando un informazione di un pacchetto di link state viene ricordata in un nodo viene decrementata ogni secondo e quando arriva a zero il nodo butta via quel pacchetto. Quindi un informazione può rimanere in un router 2 alla 16 secondi. Questo da anche tempo ad un router di ricevere informazione aggiornata da un altro router senza buttare via l'informazione vecchia che comunque mi serve per instradare.

I nodi mantengono una struttura in cui hanno per ogni sorgente: l'ultimo numero di sequenza ricevuto, la age, i dati sul vicinato e per ogni vicino i flag se devono rimandare il pacchetto di link state a quel vicino ed un insieme di flag per ogni vicino che ricorda se hanno ricevuto il pacchetto di link state e se devono rimandare un ack tra i vicini. I flag ci dicono a chi devo mandare il pacchetto

ed a chi devo mandare gli ack. Per il link state del nodo E l'ultimo numero di seq ricevuto è 21, la age residua è 59 (59 secondi e poi si dimentica i dati associati), deve rimandare al suo vicino C il pacchetto di link state ricevuto per E e mandare degli ack per i pacchetti di E ai vicini A ed F.

Source	Seq.	Age	Send flags			ACK flags			Data
			A	C	F	A	C	F	
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	

6.6.4 Distance Vector vs Link State

caratteristica	DV	LS
o/h msg	$O(n)$	$O(n^2)$
o/h computaz.	$O(n)$	$O(n^2)$
o/h memoria	$O(k)$	$O(kn)$
convergenza	count-to- ∞	oscilla loop temporanei
path	subottimo	ottimo
internet	RIP (intraAS) BGP (interAS)	OSPF (intraAS)

$n = \text{\#router}$; $k = \text{\#vicini per ogni router}$

In DV ogni nodo dice ai suoi vicini tutto quello che sa sulla rete, mentre in LS ogni nodo dice ai suoi vicini tutto quello che sa sui suoi vicini.

Il count to infinity è il problema se un nodo si rompe e gli altri si confondono pensando di poter raggiungere quel nodo passando uno dall'altro, ma in realtà non possono mai raggiungerlo perchè è isolato. E' una conseguenza della visione parziale della topologia della rete.

RIP è usato per gli instradamenti negli Autonomus Sistem, ovvero una rete che fa capo ad una associazione, che è qualcosa di dimensione abbastanza limitata. In Internet è usato BGP tra un AS e l'altro, è un Distance Vector perchè i dispositivi sono tanti e ci va bene un approccio che costa poco. RIP e BGP implementano Distance Vector. Link State può essere usato con qualsiasi metrica, mentre DV solo con il numero di hop, alcune metriche possono provocare oscillazioni nel costo degli archi del grafo (cammini) che aumenta la possibilità

di ricezione dei pacchetti in disordine. Entrambi possono soffrire di loop temporanei a causa di cambiamento della topologia, è più probabile in link state. OSPF implementa Link State.

6.7 Internet Structure

Ci sono delle reti di accesso, tra il pc ed il cabinet in strada, che è il nostro punto di connessione con la rete, oppure aziendale le LAN. Gli internet service provider possono essere regionali, da lì si collegano alle reti nazionali e ci permettono di viaggiare in internet.

Un Autonomous System è una rete sotto il controllo di un unico amministratore.

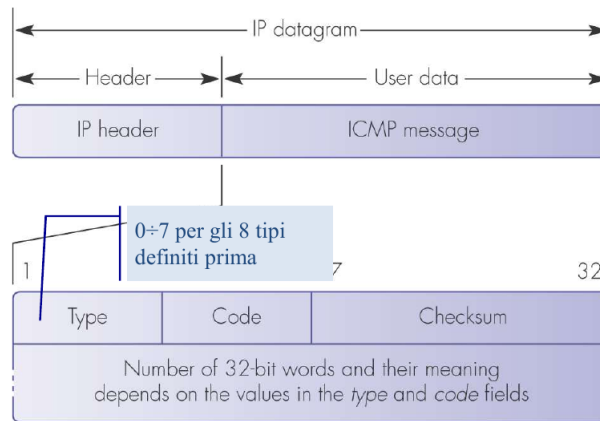
6.8 ICMP - Internet Control Message Protocol

E' un protocollo di livello 3, e lavora di fianco ad IP, esiste da tanto tempo. ICMP genera dei messaggi di controllo della rete che vengono fatti viaggiare. Mette i propri messaggi come payload all'interno di pacchetti IP. Fa girare messaggi:

- notifiche di errore:
 - destinazione irraggiungibile: il pacchetto non è consegnabile perchè la sua destinazione è irraggiungibile, questo è un tipo di pacchetto e poi il payload del messaggio ICMP ci dice qualche cosa di più, può essere causata perchè:
 - * Non esiste un cammino per raggiungere il destinatario
 - * Nello header di IP c'è un campo protocol che dice che la protocol data unit è arrivato all'end system giusto, a questo punto dobbiamo fare demultiplexing per capire a che processo è destinato questo pacchetto, guardando il numero di protocol si rende conto che non ha quel processo.
 - * Arrivati ad un punto per cui la rete deve frammentare il messaggio, il problema è che nello header di IP c'è alzato il bit di don't fragment, quindi dobbiamo buttare via il pacchetto e segnalare alla sorgente di mandarci cose più piccole
 - * Non è permessa comunicazione per ragioni amministrative, spesso negli autonomous system se arriva del traffico di applicazioni che cercano di capire la topologia della nostra rete non facciamo passare quei pacchetti.
 - Time exceeded: il TTL dello header IP è arrivato a 0 (decrementa ad ogni hop) e segnaliamo che il TTL è esaurito
 - Parameter error: c'è qualcosa nello header che non è valido, controllando il checksum dello header IP vediamo che c'è qualcosa che non va

- reachability test:
 - Echo request/reply: è il comando PING, ICMP manda dei pacchetti di echo request all'indirizzo che indichiamo, se i pacchetti arrivano a destinazione, genera una reply e misurando il tempo trascorso possiamo stimare il round trip time. Spesso sono bloccati questi comandi perchè si può specificare la dimensione e la frequenza dei pacchetti che mandiamo, quindi potremmo portare un attacco denial of service a quella destinazione.
- Controllo di congestione:
 - source quench: un router quando riceve tanto traffico può entrare in stato di congestione, se l'occupazione dei buffer supera una soglia il nostro router può mandare a 1 o più dei nostri vicini (1 hop) che ci stanno dando tanto traffico un messaggio ICMP di tipo source quench dicendo di rallentare l'invio. Si può anche rimandare un altro messaggio quando torniamo sotto una soglia.
- Route-change notification:
 - redirect: i nodi conoscono tutta la topologia e si vedono arrivare un pacchetto per una destinazione da un vicino, vede che il suo vicino non avrebbe dovuto mandare a lui il pacchetto, quindi istrada comunque il pacchetto ma rimanda indietro al server che gli ha mandato il pacchetto un pacchetto ICMP redirect dicendogli che non era lui il next hop migliore.
- Performance measuring.
 - timestamp request/reply: si chiede che tutti gli host attraversati da questo pacchetto metta nel payload di ICMP il clock dell'apparato. Questo ha pochissimo significato perchè non c'è nulla che ci dice che gli orologi degli apparati di rete siano sincronizzati.
- Subnet addressing:
 - Address mask request/reply: è il pacchetto scambiato tra due router per dirsi gli indirizzi a cui un router può accedere data una netmask. E' un supporto iniziale alle tabelle di instradamento

6.8.1 Header ICMP

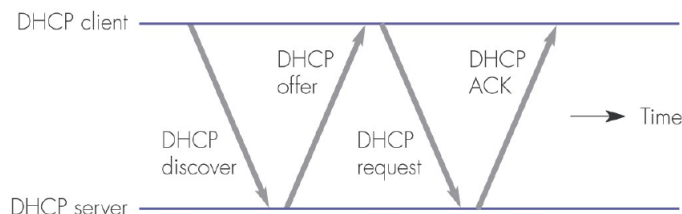
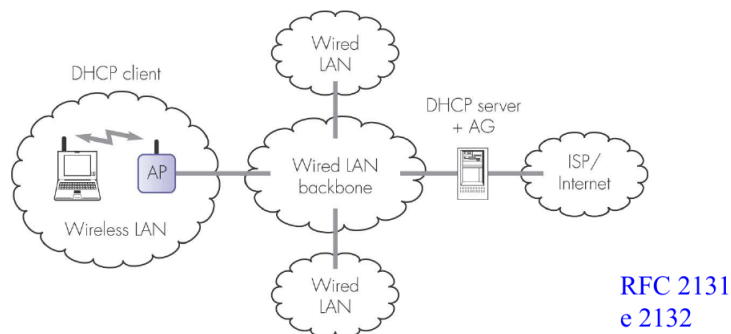


Hanno un tipo (0-7), il codice da delle sottoinformazioni sul sotto tipo, checksum di 16 bit controlla che il contenuto del messaggio ICMP non sia stato alterato durante il passaggio in rete. Poi ci sono un po' di parole a 32 bit che danno un po' di informazioni su quello che sta succedendo.

6.9 DHCP - Dynamic Host Configuration Protocol

Il nostro client che vuole avere un indirizzo di rete, perchè è un apparato mobile generalmente, manda broadcast sulla LAN una richiesta di avere una configurazione di rete, in questo momento il client ha solo come indirizzo il MAC e non ha l'IP e nel suo messaggio indica un numero pseudocasuale transaction ID che gli permette di accoppiare la richiesta con la risposta. Il o i DHCP server che ci sono sulla rete rispondono con un offerta, sempre con la transaction ID, l'IP proposto, la netmask ed una durata. Il client aspetta almeno una risposta, se ne arriva più di una sceglie la migliore, risponde in broadcast con un messaggio DHCP request e per confermare che tutto è andato bene risponde unicast al client con un ack DHCP.

Prima che scada la durata dell'offerta il client può rinnovare l'offerta.



6.10 Routing in Internet

Ci può essere una differenza tra protocollo usato dai router come protocollo usato dentro un Autonomus System (OSPF, perchè ci sono pochi dispositivi) ed il protocollo fuori (BGP, perchè ci sono tanti dispositivi). Sulla zona di confine facciamo una concatenazione di cammini.

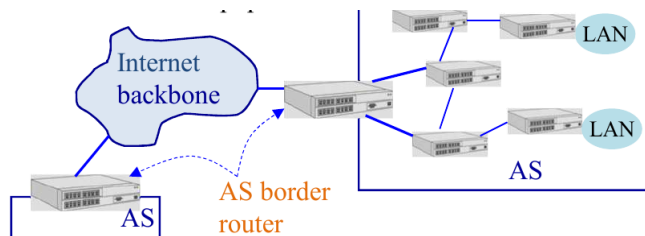
6.10.1 OSPF

E' un algoritmo aperto, usa instradamento dinamico, usa approccio link state e rappresenta la rete come un grafo pesato (può anche usare più di una metrica). Supporta qualità di servizio e può fare load balancing tra cammini.

Struttura rete

La rete si considera interna ad un AS:

- Linee punto punto
- Lan:
 - Di attraversamento: lan broadcast è un punto fittizio con l'esterno
 - Stub: non consentiamo di usare la LAN come attraversamento per andare da un'altra parte

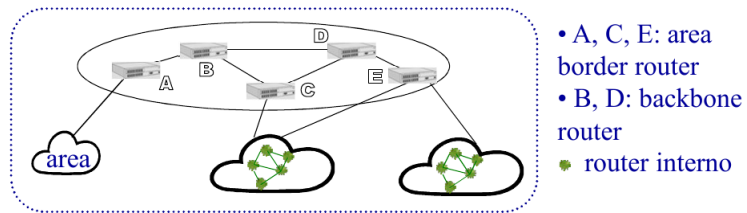


Approccio gerarchico

Una possibilità in un AS non tanto piccolo è di trovare delle aree nella rete:

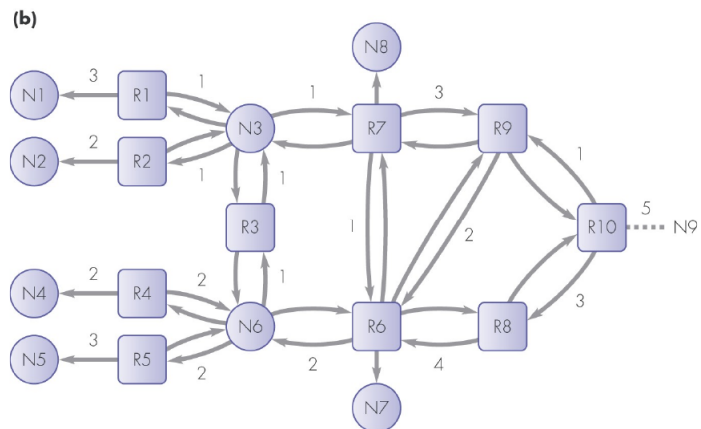
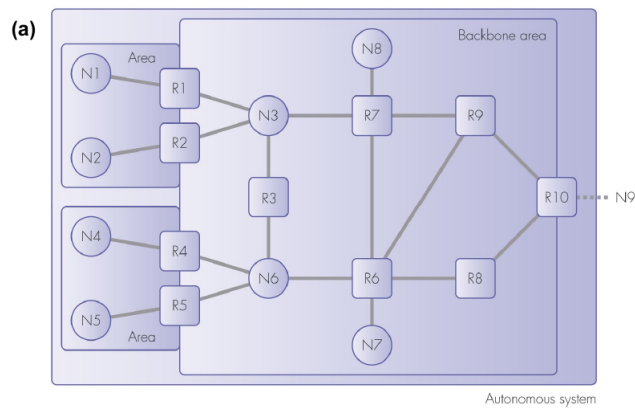
- area di backbone: area dove ci sono router non collegati ad end system
- router interni: appartengono solo ad un area
- area border router: connettono aree diverse
- AS boundary router: router che comunicano con altri AS

Il senso è che OSPF applicato su una rete dove abbiamo individuate delle aree costa meno.



Costruzione grafo

N sono le local area network, R sono i router. Da una LAN per uscire verso un router che lo porta verso il resto del mondo il costo è 0. Negli altri casi gli archi sono bidirezionali e potremmo avere costi diversi per ogni direzione.



Protocollo

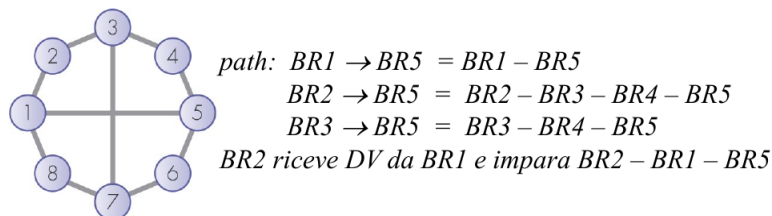
Ogni router nel momento del bootstrap manda un messaggio di hello (e periodicamente) ai suoi vicini dicendo quali sono i suoi vicini (si conoscono due a due), poi devono capire le metriche che ci sono tra di loro, devo usare dei meccanismi ICMP echo. Fatto questo il nodo è pronto per mandare periodicamente (o quando c'è un cambio di topologia importante) un link state update con dentro informazioni su tutti i suoi vicini (il messaggio visto con OSPF) e dato questo messaggio si aspetta un ack dai suoi vicini, i vicini mandano a loro volta il mio messaggio ai loro vicini (tranne me) aspettandosi gli ack.

Il link state req è un messaggio che manda al suo vicino quando ha un sensore che la topologia della rete che ha non è sufficiente o è sbagliata, chiede quindi ai vicini se gli mandano le informazioni di topologia.

Database description è un messaggio in cui mandiamo l'ultimo numero di sequenza posseduto. Per il resto è esattamente l'algoritmo di Dijkstra.

6.11 BGP - Border Gateway Protocol

E' un distance vector, anche in questo caso possiamo dire che una lan può essere stub (ovvero non può essere attraversata da traffico che non è interessata a nessun end system nella LAN). Costa poco ma a differenza di quello che è un distance vector base (quello implementato da RIP), non cade nella trappola di count to infinity, lo fa ricordando i path completi mandando dei messaggi di instradamento più lunghi (cioè in pratica gli mandiamo anche l'ultima colonna della tabella del distance vector in cui mettiamo sia il nodo che raggiungiamo ma anche il cammino con cui lo faccio).



E' un protocollo di livello 3 ma usa i servizi di TCP, passa i propri dati a TCP chiedendogli di mettere in segmenti di TCP con header IP. Fa questa cosa perchè deve essere sicuro che i distance vector arrivino in modo che si mantenga una consistente visione della topologia di rete in Internet

Tipi di messaggi:

- Open: apro una comunicazione con un border router adiacente e sotto questa open a livello 3 c'è una connessione TCP
- Update: manda gli instance vector, possono riguardare soltanto una porzione di cammini oppure tutti, se non ci stanno tutti le info dell'instance vector (mess di TCP) mandiamo gli aggiornamenti periodicamente

- Keepalive: serve per dire all'altra parte che gli mandiamo pochi dati, gli diciamo che sono vivo e non gli mandiamo nessun aggiornamento perchè tanto non è cambiato niente nella visione della topologia. Serve comunque a far viaggiare qualcosa nella connessione TCP che c'è sotto per mantenerla attiva. Viene mandato indietro come ACK anche in risposta ad un messaggio di OPEN
- Notification: non ha un periodo ma quando c'è una condizione strana lo mandiamo

6.12 Summary algoritmi di instradamento

- Host sa come instradare a altri host nella stessa subnet ,a router attaccati alla subnet e a access gateway
- Subnet router: sa come instradare ad altri subnet router nella stessa rete e a access gateway
- access gateway sa come instradare nella propria rete
- Area border router sa come instradare nella propria area e verso la backbone
- backbone router sa come instradare attraverso la backbone agli area border router
- AS boundary router sa come instradare ad altri BR in Internet